

C++ 常用语法

VR+ES+Y:R

写在前面

- 需要更详细地了解 C/C++ 的语法可以点击[这里](#)，本课件大部分内容是从[这里](#)贺的，小部分是[从这里](#)贺的。

写在前面

- 需要更详细地了解 C/C++ 的语法可以点击[这里](#)，本课件大部分内容是从[这里](#)贺的，小部分是[从这里](#)贺的。
- 由于篇幅，这里只简单介绍其中必要的部分。

写在前面

- 需要更详细地了解 C/C++ 的语法可以点击[这里](#)，本课件大部分内容是从[这里](#)贺的，小部分是从[这里](#)贺的。
- 由于篇幅，这里只简单介绍其中必要的部分。
- 由于 OI 中用到的基本上是 C++14 前的语法，因此本课件不涉及 C++17 即以上语法。

写在前面

- 需要更详细地了解 C/C++ 的语法可以点击[这里](#)，本课件大部分内容是从[这里](#)贺的，小部分是从[这里](#)贺的。
- 由于篇幅，这里只简单介绍其中必要的部分。
- 由于 OI 中用到的基本上是 C++14 前的语法，因此本课件不涉及 C++17 即以上语法。
- 使用 C++11/14 语法需要在编译指令中加入 `-std=c++11` 或 `-std=c++14` 。

写在前面

- 需要更详细地了解 C/C++ 的语法可以点击[这里](#)，本课件大部分内容是从[这里](#)贺的，小部分是从[这里](#)贺的。
- 由于篇幅，这里只简单介绍其中必要的部分。
- 由于 OI 中用到的基本上是 C++14 前的语法，因此本课件不涉及 C++17 即以上语法。
- 使用 C++11/14 语法需要在编译指令中加入 `-std=c++11` 或 `-std=c++14` 。
- 部分编译器扩展用途广泛，这里不会涉及，有兴趣的同学可以自行了解。

写在前面

- 需要更详细地了解 C/C++ 的语法可以点击[这里](#)，本课件大部分内容是从[这里](#)贺的，小部分是从[这里](#)贺的。
- 由于篇幅，这里只简单介绍其中必要的部分。
- 由于 OI 中用到的基本上是 C++14 前的语法，因此本课件不涉及 C++17 即以上语法。
- 使用 C++11/14 语法需要在编译指令中加入 `-std=c++11` 或 `-std=c++14` 。
- 部分编译器扩展用途广泛，这里不会涉及，有兴趣的同学可以自行了解。
- 由于讲课人过菜，可（一）能（定）会有疏漏之处，欢迎指出。

写在前面

- 需要更详细地了解 C/C++ 的语法可以点击[这里](#)，本课件大部分内容是从[这里](#)贺的，小部分是[从这里](#)贺的。
- 由于篇幅，这里只简单介绍其中必要的部分。
- 由于 OI 中用到的基本上是 C++14 前的语法，因此本课件不涉及 C++17 即以上语法。
- 使用 C++11/14 语法需要在编译指令中加入 `-std=c++11` 或 `-std=c++14`。
- 部分编译器扩展用途广泛，这里不会涉及，有兴趣的同学可以自行了解。
- 由于讲课人过菜，可（一）能（定）会有疏漏之处，欢迎指出。
- 不要尝试复制本课件中的任何减号，否则后果自负。

C++ 程序结构

- C++ 程序由若干声明组成，声明可以引入实体。

C++ 程序结构

- C++ 程序由若干声明组成，声明可以引入实体。
- 实体包括值、对象、引用、函数、枚举项、类型、类成员、模板、模板特化、命名空间和形参包。

C++ 程序结构

- C++ 程序由若干声明组成，声明可以引入实体。
- 实体包括值、对象、引用、函数、枚举项、类型、类成员、模板、模板特化、命名空间和形参包。
- 其中有一个特殊的函数是主函数，名称为 `main` 。C++ 通过调用主函数来执行程序。

C++ 程序结构

- C++ 程序由若干声明组成，声明可以引入实体。
- 实体包括值、对象、引用、函数、枚举项、类型、类成员、模板、模板特化、命名空间和形参包。
- 其中有一个特殊的函数是主函数，名称为 `main` 。C++ 通过调用主函数来执行程序。
- 标识某个实体的符号称为标识符。一些有特殊意义的标识符称为关键字，用于程序的各个部分。除关键字外的标识符可以作为实体的名字。

C++ 程序结构

- C++ 程序由若干声明组成，声明可以引入实体。
- 实体包括值、对象、引用、函数、枚举项、类型、类成员、模板、模板特化、命名空间和形参包。
- 其中有一个特殊的函数是主函数，名称为 `main` 。C++ 通过调用主函数来执行程序。
- 标识某个实体的符号称为标识符。一些有特殊意义的标识符称为关键字，用于程序的各个部分。除关键字外的标识符可以作为实体的名字。
- 能够定义使用一个实体所需的所有属性的声明是定义，函数的定义由一系列语句组成。

C++ 程序结构

- 语句有若干组成部分，一些部分是表达式。

C++ 程序结构

- 语句有若干组成部分，一些部分是表达式。
- 表达式是由运算符和操作数组合而成的序列，用于指定程序需要进行的操作，最基本的操作数是实体。

C++ 程序结构

- 语句有若干组成部分，一些部分是表达式。
- 表达式是由运算符和操作数组合而成的序列，用于指定程序需要进行的操作，最基本的操作数是实体。
- C++ 中的每一个对象、引用、函数和表达式都会关联一个类型。

C++ 程序结构

- 语句有若干组成部分，一些部分是表达式。
- 表达式是由运算符和操作数组合而成的序列，用于指定程序需要进行的操作，最基本的操作数是实体。
- C++ 中的每一个对象、引用、函数和表达式都会关联一个类型。
- 在程序中添加注释，可以使程序逻辑更清晰方便阅读。使用预处理命令可以更方便地生成代码。

C++ 程序结构

■ 以下是一段求两数之和的代码：

```
1  #include<iostream>// #include 预处理命令
2  #define ll long long// #define 预处理命令
3  ll f(ll a,ll b){//声明并定义函数 f 用于求和
4      return a+b;// return 语句, 表达式 a+b
5  }
6  using namespace std;//using 指令, 是一句声明
7  int main(){//声明并定义主函数 main
8      ll a,b;//声明两个 long long 类型对象 a,b
9      cin>>a>>b;//表达式语句, 用于读入 a,b
10     cout<<f(a,b);//表达式语句, 用于输出
11     return 0;// return 语句
12 }
13 //用到的运算符有: 加号 + , 流运算符 << 和 >> , 函数调用运算符 ()
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 `static_assert`

17 预处理指令

18 常用 C 库

注释

- 注释是对代码的解释和说明，注释不是真正编译、执行的代码。

注释

- 注释是对代码的解释和说明，注释不是真正编译、执行的代码。
- 编译器会忽略注释，将其当作一个空格。

注释

- 注释是对代码的解释和说明，注释不是真正编译、执行的代码。
- 编译器会忽略注释，将其当作一个空格。

- 格式:

```
1  /* 多行
2  注释 */      (1)
3  // 单行注释 (2)
```

注释

- 注释是对代码的解释和说明，注释不是真正编译、执行的代码。
- 编译器会忽略注释，将其当作一个空格。
- 格式：

```
1  /* 多行  
2  注释 */      (1)  
3  // 单行注释 (2)
```

- (1)也称为 C 风格的注释，(2)也称为 C++ 风格的注释。

注释

- 注释是对代码的解释和说明，注释不是真正编译、执行的代码。
- 编译器会忽略注释，将其当作一个空格。
- 格式：

```
1  /* 多行
2  注释 */      (1)
3  // 单行注释 (2)
```

- (1)也称为 C 风格的注释，(2)也称为 C++ 风格的注释。
- 另外还可以使用预处理命令来让代码不被编译、执行，如：

```
1  #if 0
2      被忽略
3      的代码
4  #endif
```


注释

- 注释是对代码的解释和说明，注释不是真正编译、执行的代码。
- 编译器会忽略注释，将其当作一个空格。
- 格式：

```
1 /* 多行
2 注释 */      (1)
3 // 单行注释 (2)
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

标识符

- 标识符是用户编程时使用的名字，用于命名实体。用户自定义标识符必须满足下列条件：

标识符

- 标识符是用户编程时使用的名字，用于命名实体。用户自定义标识符必须满足下列条件：
 - 1 标识符是一个由数字，下划线，大小写拉丁字母和大多数 Unicode 字符组成的任意长度的序列。

标识符

- 标识符是用户编程时使用的名字，用于命名实体。用户自定义标识符必须满足下列条件：
 - 1 标识符是一个由数字，下划线，大小写拉丁字母和大多数 Unicode 字符组成的任意长度的序列。
 - 2 不能把 C++ 关键字作为用户自定义标识符。

标识符

- 标识符是用户编程时使用的名字，用于命名实体。用户自定义标识符必须满足下列条件：
 - 1 标识符是一个由数字，下划线，大小写拉丁字母和大多数 Unicode 字符组成的任意长度的序列。
 - 2 不能把 C++ 关键字作为用户自定义标识符。
 - 3 标识符对大小写敏感。

标识符

- 标识符是用户编程时使用的名字，用于命名实体。用户自定义标识符必须满足下列条件：
 - 1 标识符是一个由数字，下划线，大小写拉丁字母和大多数 Unicode 字符组成的任意长度的序列。
 - 2 不能把 C++ 关键字作为用户自定义标识符。
 - 3 标识符对大小写敏感。
 - 4 标识符首字符必须是非数字字符。

标识符

- 标识符是用户编程时使用的名字，用于命名实体。用户自定义标识符必须满足下列条件：
 - 1 标识符是一个由数字，下划线，大小写拉丁字母和大多数 Unicode 字符组成的任意长度的序列。
 - 2 不能把 C++ 关键字作为用户自定义标识符。
 - 3 标识符对大小写敏感。
 - 4 标识符首字符必须是非数字字符。
 - 5 其中任何位置带有双下划线的标识符，以一个下划线跟着一个大写字母开头的标识符，全局命名空间中以一个下划线开头的标识符是保留标识符。用户不应使用保留标识符，否则行为未定义。

标识符

- 标识符是用户编程时使用的名字，用于命名实体。用户自定义标识符必须满足下列条件：

- 1 标识符是一个由数字，下划线，大小写拉丁字母和大多数 Unicode 字符组成的任意长度的序列。
- 2 不能把 C++ 关键字作为用户自定义标识符。
- 3 标识符对大小写敏感。
- 4 标识符首字符必须是非数字字符。
- 5 其中任何位置带有双下划线的标识符，以一个下划线跟着一个大写字母开头的标识符，全局命名空间中以一个下划线开头的标识符是保留标识符。用户不应使用保留标识符，否则行为未定义。

6

标识符

- 标识符是用户编程时使用的名字，用于命名实体。用户自定义标识符必须满足下列条件：
 - 1 标识符是一个由数字，下划线，大小写拉丁字母和大多数 Unicode 字符组成的任意长度的序列。
 - 2 不能把 C++ 关键字作为用户自定义标识符。
 - 3 标识符对大小写敏感。
 - 4 标识符首字符必须是非数字字符。
 - 5 其中任何位置带有双下划线的标识符，以一个下划线跟着一个大写字母开头的标识符，全局命名空间中以一个下划线开头的标识符是保留标识符。用户不应使用保留标识符，否则行为未定义。
 - 6 。
- 此外，与标准库函数相同的标识符不应在包含标准库的全局命名空间中使用，如 `next` 等。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

关键字

- 关键字是整个语言范围内预先保留的标识符,每个 C++ 关键字都有特殊的含义。

关键字

- 关键字是整个语言范围内预先保留的标识符,每个 C++ 关键字都有特殊的含义。
- 下列标识符均为 C++ 关键字:

```
alignas (C++11 起) alignof (C++11 起) and and_eq asm atomic_cancel (TM TS)
atomic_commit (TM TS) atomic_noexcept (TM TS) auto bitand bitor
bool break case catch char char8_t (C++20 起) char16_t (C++11 起)
char32_t (C++11 起) class compl concept (C++20 起) const consteval (C++20 起)
constexpr (C++11 起) constinit (C++20 起) const_cast continue
co_await (C++20 起) co_return (C++20 起) co_yield (C++20 起)
decltype (C++11 起) default delete do double dynamic_cast else enum explicit
export extern false float for friend goto if inline int long mutable
namespace new noexcept (C++11 起) not not_eq nullptr (C++11 起) operator or
or_eq private protected public reflexpr (反射 TS) register reinterpret_cast
requires (C++20 起) return short signed sizeof static
static_assert (C++11 起) static_cast struct switch synchronized (TM TS)
template this thread_local (C++11 起) throw true try typedef typeid typename
union unsigned using virtual void volatile wchar_t while xor xor_eq
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

声明

- 声明将名字引入（或再引入）到 C++ 程序中。每种实体的声明方式都不同。函数定义，命名空间定义，块声明等都是声明。

声明

- 声明将名字引入（或再引入）到 C++ 程序中。每种实体的声明方式都不同。函数定义，命名空间定义，块声明等都是声明。
- 块声明是能在块作用域中出现的声明。`static_assert` 声明，命名空间别名定义，`using` 指令，简单声明等都是块声明。

声明

- 声明将名字引入（或再引入）到 C++ 程序中。每种实体的声明方式都不同。函数定义，命名空间定义，块声明等都是声明。
- 块声明是能在块作用域中出现的声明。static_assert 声明，命名空间别名定义，using 指令，简单声明等都是块声明。
- 简单声明是引入、创建并可选地初始化一个或数个标识符（典型地为变量）的语句。对象声明，类声明，函数声明（非定义）等都是简单声明。

声明

- 声明将名字引入（或再引入）到 C++ 程序中。每种实体的声明方式都不同。函数定义，命名空间定义，块声明等都是声明。
- 块声明是能在块作用域中出现的声明。static_assert 声明，命名空间别名定义，using 指令，简单声明等都是块声明。
- 简单声明是引入、创建并可选地初始化一个或数个标识符（典型地为变量）的语句。对象声明，类声明，函数声明（非定义）等都是简单声明。
- 简单声明由属性（C++11 起），声明说明符序列，初始化声明符列表三部分组成（如下）。

声明

- 声明将名字引入（或再引入）到 C++ 程序中。每种实体的声明方式都不同。函数定义，命名空间定义，块声明等都是声明。
- 块声明是能在块作用域中出现的声明。static_assert 声明，命名空间别名定义，using 指令，简单声明等都是块声明。
- 简单声明是引入、创建并可选地初始化一个或数个标识符（典型地为变量）的语句。对象声明，类声明，函数声明（非定义）等都是简单声明。
- 简单声明由属性（C++11 起），声明说明符序列，初始化声明符列表三部分组成（如下）。
- 属性 声明说明符序列 初始化声明符列表；

声明

- 声明将名字引入（或再引入）到 C++ 程序中。每种实体的声明方式都不同。函数定义，命名空间定义，块声明等都是声明。
- 块声明是能在块作用域中出现的声明。static_assert 声明，命名空间别名定义，using 指令，简单声明等都是块声明。
- 简单声明是引入、创建并可选地初始化一个或数个标识符（典型地为变量）的语句。对象声明，类声明，函数声明（非定义）等都是简单声明。
- 简单声明由属性（C++11 起），声明说明符序列，初始化声明符列表三部分组成（如下）。
- 属性 声明说明符序列 初始化声明符列表；
- 定义是完全定义了声明中所引入的实体的声明，除了一些特殊情况外（如下文的 f ）的简单声明都是定义。

声明

■ 下面列出几个有效的简单声明:

```
1  int a,b=1,&c=a;  
2  //声明两个 int 类型的变量 a,b , 初始化 b 为 1  
3  //同时声明一个 int 类型引用 c , 绑定到变量 a  
4  const char*s="1234";  
5  //声明一个 char 类型的指针 s , 其指向内容不可修改, 初  
   始化为字符串 "1234" 的首地址  
6  int f(int),(*g)(int)=f;  
7  //声明一个形参与返回值均为 int 的函数 f, 以及一个同类  
   型的函数指针 g 初始化为 f
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

对象

- 对象是一段存储空间，C++ 程序可以创建、销毁、引用、访问并操作对象。

对象

- 对象是一段存储空间，C++ 程序可以创建、销毁、引用、访问并操作对象。
- C++ 中对象拥有这些性质：

对象

- 对象是一段存储空间，C++ 程序可以创建、销毁、引用、访问并操作对象。
- C++ 中对象拥有这些性质：

1 大小

对象

- 对象是一段存储空间，C++ 程序可以创建、销毁、引用、访问并操作对象。
- C++ 中对象拥有这些性质：
 - 1 大小
 - 2 对齐要求

对象

- 对象是一段存储空间，C++ 程序可以创建、销毁、引用、访问并操作对象。
- C++ 中对象拥有这些性质：
 - 1 大小
 - 2 对齐要求
 - 3 存储期

对象

- 对象是一段存储空间，C++ 程序可以创建、销毁、引用、访问并操作对象。
- C++ 中对象拥有这些性质：
 - 1 大小
 - 2 对齐要求
 - 3 存储期
 - 4 生存期

对象

- 对象是一段存储空间，C++ 程序可以创建、销毁、引用、访问并操作对象。
- C++ 中对象拥有这些性质：
 - 1 大小
 - 2 对齐要求
 - 3 存储期
 - 4 生存期
 - 5 类型

对象

- 对象是一段存储空间，C++ 程序可以创建、销毁、引用、访问并操作对象。
- C++ 中对象拥有这些性质：
 - 1 大小
 - 2 对齐要求
 - 3 存储期
 - 4 生存期
 - 5 类型
 - 6 值

对象

- 对象是一段存储空间，C++ 程序可以创建、销毁、引用、访问并操作对象。
- C++ 中对象拥有这些性质：
 - 1 大小
 - 2 对齐要求
 - 3 存储期
 - 4 生存期
 - 5 类型
 - 6 值
 - 7 可以有或没有名字，有名字的对象称为变量

对象

- 对象是一段存储空间，C++ 程序可以创建、销毁、引用、访问并操作对象。
- C++ 中对象拥有这些性质：
 - 1 大小
 - 2 对齐要求
 - 3 存储期
 - 4 生存期
 - 5 类型
 - 6 值
 - 7 可以有或没有名字，有名字的对象称为变量
- 对象能由定义、new 表达式、throw 表达式、更改联合体的活跃成员和求值要求临时对象的表达式显式创建，也可以在 unsigned char 或 char 数组中隐式创建。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

引用

- 引用是既存对象或函数的别名。一旦把引用初始化为某个对象，就可以使用该引用名称来指代所绑定的对象。

引用

- 引用是既存对象或函数的别名。一旦把引用初始化为某个对象，就可以使用该引用名称来指代所绑定的对象。
- 定义引用时，必须初始化为指代一个有效的对象或函数。引用一旦绑定到对象，此后不可重新绑定到另一个对象，因此引用不能在顶层加 `cv` 限定符。

引用

- 引用是既存对象或函数的别名。一旦把引用初始化为某个对象，就可以使用该引用名称来指代所绑定的对象。
- 定义引用时，必须初始化为指代一个有效的对象或函数。引用一旦绑定到对象，此后不可重新绑定到另一个对象，因此引用不能在顶层加 `cv` 限定符。
- 不存在 `void` 的引用。引用不是对象或函数，因此不存在引用的引用，不存在引用的数组，不存在指向引用的指针。

引用

- 引用是既存对象或函数的别名。一旦把引用初始化为某个对象，就可以使用该引用名称来指代所绑定的对象。
- 定义引用时，必须初始化为指代一个有效的对象或函数。引用一旦绑定到对象，此后不可重新绑定到另一个对象，因此引用不能在顶层加 `cv` 限定符。
- 不存在 `void` 的引用。引用不是对象或函数，因此不存在引用的引用，不存在引用的数组，不存在指向引用的指针。
- 引用可以作为函数的形式参数，以便在函数内部修改传入的实际参数。这种方法称为将参数按引用传递。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

变量

- 变量是一段命名的存储空间，是有名字的对象（或其引用）。

变量

- 变量是一段命名的存储空间，是有名字的对象（或其引用）。
- 变量需要通过声明来创建，格式为：

1 类型说明符 初始化声明符列表；

变量

- 变量是一段命名的存储空间，是有名字的对象（或其引用）。
- 变量需要通过声明来创建，格式为：

1 类型说明符 初始化声明符列表；

- 可以在前面加上存储类说明符与 `constexpr` 说明符，其与类型说明符的顺序任意。

变量

- 变量是一段命名的存储空间，是有名字的对象（或其引用）。
- 变量需要通过声明来创建，格式为：

1 类型说明符 初始化声明符列表；

- 可以在前面加上存储类说明符与 `constexpr` 说明符，其与类型说明符的顺序任意。
- 类型说明符是指名一个类型的说明符的序列，声明所引入的每个实体均为此类型。

变量

- 变量是一段命名的存储空间，是有名字的对象（或其引用）。
- 变量需要通过声明来创建，格式为：

1 类型说明符 初始化声明符列表；

- 可以在前面加上存储类说明符与 `constexpr` 说明符，其与类型说明符的顺序任意。
- 类型说明符是指名一个类型的说明符的序列，声明所引入的每个实体均为此类型。
- 初始化声明符列表是一或多个初始化声明符的逗号分隔列表。最简单的初始化声明符由标识符和可选的初始化器组成。

变量

- 根据变量的出现位置，可以将变量分为成员变量，形式参数，（一般的）全局变量，（一般的）局部变量。

变量

- 根据变量的出现位置，可以将变量分为成员变量，形式参数，（一般的）全局变量，（一般的）局部变量。
- 前两个稍后会进行说明，这里先说一下后两个。

变量

- 根据变量的出现位置，可以将变量分为成员变量，形式参数，（一般的）全局变量，（一般的）局部变量。
- 前两个稍后会进行说明，这里先说一下后两个。
- 全局变量是不属于任何函数的变量，定义于所有函数的外部，在所有函数执行前初始化，在定义时若不指定初始化值，则初始化为默认值（0）。所有函数均可访问声明在其上的全局变量。全局变量存储于内存中的静态变量区，在程序结束时被销毁，结束生存期。

变量

- 根据变量的出现位置，可以将变量分为成员变量，形式参数，（一般的）全局变量，（一般的）局部变量。
- 前两个稍后会进行说明，这里先说一下后两个。
- 全局变量是不属于任何函数的变量，定义于所有函数的外部，在所有函数执行前初始化，在定义时若不指定初始化值，则初始化为默认值（0）。所有函数均可访问声明在其上的全局变量。全局变量存储于内存中的静态变量区，在程序结束时被销毁，结束生存期。
- 局部变量定义在函数或代码块内部，在其声明到函数或代码块结尾前可以使用。可以分为自动存储期局部变量与静态局部变量两类。

变量

- 自动存储期局部变量在声明时不加存储类说明符 `static` 。自动存储期局部变量在每次访问到其定义时初始化,在定义时若不指定初始化值,则无法预测其值为多少,使用其值将导致未定义行为(Undefined Behavior,UB)。自动存储期局部变量存储于内存中的栈空间,在退出作用域时被销毁。

变量

- 自动存储期局部变量在声明时不加存储类说明符 `static` 。自动存储期局部变量在每次访问到其定义时初始化,在定义时若不指定初始化值,则无法预测其值为多少,使用其值将导致未定义行为(Undefined Behavior, UB)。自动存储期局部变量存储于内存中的栈空间,在退出作用域时被销毁。
- 静态局部变量在声明时加上存储类说明符 `static` 。静态局部变量在第一次访问到其定义时初始化,此后不再重新初始化。在定义时若不指定初始化值,则初始化为默认值 (0)。类似于全局变量,静态局部变量存储于内存中的静态变量区,在程序结束时被销毁。

变量

- 自动存储期局部变量在声明时不加存储类说明符 `static` 。自动存储期局部变量在每次访问到其定义时初始化,在定义时若不指定初始化值,则无法预测其值为多少,使用其值将导致未定义行为(Undefined Behavior,UB)。自动存储期局部变量存储于内存中的栈空间,在退出作用域时被销毁。
- 静态局部变量在声明时加上存储类说明符 `static` 。静态局部变量在第一次访问到其定义时初始化,此后不再重新初始化。在定义时若不指定初始化值,则初始化为默认值 (0)。类似于全局变量,静态局部变量存储于内存中的静态变量区,在程序结束时被销毁。
- 若在自动存储期局部变量声明时加上存储类说明符 `register` 则可建议编译器将其存储于寄存器中,因此这种变量没有地址。C++11 弃用该关键字, C++17 移除该关键字。

变量

- 自动存储期局部变量在声明时不加存储类说明符 `static` 。自动存储期局部变量在每次访问到其定义时初始化,在定义时若不指定初始化值,则无法预测其值为多少,使用其值将导致未定义行为(Undefined Behavior,UB)。自动存储期局部变量存储于内存中的栈空间,在退出作用域时被销毁。
- 静态局部变量在声明时加上存储类说明符 `static` 。静态局部变量在第一次访问到其定义时初始化,此后不再重新初始化。在定义时若不指定初始化值,则初始化为默认值 (0)。类似于全局变量,静态局部变量存储于内存中的静态变量区,在程序结束时被销毁。
- 若在自动存储期局部变量声明时加上存储类说明符 `register` 则可建议编译器将其存储于寄存器中,因此这种变量没有地址。C++11 弃用该关键字, C++17 移除该关键字。
- 注意一个变量声明中只能使用一个存储类说明符。

变量

- 自动存储期局部变量在声明时不加存储类说明符 `static` 。自动存储期局部变量在每次访问到其定义时初始化,在定义时若不指定初始化值,则无法预测其值为多少,使用其值将导致未定义行为(Undefined Behavior, UB)。自动存储期局部变量存储于内存中的栈空间,在退出作用域时被销毁。
- 静态局部变量在声明时加上存储类说明符 `static` 。静态局部变量在第一次访问到其定义时初始化,此后不再重新初始化。在定义时若不指定初始化值,则初始化为默认值 (0)。类似于全局变量,静态局部变量存储于内存中的静态变量区,在程序结束时被销毁。
- 若在自动存储期局部变量声明时加上存储类说明符 `register` 则可建议编译器将其存储于寄存器中,因此这种变量没有地址。C++11 弃用该关键字, C++17 移除该关键字。
- 注意一个变量声明中只能使用一个存储类说明符。
- 若在变量声明时加上说明符 `constexpr` 可以指定变量的值可以在常量表达式中出现,它必须立即初始化为常量表达式。

变量

■ 示例:

```
1  int a,b=2;//int 类型全局变量 a=0,b=2
2  int main(){
3      {
4          long long a=1,b;
5          //long long 类型自动存储期局部变量 a=1 , b 未初始化
6          //局部变量会覆盖外部的同名变量, 要使用全局变量 a, b 使用 ::a,::b
7          int static c,d(-1);//int 类型静态局部变量 c=0,d=-1
8      }//这里出了代码块后局部变量 a,b,c,d 均不可用, 全局变量 a,b 可用
9      //局部变量 a,b 被销毁, c,d 需要等到程序结束时才销毁
10     register char*i;//char 指针类型寄存器变量 i
11     constexpr int num=2*2+1;//int 类型局部常量 num=5
12 }
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

字面量

- 字面量是 C++ 程序中用以表现嵌入到源代码中的常量值的记号，可以直接使用。

字面量

- 字面量是 C++ 程序中用以表现嵌入到源代码中的常量值的记号，可以直接使用。
- C++ 中字面量包括以下几种：

字面量

- 字面量是 C++ 程序中用以表现嵌入到源代码中的常量值的记号，可以直接使用。
- C++ 中字面量包括以下几种：
 - 1 布尔类型字面量：true, false 分别表示真, 假, 其转化为数值分别是 1, 0 。

字面量

- 字面量是 C++ 程序中用以表现嵌入到源代码中的常量值的记号，可以直接使用。
- C++ 中字面量包括以下几种：
 - 1 布尔类型字面量：true, false 分别表示真,假,其转化为数值分别是 1,0 。
 - 2 指针字面量：nullptr 是，指定一个空指针值(C++11 起)

字面量

- 字面量是 C++ 程序中用以表现嵌入到源代码中的常量值的记号，可以直接使用。
- C++ 中字面量包括以下几种：
 - 1 布尔类型字面量：true,false 分别表示真,假,其转化为数值分别是 1,0 。
 - 2 指针字面量：nullptr 是, 指定一个空指针值(C++11 起)
 - 3 整数字面量：以 0,0x (大小写均可,下同),0b (C++14 起或 gnu 扩展)或空为前缀,分别表示 8,16,2,10 进制,后接其进制定范围内的数字序列(16 进制可使用 a f , C++14 起序列中可使用单引号隔开)。
可加后缀表明数的类型,后缀 u,l,ll,ul/lu,ull/llu 分别代表 unsigned int,long,long long,unsigned long,unsigned long long 类型。不加则由编译器判断。

字面量

- 4 浮点字面量：可以直接用带小数点的表示方法，小数点前后可以没有数字。也可以以科学计数法表示，用 e 后随数字来表示指数。

可加后缀表明数的类型，f/l/无后缀分别表示 float/long double/double。

字面量

- 4 浮点字面量：可以直接用带小数点的表示方法，小数点前后可以没有数字。也可以以科学计数法表示，用 e 后随数字来表示指数。

可加后缀表明数的类型，f/l/无后缀分别表示 float/long double/double。

- 5 字符字面量：由单引号引起的 c 字符，用前缀大写 L，小写 u (C++11 起)，大写 U (C++11 起)，无前缀表示 wchar_t, char16_t, char32_t, char 类型。

其中 c 字符指转义序列，或者源字符集除去单引号 (')、反斜杠 (\) 和换行符后的字符。

转义序列分为简单转义序列

(\', \", \?, \\, \a, \b, \f, \n, \r, \t, \v) 和数值转义序列（以反斜杠开头的 8 进制序列，长度不超过 3，或者以反斜杠加字符 x 开头的 16 进制序列，长度不限。代表的字符为 ASCII 码中所对应的字符）以及通用字符名。

L 前缀和无前缀可以使用多个 c 字符，类型分别为 wchar_t, int。

字面量

- 6** 字符串字面量：由双引号引起的 `s` 字符序列，用前缀大写 `L`，小写 `u8` (C++11 起)，小写 `u` (C++11 起)，大写 `U` (C++11 起)，无前缀表示 `const wchar_t[N]`, `const char[N]`, `const char16_t[N]`, `const char32_t[N]`, `const char[N]` 类型，其中 `N` 是该字符串的大小（包含空终止符）。

其中 `s` 字符指转义序列，或者源字符集除去双引号 (")、反斜杠 (\) 和换行符后的字符。

C++11 起，支持原生字符串，格式为 `R"(d(r)d)"`。其中 `d` 表示同一个 `d` 字符（基础源字符集除去括号、反斜杠和空格后的字符）序列，最多 16 个字符，可以为空。`r` 表示 `r` 字符（源字符集字符）序列，不得包含结束符 `d`。

字面量

- 6** 字符串字面量：由双引号引起的 `s` 字符序列，用前缀大写 `L`，小写 `u8` (C++11 起)，小写 `u` (C++11 起)，大写 `U` (C++11 起)，无前缀表示 `const wchar_t[N]`, `const char[N]`, `const char16_t[N]`, `const char32_t[N]`, `const char[N]` 类型，其中 `N` 是该字符串的大小（包含空终止符）。

其中 `s` 字符指转义序列，或者源字符集除去双引号 (")、反斜杠 (\) 和换行符后的字符。

C++11 起，支持原生字符串，格式为 `R"(d(r)d)"`。其中 `d` 表示同一个 `d` 字符（基础源字符集除去括号、反斜杠和空格后的字符）序列，最多 16 个字符，可以为空。`r` 表示 `r` 字符（源字符集字符）序列，不得包含结束符 `d`。

- 7** 用户定义字面量：用户指定的类型的常量值(C++11 起)

字面量

■ 示例:

```
1  bool flag1=true,flag2=false;
2  //flag1 为真, flag2 为假
3  std::nullptr_t ptr=nullptr;//s 是空指针
4  int a=1,b=01234567,c=0b0001010,d=0x3f3f3f3f;
5  //abcd 分别为 1 342391 10 1061109567
6  double e=1e10,f=1.,g=.21,h=1.7e-7;float pi=3.14f;
7  //efgh 分别为 1e10 ,1.0,0.21,1.7e-7
8  char l='\n',s='',t='\\',A='\x000041',B='\102';
9  //l,s,t,A,B 代表的字符为换行,单引号,反斜杠,A,B
10 char str1[]="w\x63\162\x0000061k\""?\\'\n";
11 char str2[]=R"%(wcrak"?\'
12 )%";
13 char str3[]={119,99,114,97,107,34,63,92,39,10,0};
14 //str1,str2,str3 是同一个字符串
```


目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

- 基础类型
- 引用类型
- 数组类型
- 函数类型
- 枚举类型
- 类类型
- 指针类型
- 成员指针类型
- cv 限定符
- typedef
- auto 和

decltype

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

类型

- C++ 中每一个对象、引用、函数（包括函数模板特化）和表达式具有称为类型的性质，它限制了对这些实体所容许的操作。

类型

- C++ 中每一个对象、引用、函数（包括函数模板特化）和表达式具有称为类型的性质，它限制了对这些实体所容许的操作。
- C++ 类型系统由以下类型组成：

类型

- C++ 中每一个对象、引用、函数（包括函数模板特化）和表达式具有称为类型的性质，它限制了对这些实体所容许的操作。
- C++ 类型系统由以下类型组成：
 - 1 基础类型：包括 `void` 类型，`std::nullptr_t` 类型(C++11起)和算术类型

类型

- C++ 中每一个对象、引用、函数（包括函数模板特化）和表达式具有称为类型的性质，它限制了对这些实体所容许的操作。
- C++ 类型系统由以下类型组成：
 - 1 基础类型：包括 `void` 类型，`std::nullptr_t` 类型(C++11起)和算术类型
 - 2 复合类型：包括引用类型，指针类型，成员指针类型，数组类型，函数类型，枚举类型和类类型。

类型

- C++ 中每一个对象、引用、函数（包括函数模板特化）和表达式具有称为类型的性质，它限制了对这些实体所容许的操作。
- C++ 类型系统由以下类型组成：
 - 1 基础类型：包括 `void` 类型，`std::nullptr_t` 类型(C++11起)和算术类型
 - 2 复合类型：包括引用类型，指针类型，成员指针类型，数组类型，函数类型，枚举类型和类类型。
 - 3 对于除引用和函数以外的每个类型，类型系统还支持该类型的三个附加 `cv` 限定版本。

类型

- C++ 中每一个对象、引用、函数（包括函数模板特化）和表达式具有称为类型的性质，它限制了对这些实体所容许的操作。
- C++ 类型系统由以下类型组成：
 - 1 基础类型：包括 `void` 类型，`std::nullptr_t` 类型(C++11起)和算术类型
 - 2 复合类型：包括引用类型，指针类型，成员指针类型，数组类型，函数类型，枚举类型和类类型。
 - 3 对于除引用和函数以外的每个类型，类型系统还支持该类型的三个附加 `cv` 限定版本。
- 对象类型是非函数、引用、`void` 类型的（可有 `cv` 限定的）类型。

类型

- C++ 中每一个对象、引用、函数（包括函数模板特化）和表达式具有称为类型的性质，它限制了对这些实体所容许的操作。
- C++ 类型系统由以下类型组成：
 - 1 基础类型：包括 `void` 类型，`std::nullptr_t` 类型(C++11 起)和算术类型
 - 2 复合类型：包括引用类型，指针类型，成员指针类型，数组类型，函数类型，枚举类型和类类型。
 - 3 对于除引用和函数以外的每个类型，类型系统还支持该类型的三个附加 `cv` 限定版本。
- 对象类型是非函数、引用、`void` 类型的（可有 `cv` 限定的）类型。
- 标量类型是（可有 `cv` 限定的）算术、指针、成员指针、枚举和 `std::nullptr_t` (C++11 起) 类型。

类型

- C++ 中每一个对象、引用、函数（包括函数模板特化）和表达式具有称为类型的性质，它限制了对这些实体所容许的操作。
- C++ 类型系统由以下类型组成：
 - 1 基础类型：包括 `void` 类型，`std::nullptr_t` 类型(C++11 起)和算术类型
 - 2 复合类型：包括引用类型，指针类型，成员指针类型，数组类型，函数类型，枚举类型和类类型。
 - 3 对于除引用和函数以外的每个类型，类型系统还支持该类型的三个附加 `cv` 限定版本。
- 对象类型是非函数、引用、`void` 类型的（可有 `cv` 限定的）类型。
- 标量类型是（可有 `cv` 限定的）算术、指针、成员指针、枚举和 `std::nullptr_t` (C++11 起) 类型。
- 不完整类型是（可有 `cv` 限定的）`void` 类型、已声明（例如由前置声明）但未定义的类型、未知边界数组、不完整类型元素的数组和从其声明点到确定其底层类型之间的枚举类型。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

- 基础类型

- 引用类型

- 数组类型

- 函数类型

- 枚举类型

- 类类型

- 指针类型

- 成员指针类型

- cv 限定符

- typedef

- auto 和

decltype

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

基础类型

- `void` : 值为空的类型，它是无法变为完整的不完整类型（因此不存在 `void` 类型的对象）。不存在含有 `void` 的数组以及到 `void` 的引用

基础类型

- `void` : 值为空的类型，它是无法变为完整的不完整类型（因此不存在 `void` 类型的对象）。不存在含有 `void` 的数组以及到 `void` 的引用
- `std::nullptr_t` (C++11 起): 空指针字面量 `nullptr` 的类型。它是独立类型，既非指针类型，亦非成员指针类型。

基础类型

- `void` : 值为空的类型，它是无法变为完整的不完整类型（因此不存在 `void` 类型的对象）。不存在含有 `void` 的数组以及到 `void` 的引用
- `std::nullptr_t` (C++11 起): 空指针字面量 `nullptr` 的类型。它是独立类型，既非指针类型，亦非成员指针类型。
- `bool` : 足以存放两个值 `true` 或 `false` 之一的类型。

基础类型

- `void` : 值为空的类型，它是无法变为完整的不完整类型（因此不存在 `void` 类型的对象）。不存在含有 `void` 的数组以及到 `void` 的引用
- `std::nullptr_t` (C++11 起): 空指针字面量 `nullptr` 的类型。它是独立类型，既非指针类型，亦非成员指针类型。
- `bool` : 足以存放两个值 `true` 或 `false` 之一的类型。
- 整数类型: 基本整数类型 `int` , 可加修饰符 `signed, unsigned, short, long` 表示有符号（默认），无符号，短（为空间优化，至少 16 位），长（至少 32 位）。若加其他修饰符，则可省略修饰符 `int`。 `long` 可以使用两次，表示至少 64 位（C++11 起或 `gnu` 扩展）。修饰符的顺序可以是任何顺序。

基础类型

- 字符类型: `signed char` 有符号字符类型, `unsigned char` 无符号字符类型, `char` 能在目标系统上最有效地处理的字符表示的类型, `wchar_t` 宽字符类型 (至少 16 位), `char16_t` UTF-16 字符表示的类型 (C++11 起), `char32_t` UTF-32 字符表示的类型 (C++11 起)。

C++ 标准保证 $1 = \text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$

基础类型

- 字符类型: `signed char` 有符号字符类型, `unsigned char` 无符号字符类型, `char` 能在目标系统上最有效地处理的字符表示的类型, `wchar_t` 宽字符类型 (至少 16 位), `char16_t` UTF-16 字符表示的类型 (C++11 起), `char32_t` UTF-32 字符表示的类型 (C++11 起)。

C++ 标准保证 $1 = \text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$

- 浮点类型: `float` 单精度浮点类型, `double` 双精度浮点类型, `long double` 扩展精度浮点类型。
浮点类型可以支持一些特殊值无穷 (`INFINITY`, `-INFINITY`), 负零 (`-0.0`), 非数(`NaN`)。

基础类型

- 字符类型: signed char 有符号字符类型, unsigned char 无符号字符类型, char 能在目标系统上最有效地处理的字符表示的类型, wchar_t 宽字符类型 (至少 16 位), char16_t UTF-16 字符表示的类型 (C++11 起), char32_t UTF-32 字符表示的类型 (C++11 起)。

C++ 标准保证 $1 = \text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$

- 浮点类型: float 单精度浮点类型, double 双精度浮点类型, long double 扩展精度浮点类型。

浮点类型可以支持一些特殊值无穷 (INFINITY, -INFINITY), 负零 (-0.0), 非数(NaN)。

- 算术类型可以相互转换, 也可以显式转换为空类型。算术类型可以进行算术运算, 运算前对 (非浮点类型) 操作数实施整型提升, 对于 (除移位之外的) 二元运算符, 若提升后操作类型不同, 则转换为两者中范围更广的类型。

基础类型

■ 示例:

```
1 //void z;//无法定义 void 类型对象
2 std::nullptr_t p;
3 bool flag;
4 int a;signed b;signed int c;int signed d;
5 short e;unsigned int short f;unsigned g;
6 long signed h;int long unsigned i;
7 long long j;unsigned long int long k;
8 //abcd 同类型,d~k 为不同的整数类型
9 char c1;char unsigned c2;
10 signed char c3;wchar_t c4;
11 float f1;double f2;long double f3;
```

目录

1	注释	8	字面量	11	语句
2	标识符	9	类型	12	函数
3	关键字		■ 基础类型	13	模板
4	声明		■ 引用类型	14	命名空间
5	对象		■ 数组类型	15	属性
6	引用		■ 函数类型	16	static_assert
7	变量		■ 枚举类型	17	预处理指令
			■ 类类型	18	常用 C 库
			■ 指针类型		
			■ 成员指针类型		
			■ cv 限定符		
			■ typedef		
			■ auto 和		
			decltype		
		10	表达式		

引用类型

- 引用类型分为左值引用或右值引用（C++11 起）。

引用类型

- 引用类型分为左值引用或右值引用（C++11 起）。
- 左值引用可以绑定到左值，右值引用可以绑定到右值。特殊的，拥有 `const`（只读）限定修饰符的左值引用可以绑定到左值和右值，但无法对绑定到的对象进行修改。

引用类型

- 引用类型分为左值引用或右值引用（C++11 起）。
- 左值引用可以绑定到左值，右值引用可以绑定到右值。特殊的，拥有 `const`（只读）限定修饰符的左值引用可以绑定到左值和右值，但无法对绑定到的对象进行修改。
- 引用变量声明是声明符拥有下列形式的简单声明（属性先忽略）：

```
1      & 属性 声明符 //左值引用  
2      && 属性 声明符 //右值引用
```

引用类型

- 引用类型分为左值引用或右值引用（C++11 起）。
- 左值引用可以绑定到左值，右值引用可以绑定到右值。特殊的，拥有 `const`（只读）限定修饰符的左值引用可以绑定到左值和右值，但无法对绑定到的对象进行修改。
- 引用变量声明是声明符拥有下列形式的简单声明（属性先忽略）：

```
1      & 属性 声明符 //左值引用
2      && 属性 声明符 //右值引用
```
- 除作为函数形参声明，函数返回类型声明，类成员声明，以及带 `extern` 说明符外，引用声明必须和初始化器一起出现。

引用类型

- 引用类型分为左值引用或右值引用（C++11 起）。
- 左值引用可以绑定到左值，右值引用可以绑定到右值。特殊的，拥有 `const`（只读）限定修饰符的左值引用可以绑定到左值和右值，但无法对绑定到的对象进行修改。
- 引用变量声明是声明符拥有下列形式的简单声明（属性先忽略）：

```
1      & 属性 声明符 //左值引用
2      && 属性 声明符 //右值引用
```

- 除作为函数形参声明，函数返回类型声明，类成员声明，以及带 `extern` 说明符外，引用声明必须和初始化器一起出现。
- 无法定义引用的引用，但是通过模板或 `typedef` 中的类型操作可以构成引用的引用，此时适用引用折叠（reference collapsing）规则：右值引用的右值引用折叠成右值引用，所有其他组合均折叠成左值引用。

引用类型

- 引用类型分为左值引用或右值引用（C++11 起）。
- 左值引用可以绑定到左值，右值引用可以绑定到右值。特殊的，拥有 `const`（只读）限定修饰符的左值引用可以绑定到左值和右值，但无法对绑定到的对象进行修改。
- 引用变量声明是声明符拥有下列形式的简单声明（属性先忽略）：

```
1      & 属性 声明符 //左值引用
2      && 属性 声明符 //右值引用
```

- 除作为函数形参声明，函数返回类型声明，类成员声明，以及带 `extern` 说明符外，引用声明必须和初始化器一起出现。
- 无法定义引用的引用，但是通过模板或 `typedef` 中的类型操作可以构成引用的引用，此时适用引用折叠（reference collapsing）规则：右值引用的右值引用折叠成右值引用，所有其他组合均折叠成左值引用。
- 除用作函数形式参数和返回值外，右值引用和只读左值引用可以用于为临时对象延长生存期，以减少不必要的复制操作。

引用类型

- 绑定到的对象生存期结束后，引用有可能仍保持可访问。此时将会产生悬垂引用，访问这种引用是未定义行为。

引用类型

- 绑定到的对象生存期结束后，引用有可能仍保持可访问。此时将会产生悬垂引用，访问这种引用是未定义行为。

- 示例：

```
1 void inc(int&x){x++;}  
2 //引用作为函数形式参数, 对实际参数自增  
3 const int&f(){return 1;}  
4 //返回临时对象 1 的悬垂引用  
5 void g(){  
6     int a=114,&b=a,&&c=514;  
7     // b 是非只读左值引用, 绑定到左值 a  
8     // c 是右值引用, 绑定到右值 514  
9     const int&d=a,&e=b+c;  
10    // d,e 是非只读左值引用, 绑定到左值 a, 右值 b+c  
11    inc(b),c++;//a,b 的值变为 115,c 的值变为 515  
12    //d++,e++;//只读引用不可修改  
13    //int h=f();//未定义行为, 访问悬垂引用 f()  
14 }
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

- 基础类型

- 引用类型

- **数组类型**

- 函数类型

- 枚举类型

- 类类型

- 指针类型

- 成员指针类型

- cv 限定符

- typedef

- auto 和

decltype

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

数组类型

- 数组是固定大小的元素序列，用于储存多个相同类型数据。

数组类型

- 数组是固定大小的元素序列，用于储存多个相同类型数据。
- 数组声明是声明符拥有下列形式的简单声明：

1 非指针声明符 [表达式(可选)] 属性(可选)

数组类型

- 数组是固定大小的元素序列，用于储存多个相同类型数据。
- 数组声明是声明符拥有下列形式的简单声明：

1 非指针声明符 [表达式(可选)] 属性(可选)

- 非指针声明符：任意声明符，但若以`&`,`&&`,`*` 起始（分别用于声明数组左/右值引用，指向数组的指针），则需要以括号包围。若以`*` 起始且无括号包围，则声明指针的数组。

数组类型

- 数组是固定大小的元素序列，用于储存多个相同类型数据。
- 数组声明是声明符拥有下列形式的简单声明：

1 非指针声明符 [表达式(可选)] 属性(可选)

- 非指针声明符：任意声明符，但若以`&`、`&&`、`*` 起始（分别用于声明数组左/右值引用，指向数组的指针），则需要以括号包围。若以 `*` 起始且无括号包围，则声明指针的数组。
- 只有除了未知边界数组以外的对象类型能成为数组类型的元素类型，不存在引用的数组或函数的数组，不存在 `void` 的数组或函数的数组。

数组类型

- 数组是固定大小的元素序列，用于储存多个相同类型数据。
- 数组声明是声明符拥有下列形式的简单声明：

1 非指针声明符 [表达式(可选)] 属性(可选)

- 非指针声明符：任意声明符，但若以`&`,`&&`,`*` 起始（分别用于声明数组左/右值引用，指向数组的指针），则需要以括号包围。若以`*` 起始且无括号包围，则声明指针的数组。
- 只有除了未知边界数组以外的对象类型能成为数组类型的元素类型，不存在引用的数组或函数的数组，不存在 `void` 的数组或函数的数组。
- 数组初始化器可以使用花括号（聚合初始化），元素个数不足时自动补零。

数组类型

- 数组是固定大小的元素序列，用于储存多个相同类型数据。
- 数组声明是声明符拥有下列形式的简单声明：

1 非指针声明符 [表达式(可选)] 属性(可选)

- 非指针声明符：任意声明符，但若以`&`, `&&`, `*` 起始（分别用于声明数组左/右值引用，指向数组的指针），则需要以括号包围。若以 `*` 起始且无括号包围，则声明指针的数组。
- 只有除了未知边界数组以外的对象类型能成为数组类型的元素类型，不存在引用的数组或函数的数组，不存在 `void` 的数组或函数的数组。
- 数组初始化器可以使用花括号（聚合初始化），元素个数不足时自动补零。
- 数组大小必须是求值为大于零的值的整型常量表达式。编译器扩展可能支持非常量大小的数组。

数组类型

- 省略数组大小时，该数组类型是不完整类型，除非声明时指定聚合初始化器使其完整才能定义数组对象。

数组类型

- 省略数组大小时，该数组类型是不完整类型，除非声明时指定聚合初始化器使其完整才能定义数组对象。
- 数组对象不能整体赋值，即使是左值。尽管作为类成员时复制赋值运算符可以对该成员赋值

数组类型

- 省略数组大小时，该数组类型是不完整类型，除非声明时指定聚合初始化器使其完整才能定义数组对象。
- 数组对象不能整体赋值，即使是左值。尽管作为类成员时复制赋值运算符可以对该成员赋值
- 数组可以以下标（中括号）形式访问元素，下标从 0 到 N-1 标号（N 是数组大小）。
访问小于 0 或大于等于 N 的下标是未定义行为，称为数组访问越界。

数组类型

- 省略数组大小时，该数组类型是不完整类型，除非声明时指定聚合初始化器使其完整才能定义数组对象。
- 数组对象不能整体赋值，即使是左值。尽管作为类成员时复制赋值运算符可以对该成员赋值
- 数组可以以下标（中括号）形式访问元素，下标从 0 到 N-1 标号（N 是数组大小）。
访问小于 0 或大于等于 N 的下标是未定义行为，称为数组访问越界。
- 当数组的元素类型是另一数组时，可以定义多维数组。因为数组元素不能具有未知边界数组类型，所以多维数组只能在第一个维度中有未知边界。

数组类型

■ 示例:

```
1  int a[8]={0,4,2,6,1,5,3,7},b[]={1,2,3},c[8];
2  //int arr[]; //不完整数组类型不能定义数组对象
3  //c=a; //数组对象不能整体赋值
4  b[0]=3,b[1]++,b[2]=b[0]+b[1];
5  //b[-1]=b[3]=0; //数组访问越界
6  int map1[][3]={ {}, {}, {} }; //二维数组
7  map1[0][0]=map1[2][1]=1;
8  //int map2[3][]={ {}, {}, {1,2,3} };
9  //只有第一维可有未知边界
10 int(&d)[8]=a; //定义对数组的左值引用
11 //int&e[8]; //不存在引用的数组
12 int*f[8],(*g)[8];
13 //f是指针的数组, g是指针的指针
14 //int s[1000007]={}; //编译超时
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

- 基础类型
- 引用类型
- 数组类型
- **函数类型**
- 枚举类型
- 类类型
- 指针类型
- 成员指针类型
- cv 限定符
- typedef
- auto 和
decltype

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

函数类型

- 函数 (function) 是把一个语句序列 (函数体, function body) 关联到一个名字和零或更多个函数形参 (function parameter) 的列表的 C++ 实体。

函数类型

- 函数 (function) 是把一个语句序列 (函数体, function body) 关联到一个名字和零或更多个函数形参 (function parameter) 的列表的 C++ 实体。
- 函数声明可出现于任何作用域。在类作用域中的函数声明引入成员函数 (除非使用 friend 说明符)。

函数类型

- 函数 (function) 是把一个语句序列 (函数体, function body) 关联到一个名字和零或更多个函数形参 (function parameter) 的列表的 C++ 实体。
- 函数声明可出现于任何作用域。在类作用域中的函数声明引入成员函数 (除非使用 friend 说明符)。
- 被声明的函数类型由返回类型 (return type) (由声明语法的声明说明符序列提供) 和函数声明符 (声明符) 组成。

函数类型

- 函数 (function) 是把一个语句序列 (函数体, function body) 关联到一个名字和零或更多个函数形参 (function parameter) 的列表的 C++ 实体。
- 函数声明可出现于任何作用域。在类作用域中的函数声明引入成员函数 (除非使用 friend 说明符)。
- 被声明的函数类型由返回类型 (return type) (由声明语法的声明说明符序列提供) 和函数声明符 (声明符) 组成。
- 函数声明 (非函数定义) 是声明符拥有下列形式的简单声明:
 - 1 非指针声明符 (形参列表) cv限定符 (可选)
 - 2 引用限定符 (可选) 异常说明 (可选) 属性 (可选) (1)
 - 3 非指针声明符 (形参列表) cv限定符 (可选)
 - 4 引用限定符 (可选) 异常说明 (可选) 属性 (可选) -> 尾随返回类型 (2) (C++11 起)

函数类型

- 函数的返回类型不能是函数类型或数组类型。

函数类型

- 函数的返回类型不能是函数类型或数组类型。
- cv 限定符和引用限定符（C++11 起）只允许在非静态成员函数中使用。

函数类型

- 函数的返回类型不能是函数类型或数组类型。
- cv 限定符和引用限定符（C++11 起）只允许在非静态成员函数中使用。
- 尾随返回类型（C++11 起）用于返回类型取决于实参名，或者返回类型复杂时。此时声明说明符序列必须包含关键词 `auto`。

函数类型

- 函数的返回类型不能是函数类型或数组类型。
- cv 限定符和引用限定符（C++11 起）只允许在非静态成员函数中使用。
- 尾随返回类型（C++11 起）用于返回类型取决于实参名，或者返回类型复杂时。此时声明说明符序列必须包含关键词 `auto`。
- 如果函数声明的声明说明符序列包含关键词 `auto`，那么尾随返回类型可以省略（C++14 起）。编译器将进行返回类型推导。

函数类型

- 函数的返回类型不能是函数类型或数组类型。
- cv 限定符和引用限定符（C++11 起）只允许在非静态成员函数中使用。
- 尾随返回类型（C++11 起）用于返回类型取决于实参名，或者返回类型复杂时。此时声明说明符序列必须包含关键词 `auto`。
- 如果函数声明的声明说明符序列包含关键词 `auto`，那么尾随返回类型可以省略（C++14 起）。编译器将进行返回类型推导。
- 形参列表：函数形参声明的逗号分隔列表，可以为空。

函数类型

- 函数的返回类型不能是函数类型或数组类型。
- cv 限定符和引用限定符（C++11 起）只允许在非静态成员函数中使用。
- 尾随返回类型（C++11 起）用于返回类型取决于实参名，或者返回类型复杂时。此时声明说明符序列必须包含关键词 `auto`。
- 如果函数声明的声明说明符序列包含关键词 `auto`，那么尾随返回类型可以省略（C++14 起）。编译器将进行返回类型推导。
- 形参列表：函数形参声明的逗号分隔列表，可以为空。
- 其中每一项形参声明是简单声明，声明可拥有抽象声明符（声明无名形参）和初始化器（声明带有默认值的形参）。

函数类型

- 函数的返回类型不能是函数类型或数组类型。
- cv 限定符和引用限定符（C++11 起）只允许在非静态成员函数中使用。
- 尾随返回类型（C++11 起）用于返回类型取决于实参名，或者返回类型复杂时。此时声明说明符序列必须包含关键词 `auto`。
- 如果函数声明的声明说明符序列包含关键词 `auto`，那么尾随返回类型可以省略（C++14 起）。编译器将进行返回类型推导。
- 形参列表：函数形参声明的逗号分隔列表，可以为空。
- 其中每一项形参声明是简单声明，声明可拥有抽象声明符（声明无名形参）和初始化器（声明带有默认值的形参）。
- 默认值只能为形参列表的一个后缀。

函数类型

- 函数的返回类型不能是函数类型或数组类型。
- cv 限定符和引用限定符（C++11 起）只允许在非静态成员函数中使用。
- 尾随返回类型（C++11 起）用于返回类型取决于实参名，或者返回类型复杂时。此时声明说明符序列必须包含关键词 `auto`。
- 如果函数声明的声明说明符序列包含关键词 `auto`，那么尾随返回类型可以省略（C++14 起）。编译器将进行返回类型推导。
- 形参列表：函数形参声明的逗号分隔列表，可以为空。
- 其中每一项形参声明是简单声明，声明可拥有抽象声明符（声明无名形参）和初始化器（声明带有默认值的形参）。
- 默认值只能为形参列表的一个后缀。
- 形参列表也可以只有一项 `void`，与空参数列表意义相同。

函数类型

- 函数的返回类型不能是函数类型或数组类型。
- cv 限定符和引用限定符（C++11 起）只允许在非静态成员函数中使用。
- 尾随返回类型（C++11 起）用于返回类型取决于实参名，或者返回类型复杂时。此时声明说明符序列必须包含关键词 `auto`。
- 如果函数声明的声明说明符序列包含关键词 `auto`，那么尾随返回类型可以省略（C++14 起）。编译器将进行返回类型推导。
- 形参列表：函数形参声明的逗号分隔列表，可以为空。
- 其中每一项形参声明是简单声明，声明可拥有抽象声明符（声明无名形参）和初始化器（声明带有默认值的形参）。
- 默认值只能为形参列表的一个后缀。
- 形参列表也可以只有一项 `void`，与空参数列表意义相同。
- 形参列表末尾可出现省略号...，这声明一个变参函数（`variadic function`）。省略号前可以出现一个逗号。

函数类型

- 形参列表中的每个函数形参的类型根据下列规则确定:

函数类型

- 形参列表中的每个函数形参的类型根据下列规则确定：
 - 1 首先组合声明说明符序列和声明符以确定其类型。

函数类型

- 形参列表中的每个函数形参的类型根据下列规则确定：
 - 1 首先组合声明说明符序列和声明符以确定其类型。
 - 2 如果类型是“T 的数组”或“T 的未知边界数组”，那么它被替换成类型“T 的指针”

函数类型

- 形参列表中的每个函数形参的类型根据下列规则确定：
 - 1 首先组合声明说明符序列和声明符以确定其类型。
 - 2 如果类型是“T 的数组”或“T 的未知边界数组”，那么它被替换成类型“T 的指针”
 - 3 如果类型是“函数类型 F”，那么它被替换成类型“F 的指针”

函数类型

- 形参列表中的每个函数形参的类型根据下列规则确定：
 - 1 首先组合声明说明符序列和声明符以确定其类型。
 - 2 如果类型是“T 的数组”或“T 的未知边界数组”，那么它被替换成类型“T 的指针”
 - 3 如果类型是“函数类型 F”，那么它被替换成类型“F 的指针”
 - 4 从形参类型中丢弃顶层 cv 限定符（此举只影响函数的类型，实际的形参类型不变）

函数类型

- 形参列表中的每个函数形参的类型根据下列规则确定：
 - 1 首先组合声明说明符序列和声明符以确定其类型。
 - 2 如果类型是“T 的数组”或“T 的未知边界数组”，那么它被替换成类型“T 的指针”
 - 3 如果类型是“函数类型 F”，那么它被替换成类型“F 的指针”
 - 4 从形参类型中丢弃顶层 cv 限定符（此举只影响函数的类型，实际的形参类型不变）
- 其中，规则 2 称为数组退化。

函数类型

■ 示例:

```
1  int f(int);  
2  //声明一个函数 f 返回值为 int, int 作为无名形参  
3  int g(int a, int b=1, int=3);  
4  //声明一个函数 g 返回值为 int, 接受 3 个 int 参数  
5  //int g2(int=3, int, int=1)  
6  //默认值只能为形参列表的一个后缀  
7  int&h1(void), &h2(); //h1, h2 同类型  
8  //int[3] f1(); int() f2();  
9  //函数不能返回函数或数组类型  
10 int(&f3())[3]; int(*f4())();  
11 //但可以是到它们的指针或引用  
12 int a(int[8]), b(int[]), c(int(...)),  
13 d(int(*) (...)), e(int* const);  
14 auto h(int*)->int;  
15 //abeh 同类型, cd 同类型
```

目录

1	注释	8	字面量	11	语句
2	标识符	9	类型	12	函数
3	关键字		■ 基础类型	13	模板
4	声明		■ 引用类型	14	命名空间
5	对象		■ 数组类型	15	属性
6	引用		■ 函数类型	16	static_assert
7	变量		■ 枚举类型	17	预处理指令
			■ 类类型	18	常用 C 库
			■ 指针类型		
			■ 成员指针类型		
			■ cv 限定符		
			■ typedef		
			■ auto 和		
			decltype		
		10	表达式		

枚举类型

- 枚举（enumeration）是独立的类型，它的值被限制在一个取值范围内。

枚举类型

- 枚举 (enumeration) 是独立的类型，它的值被限制在一个取值范围内。
- 它可以包含数个明确命名的常量 (枚举项 (enumerator))，各常量的值是某个整型类型 (称为该枚举的底层类型 (underlying type)) 的值。

枚举类型

- 枚举 (enumeration) 是独立的类型, 它的值被限制在一个取值范围内。
- 它可以包含数个明确命名的常量 (枚举项 (enumerator)), 各常量的值是某个整型类型 (称为该枚举的底层类型 (underlying type)) 的值。

- 枚举以下列语法定义:

- 1 枚举关键词 属性(可选) 枚举名(可选) 枚举基(可选)
- 2 { 枚举项列表(可选) }; (1)
- 3 枚举关键词 属性(可选) 枚举名 枚举基(可选) ; (2)
- 4 (C++11 起)

枚举类型

- 枚举 (enumeration) 是独立的类型，它的值被限制在一个取值范围内。
- 它可以包含数个明确命名的常量 (枚举项 (enumerator))，各常量的值是某个整型类型 (称为该枚举的底层类型 (underlying type)) 的值。

- 枚举以下列语法定义：

```
1 枚举关键词 属性(可选) 枚举名(可选) 枚举基(可选)
2  { 枚举项列表(可选) }; (1)
3  枚举关键词 属性(可选) 枚举名 枚举基(可选) ; (2)
4  (C++11 起)
```

- (2) 是不可见 (opaque) 枚举声明：定义枚举类型，但不定义其枚举项：在此声明后，该类型是完整类型，且其大小已知。

枚举类型

- 枚举 (enumeration) 是独立的类型，它的值被限制在一个取值范围内。
- 它可以包含数个明确命名的常量 (枚举项 (enumerator))，各常量的值是某个整型类型 (称为该枚举的底层类型 (underlying type)) 的值。

- 枚举以下列语法定义：

```
1 枚举关键词 属性(可选) 枚举名(可选) 枚举基(可选)
2  { 枚举项列表(可选) }; (1)
3 枚举关键词 属性(可选) 枚举名 枚举基(可选) ; (2)
4  (C++11 起)
```

- (2) 是不可见 (opaque) 枚举声明：定义枚举类型，但不定义其枚举项：在此声明后，该类型是完整类型，且其大小已知。
- 枚举关键词为 `enum`，`enum class` (C++11 起) 或 `enum struct` (C++11 起)。前者定义无作用域枚举，后两者定义有作用域枚举。

枚举类型

- 枚举名是所声明的枚举的名字。枚举基（C++11 起）是冒号后随某个整型类型的类型说明符序列，用于固定该枚举的底层类型。

枚举类型

- 枚举名是所声明的枚举的名字。枚举基（C++11 起）是冒号后随某个整型类型的类型说明符序列，用于固定该枚举的底层类型。
- 枚举项列表是标识符的逗号分隔列表，可带初始化器。无初始化器的枚举项的值为前一项加 1，首项则为 0。

枚举类型

- 枚举名是所声明的枚举的名字。枚举基（C++11 起）是冒号后随某个整型类型的类型说明符序列，用于固定该枚举的底层类型。
- 枚举项列表是标识符的逗号分隔列表，可带初始化器。无初始化器的枚举项的值为前一项加 1，首项则为 0。
- 每个枚举项都成为该枚举类型（即名字）的一个具名常量，无作用域枚举枚举项的在其外围作用域可见，且可用于要求常量的任何位置。

枚举类型

- 枚举名是所声明的枚举的名字。枚举基（C++11 起）是冒号后随某个整型类型的类型说明符序列，用于固定该枚举的底层类型。
- 枚举项列表是标识符的逗号分隔列表，可带初始化器。无初始化器的枚举项的值为前一项加 1，首项则为 0。
- 每个枚举项都成为该枚举类型（即名字）的一个具名常量，无作用域枚举枚举项的在其外围作用域可见，且可用于要求常量的任何位置。
- 无作用域枚举的不可见枚举声明必须指定名字与底层类型。

枚举类型

- 枚举名是所声明的枚举的名字。枚举基（C++11 起）是冒号后随某个整型类型的类型说明符序列，用于固定该枚举的底层类型。
- 枚举项列表是标识符的逗号分隔列表，可带初始化器。无初始化器的枚举项的值为前一项加 1，首项则为 0。
- 每个枚举项都成为该枚举类型（即名字）的一个具名常量，无作用域枚举枚举项的在其外围作用域可见，且可用于要求常量的任何位置。
- 无作用域枚举的不可见枚举声明必须指定名字与底层类型。
- 无作用域枚举类型的值可隐式转换为整型类型，反之需要显式类型转换。

枚举类型

- 枚举名是所声明的枚举的名字。枚举基（C++11 起）是冒号后随某个整型类型的类型说明符序列，用于固定该枚举的底层类型。
- 枚举项列表是标识符的逗号分隔列表，可带初始化器。无初始化器的枚举项的值为前一项加 1，首项则为 0。
- 每个枚举项都成为该枚举类型（即名字）的一个具名常量，无作用域枚举项的在其外围作用域可见，且可用于要求常量的任何位置。
- 无作用域枚举的不可见枚举声明必须指定名字与底层类型。
- 无作用域枚举类型的值可隐式转换为整型类型，反之需要显式类型转换。
- 有作用域枚举底层类型默认为 `int`，枚举项用作用域解析运算符 `::` 访问。有作用域枚举不可隐式转换为整型类型。

枚举类型

■ 示例:

```
1  enum color{red,green=6,blue};
2  //red=0,green=6,blue=7
3  int a=red,b=green;
4  color c=red,d=(color)2;
5  //color e=(color)9;
6  //整型转换为枚举类型必须在枚举类型范围内
7  //数值可以不等于所有枚举项
8  enum class day{Monday=1,Sunday=7};
9  day a1=day::Sunday;
10 //day a2=Monday;//有作用域枚举必须用 :: 访问
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

- 基础类型
- 引用类型
- 数组类型
- 函数类型
- 枚举类型
- 类类型
- 指针类型
- 成员指针类型
- cv 限定符
- typedef
- auto 和

decltype

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

类类型

- 类是一种用户定义类型，用于存储多个可以不同的数据。C++ 通过创建类来支持封装和数据隐藏。

类类型

- 类是一种用户定义类型，用于存储多个可以不同的数据。C++ 通过创建类来支持封装和数据隐藏。
- 类的类型可以由类说明符定义，它在声明语法的声明说明符序列中出现

类类型

- 类是一种用户定义类型，用于存储多个可以不同的数据。C++ 通过创建类来支持封装和数据隐藏。
- 类的类型可以由类说明符定义，它在声明语法的声明说明符序列中出现
- 类以下列语法声明：

```
1      类关键词 属性(可选) 类头名(可选)  
2      基类子句(可选) { 成员说明 } (1)  
3      类关键词 属性(可选) 标识符 ; (2)
```

类类型

- 类是一种用户定义类型，用于存储多个可以不同的数据。C++ 通过创建类来支持封装和数据隐藏。
- 类的类型可以由类说明符定义，它在声明语法的声明说明符序列中出现
- 类以下列语法声明：

```
1  类关键词 属性(可选) 类头名(可选)  
2  基类子句(可选) { 成员说明 } (1)  
3  类关键词 属性(可选) 标识符 ; (2)
```

- (1) 是类类型的定义，(2) 是前置声明。由(2) 声明的类在定义前是不完整类型。

类类型

- 类是一种用户定义类型，用于存储多个可以不同的数据。C++ 通过创建类来支持封装和数据隐藏。
- 类的类型可以由类说明符定义，它在声明语法的声明说明符序列中出现
- 类以下列语法声明：

```
1      类关键词 属性(可选) 类头名(可选)  
2      基类子句(可选) { 成员说明 } (1)  
3      类关键词 属性(可选) 标识符 ; (2)
```

- (1) 是类类型的定义，(2) 是前置声明。由(2) 声明的类在定义前是不完整类型。
- 类关键词是 `class`, `struct`, `union` 之一。`class` 与 `struct` 除了默认成员访问和默认基类访问之外无区别。

类类型

- 类是一种用户定义类型，用于存储多个可以不同的数据。C++ 通过创建类来支持封装和数据隐藏。
- 类的类型可以由类说明符定义，它在声明语法的声明说明符序列中出现
- 类以下列语法声明：

```
1  类关键词 属性(可选) 类头名(可选)
2  基类子句(可选) { 成员说明 } (1)
3  类关键词 属性(可选) 标识符 ; (2)
```

- (1) 是类类型的定义，(2) 是前置声明。由(2) 声明的类在定义前是不完整类型。
- 类关键词是 `class`, `struct`, `union` 之一。`class` 与 `struct` 除了默认成员访问和默认基类访问之外无区别。
- 基类子句是以冒号开头的一或多个基类以及各自所用的继承模型的可选列表。该类称为基类的派生类，派生类继承了基类的内容。

类类型

- 类头名是被定义类名。可以前接嵌套名说明符（名字和作用域解析运算符`::` 的序列，以`::` 结尾），可以后随关键词 `final`（C++11 起），表示该类不可被继承。

类类型

- 类头名是被定义类名。可以前接嵌套名说明符（名字和作用域解析运算符`::` 的序列，以`::` 结尾），可以后随关键词 `final`（C++11 起），表示该类不可被继承。
- 可以忽略类头名，此时类是无名的（注意无名类不可以是 `final` 的）。

类类型

- 类头名是被定义类名。可以前接嵌套名说明符（名字和作用域解析运算符`::` 的序列，以`::` 结尾），可以后随关键词 `final`（C++11 起），表示该类不可被继承。
- 可以忽略类头名，此时类是无名的（注意无名类不可以是 `final` 的）。
- 成员说明是访问说明符、成员对象和成员函数的声明和定义的列表，是任何数量下列各项的序列：

类类型

- 类头名是被定义类名。可以前接嵌套名说明符（名字和作用域解析运算符`::` 的序列，以`::` 结尾），可以后随关键词 `final`（C++11 起），表示该类不可被继承。
- 可以忽略类头名，此时类是无名的（注意无名类不可以是 `final` 的）。
- 成员说明是访问说明符、成员对象和成员函数的声明和定义列表，是任何数量下列各项的序列：
 - 1 成员对象、函数声明和友元声明：形式同简单声明，额外允许位域定义、纯说明符和虚说明符（`override` 或 `final`）（C++11 起），并且不允许直接非列表初始化语法。

类类型

- 类头名是被定义类名。可以前接嵌套名说明符（名字和作用域解析运算符`::` 的序列，以`::` 结尾），可以后随关键词 `final`（C++11 起），表示该类不可被继承。
- 可以忽略类头名，此时类是无名的（注意无名类不可以是 `final` 的）。
- 成员说明是访问说明符、成员对象和成员函数的声明和定义的列表，是任何数量下列各项的序列：
 - 1 成员对象、函数声明和友元声明：形式同简单声明，额外允许位域定义、纯说明符和虚说明符（`override` 或 `final`）（C++11 起），并且不允许直接非列表初始化语法。
 - 2 函数定义：同时声明并定义成员函数或者友元函数。

类类型

- 类头名是被定义类名。可以前接嵌套名说明符（名字和作用域解析运算符`::`的序列，以`::`结尾），可以后随关键词 `final`（C++11 起），表示该类不可被继承。
- 可以忽略类头名，此时类是无名的（注意无名类不可以是 `final` 的）。
- 成员说明是访问说明符、成员对象和成员函数的声明和定义的列表，是任何数量下列各项的序列：
 - 1 成员对象、函数声明和友元声明：形式同简单声明，额外允许位域定义、纯说明符和虚说明符（`override` 或 `final`）（C++11 起），并且不允许直接非列表初始化语法。
 - 2 函数定义：同时声明并定义成员函数或者友元函数。
 - 3 访问说明符 `public:`、`protected:` 和 `private:`

类类型

- 类头名是被定义类名。可以前接嵌套名说明符（名字和作用域解析运算符`::` 的序列，以`::` 结尾），可以后随关键词 `final`（C++11 起），表示该类不可被继承。
- 可以忽略类头名，此时类是无名的（注意无名类不可以是 `final` 的）。
- 成员说明是访问说明符、成员对象和成员函数的声明和定义的列表，是任何数量下列各项的序列：
 - 1 成员对象、函数声明和友元声明：形式同简单声明，额外允许位域定义、纯说明符和虚说明符（`override` 或 `final`）（C++11 起），并且不允许直接非列表初始化语法。
 - 2 函数定义：同时声明并定义成员函数或者友元函数。
 - 3 访问说明符 `public:`、`protected:` 和 `private:`
 - 4 `using` 声明、`static_assert` 声明、成员模板声明、别名声明

类类型

- 其中由 1, 2 (非友元声明) 声明的对象称为成员对象、函数称为成员函数。

类类型

- 其中由 1, 2 (非友元声明) 声明的对象称为成员对象、函数称为成员函数。
- 成员 (对象、函数, 以下省略) 的声明前若有 `static` 修饰符则称为静态成员, 反之为非静态成员。类的静态成员不与类的对象关联, 只属于整个类类型。非静态成员与类的对象关联, 不同对象的非静态成员是独立的。

类类型

- 其中由 1, 2 (非友元声明) 声明的对象称为成员对象、函数称为成员函数。
- 成员 (对象、函数, 以下省略) 的声明前若有 `static` 修饰符则称为静态成员, 反之为非静态成员。类的静态成员不与类的对象关联, 只属于整个类类型。非静态成员与类的对象关联, 不同对象的非静态成员是独立的。
- 类类型对象可以通过成员访问运算符 (点号) 来访问成员。静态成员也可以通过作用域解析运算符来访问。

类类型

- 其中由 1, 2 (非友元声明) 声明的对象称为成员对象、函数称为成员函数。
- 成员 (对象、函数, 以下省略) 的声明前若有 `static` 修饰符则称为静态成员, 反之为非静态成员。类的静态成员不与类的对象关联, 只属于整个类类型。非静态成员与类的对象关联, 不同对象的非静态成员是独立的。
- 类类型对象可以通过成员访问运算符 (点号) 来访问成员。静态成员也可以通过作用域解析运算符来访问。
- 访问说明符可以限制类体外对成员的访问: `public:` (公开, 无限制), `protected:` (保护, 只有派生类、友元、自身可访问), `private:` (私有, 只有友元, 自身可访问)。

类类型

- 其中由 1, 2 (非友元声明) 声明的对象称为成员对象、函数称为成员函数。
- 成员 (对象、函数, 以下省略) 的声明前若有 `static` 修饰符则称为静态成员, 反之为非静态成员。类的静态成员不与类的对象关联, 只属于整个类类型。非静态成员与类的对象关联, 不同对象的非静态成员是独立的。
- 类类型对象可以通过成员访问运算符 (点号) 来访问成员。静态成员也可以通过作用域解析运算符来访问。
- 访问说明符可以限制类体外对成员的访问: `public:` (公开, 无限制), `protected:` (保护, 只有派生类、友元、自身可访问), `private:` (私有, 只有友元, 自身可访问)。
- `class` 默认访问说明符为 `private:` 其他类关键词默认访问说明符为 `public:`。

类类型

- 基类访问说明符用于限制继承得到的成员的访问, `protected` 让所有基类成员变成 `protected:`, `private` 让所有基类成员变成 `private:`。 `class` 默认是 `private`, `struct` 和 `union` 默认是 `public` 即不修改。

类类型

- 基类访问说明符用于限制继承得到的成员的访问, `protected` 让所有基类成员变成 `protected:`, `private` 让所有基类成员变成 `private:`。 `class` 默认是 `private`, `struct` 和 `union` 默认是 `public` 即不修改。
- `union` 联合体是特殊的类类型, 它在一个时刻只能保有其一个非静态数据成员。位域声明是在非静态整型或枚举类型成员声明的标识符后加上: 大小, 标识符可省略(无名位域)。两者常用于优化内存。

类类型

- 基类访问说明符用于限制继承得到的成员的访问, `protected` 让所有基类成员变成 `protected:`, `private` 让所有基类成员变成 `private:`。 `class` 默认是 `private`, `struct` 和 `union` 默认是 `public` 即不修改。
- `union` 联合体是特殊的类类型, 它在一个时刻只能保有其一个非静态数据成员。位域声明是在非静态整型或枚举类型成员声明的标识符后加上: 大小, 标识符可省略(无名位域)。两者常用于优化内存。
- 类类型对象初始化器可以使用花括号(聚合初始化), 前提是成员对象均为 `public`。

类类型

■ 示例:

```
1  class node{int x;public:int y;void inc(){y++;}};
2  //node a={1,2};//node::x 为private
3  node b;b.y=1,b.inc();
4  //b.x++;//node::x 为private
5  struct f{static int val;int s:8,h:9};
6  f n={1,2};//n.s=1,n.h=2
7  int f::val=7;//非静态成员需要在类外定义
8  struct ff{static const int m=10};
9  //除非该成员为 const 且在类体内部初始化
10 union s{int a;double b;}p;
11 p.a=f::val,p.b=1.0;//p.a 失效
12 //int pa=p.a;//访问非活跃成员, 未定义行为
13 struct node2:public node{int s;}a2;
14 a2.s=1,a2.y=2,++a2.node::y,a2.inc();
15 //a2 继承了 node 的 y,inc
16 //a2.x=7;//x 是 node 的 private 成员
```


目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

- 基础类型

- 引用类型

- 数组类型

- 函数类型

- 枚举类型

- 类类型

- **指针类型**

- 成员指针类型

- cv 限定符

- typedef

- auto 和

decltype

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

指针类型

- 指针类型是对象类型，用来存放对象或函数的内存地址。存在指向指针的指针，存在指向 void 的指针，不存在指向引用的指针和指向位域的指针。

指针类型

- 指针类型是对象类型，用来存放对象或函数的内存地址。存在指向指针的指针，存在指向 void 的指针，不存在指向引用的指针和指向位域的指针。
- 指针变量声明是声明符拥有下列形式的简单声明

1 * 属性(可选) cv限定符(可选) 声明符

指针类型

- 指针类型是对象类型，用来存放对象或函数的内存地址。存在指向指针的指针，存在指向 void 的指针，不存在指向引用的指针和指向位域的指针。
- 指针变量声明是声明符拥有下列形式的简单声明

1 * 属性(可选) cv限定符(可选) 声明符

- 取址运算符& 可以获取左值对象或函数的地址。(由于存在函数到指针的隐式转换，取址运算符可以忽略)

指针类型

- 指针类型是对象类型，用来存放对象或函数的内存地址。存在指向指针的指针，存在指向 void 的指针，不存在指向引用的指针和指向位域的指针。
- 指针变量声明是声明符拥有下列形式的简单声明

1 * 属性(可选) cv限定符(可选) 声明符

- 取址运算符& 可以获取左值对象或函数的地址。(由于存在函数到指针的隐式转换，取址运算符可以忽略)
- 间接寻址(解引用)运算符 可以返回指代被指向对象或函数的左值表达式(解引用)。

指针类型

- 指针类型是对象类型，用来存放对象或函数的内存地址。存在指向指针的指针，存在指向 `void` 的指针，不存在指向引用的指针和指向位域的指针。
- 指针变量声明是声明符拥有下列形式的简单声明

1 * 属性(可选) cv限定符(可选) 声明符

- 取址运算符 `&` 可以获取左值对象或函数的地址。（由于存在函数到指针的隐式转换，取址运算符可以忽略）
- 间接寻址（解引用）运算符 `*` 可以返回指代被指向对象或函数的左值表达式（解引用）。
- 指针也可以像数组一样，以下标形式访问元素。`p[i]` 等价于 `i[p]` 等价于 `*(p+i)`，其中 `p` 为指针（或数组），`i` 为下标（整数类型）。
特别的，`p[0]` 等价于 `*p`。

指针类型

- 指针的值是以下之一：

指针类型

- 指针的值是以下之一：

- 1 对象或函数的所占用的内存的首字节地址（称该指针指向函数或对象）

指针类型

- 指针的值是以下之一：
 - 1 对象或函数的所占用的内存的首字节地址（称该指针指向函数或对象）
 - 2 对象所占用的存储之后的首字节地址

指针类型

- 指针的值是以下之一：
 - 1 对象或函数的所占用的内存的首字节地址（称该指针指向函数或对象）
 - 2 对象所占用的存储之后的首字节地址
 - 3 该类型的空指针值

指针类型

- 指针的值是以下之一：

- 1 对象或函数的所占用的内存的首字节地址（称该指针指向函数或对象）
- 2 对象所占用的存储之后的首字节地址
- 3 该类型的空指针值
- 4 无效指针值

指针类型

- 指针的值是以下之一：
 - 1 对象或函数的所占用的内存的首字节地址（称该指针指向函数或对象）
 - 2 对象所占用的存储之后的首字节地址
 - 3 该类型的空指针值
 - 4 无效指针值
- 只有 1 可以解引用，得到被指向对象或函数的左值表达式。

指针类型

- 指针的值是以下之一：
 - 1 对象或函数的所占用的内存的首字节地址（称该指针指向函数或对象）
 - 2 对象所占用的存储之后的首字节地址
 - 3 该类型的空指针值
 - 4 无效指针值
- 只有 1 可以解引用，得到被指向对象或函数的左值表达式。
- 部分实现会将复制无效指针值的行为定义为造成由系统生成的运行时错误（Runtime Error）。

指针类型

- 指针的值是以下之一：
 - 1 对象或函数的所占用的内存的首字节地址（称该指针指向函数或对象）
 - 2 对象所占用的存储之后的首字节地址
 - 3 该类型的空指针值
 - 4 无效指针值
- 只有 1 可以解引用，得到被指向对象或函数的左值表达式。
- 部分实现会将复制无效指针值的行为定义为造成由系统生成的运行时错误（Runtime Error）。
- 基类指针可以指向派生类，反之需要 `dynamic_cast` 类型转换，限于篇幅，这里不进行展开。

指针类型

- 指针的值是以下之一：
 - 1 对象或函数的所占用的内存的首字节地址（称该指针指向函数或对象）
 - 2 对象所占用的存储之后的首字节地址
 - 3 该类型的空指针值
 - 4 无效指针值
- 只有 1 可以解引用，得到被指向对象或函数的左值表达式。
- 部分实现会将复制无效指针值的行为定义为造成由系统生成的运行时错误（Runtime Error）。
- 基类指针可以指向派生类，反之需要 `dynamic_cast` 类型转换，限于篇幅，这里不进行展开。
- 可对完整类型对象指针做算术运算：一元加（提升，将数组或函数转换为同类型指针），加法（指针加整数，或反过来），减法（指针减整数，指针减指针）。

指针类型

- 二元指针算术的结果按以下方法确定，其它可用于指针的运算（自增自减，复合赋值）也适用：

指针类型

- 二元指针算术的结果按以下方法确定，其它可用于指针的运算（自增自减，复合赋值）也适用：
 - 1 将指向非数组对象的指针看作指向长度为 1 的该对象数组的首元素的指针。指向对象所占用的存储之后的指针看作指向下标为 N 的元素，N 为数组大小。

指针类型

- 二元指针算术的结果按以下方法确定，其它可用于指针的运算（自增自减，复合赋值）也适用：
 - 1 将指向非数组对象的指针看作指向长度为 1 的该对象数组的首元素的指针。指向对象所占用的存储之后的指针看作指向下标为 N 的元素，N 为数组大小。
 - 2 设 p, q 分别为指向同一数组的第 i, j 个元素，则表达式 $p - q$ 的值为 $i - j$ 。

指针类型

- 二元指针算术的结果按以下方法确定，其它可用于指针的运算（自增自减，复合赋值）也适用：
 - 1 将指向非数组对象的指针看作指向长度为 1 的该对象数组的首元素的指针。指向对象所占用的存储之后的指针看作指向下标为 N 的元素，N 为数组大小。
 - 2 设 p, q 分别为指向同一数组的第 i, j 个元素，则表达式 $p - q$ 的值为 $i - j$ 。
 - 3 否则若 p, q 均为空指针，则生成 θ ，否则行为未定义。

指针类型

- 二元指针算术的结果按以下方法确定，其它可用于指针的运算（自增自减，复合赋值）也适用：
 - 1 将指向非数组对象的指针看作指向长度为 1 的该对象数组的首元素的指针。指向对象所占用的存储之后的指针看作指向下标为 N 的元素，N 为数组大小。
 - 2 设 p, q 分别为指向同一数组的第 i, j 个元素，则表达式 $p - q$ 的值为 $i - j$ 。
 - 3 否则若 p, q 均为空指针，则生成 0，否则行为未定义。
 - 4 设 p 为指向数组的第 i 个元素， n 为整数，则表达式 $p + n$ 、 $n + p$ 和 $p - n$ 的值分别是指向同一数组的第 $i + n$ 、第 $i + n$ 和第 $i - n$ 个元素的同类型指针。

指针类型

- 二元指针算术的结果按以下方法确定，其它可用于指针的运算（自增自减，复合赋值）也适用：
 - 1 将指向非数组对象的指针看作指向长度为 1 的该对象数组的首元素的指针。指向对象所占用的存储之后的指针看作指向下标为 N 的元素， N 为数组大小。
 - 2 设 p, q 分别为指向同一数组的第 i, j 个元素，则表达式 $p - q$ 的值为 $i - j$ 。
 - 3 否则若 p, q 均为空指针，则生成 θ ，否则行为未定义。
 - 4 设 p 为指向数组的第 i 个元素， n 为整数，则表达式 $p + n$ 、 $n + p$ 和 $p - n$ 的值分别是指向同一数组的第 $i + n$ 、第 $i + n$ 和第 $i - n$ 个元素的同类型指针。
 - 5 若生成非法指针值，则引起未定义行为。

指针类型

- 二元指针算术的结果按以下方法确定，其它可用于指针的运算（自增自减，复合赋值）也适用：
 - 1 将指向非数组对象的指针看作指向长度为 1 的该对象数组的首元素的指针。指向对象所占用的存储之后的指针看作指向下标为 N 的元素，N 为数组大小。
 - 2 设 p, q 分别为指向同一数组的第 i, j 个元素，则表达式 $p - q$ 的值为 $i - j$ 。
 - 3 否则若 p, q 均为空指针，则生成 0，否则行为未定义。
 - 4 设 p 为指向数组的第 i 个元素， n 为整数，则表达式 $p + n$ 、 $n + p$ 和 $p - n$ 的值分别是指向同一数组的第 $i + n$ 、第 $i + n$ 和第 $i - n$ 个元素的同类型指针。
 - 5 若生成非法指针值，则引起未定义行为。
 - 6 任何情况下，如果所指向的类型与数组元素类型不同（不考虑 cv 限定），则二元指针算术的行为未定义。如对指向派生类的基类指针进行二元指针算术。

指针类型

- 还可以对指针进行比较运算，规则比较复杂。对于 $p-q$ 有效的指针，可以根据结果的大小来判定比较运算的结果。

指针类型

- 还可以对指针进行比较运算，规则比较复杂。对于 $p-q$ 有效的指针，可以根据结果的大小来判定比较运算的结果。
- 对于指向类类型的指针，可以使用成员访问运算符 $\rightarrow$ 来访问成员，而无需使用先解引用再加括号再用点号访问成员。对于内建的 $\rightarrow$ 运算符， $E1 \rightarrow E2$ 严格等价于 $(*E1).E2$ 。

指针类型

- 还可以对指针进行比较运算，规则比较复杂。对于 $p-q$ 有效的指针，可以根据结果的大小来判定比较运算的结果。
- 对于指向类类型的指针，可以使用成员访问运算符 $\rightarrow$ 来访问成员，而无需使用先解引用再加括号再用点号访问成员。对于内建的 $\rightarrow$ 运算符， $E1 \rightarrow E2$ 严格等价于 $(*E1).E2$ 。
- 任何类型的指针均可以隐式转换为 `void` 类型的指针，反之则需要 `static_cast` 或显示强制转换。`void` 类型指针不能解引用，不能参与指针算术，但可以进行比较。

指针类型

- 还可以对指针进行比较运算，规则比较复杂。对于 $p-q$ 有效的指针，可以根据结果的大小来判定比较运算的结果。
- 对于指向类类型的指针，可以使用成员访问运算符 $\rightarrow$ 来访问成员，而无需使用先解引用再加括号再用点号访问成员。对于内建的 $\rightarrow$ 运算符， $E1 \rightarrow E2$ 严格等价于 $(*E1).E2$ 。
- 任何类型的指针均可以隐式转换为 `void` 类型的指针，反之则需要 `static_cast` 或显示强制转换。`void` 类型指针不能解引用，不能参与指针算术，但可以进行比较。
- `void` 类型的指针常用于存储未知类型的地址。

指针类型

- 还可以对指针进行比较运算，规则比较复杂。对于 $p-q$ 有效的指针，可以根据结果的大小来判定比较运算的结果。
- 对于指向类类型的指针，可以使用成员访问运算符 $\rightarrow$ 来访问成员，而无需使用先解引用再加括号再用点号访问成员。对于内建的 $\rightarrow$ 运算符， $E1 \rightarrow E2$ 严格等价于 $(*E1).E2$ 。
- 任何类型的指针均可以隐式转换为 `void` 类型的指针，反之则需要 `static_cast` 或显示强制转换。`void` 类型指针不能解引用，不能参与指针算术，但可以进行比较。
- `void` 类型的指针常用于存储未知类型的地址。
- 函数指针可以解引用后作为函数调用运算符的左操作数，也可以不显式写出解引用。

指针类型

- 还可以对指针进行比较运算，规则比较复杂。对于 $p-q$ 有效的指针，可以根据结果的大小来判定比较运算的结果。
- 对于指向类类型的指针，可以使用成员访问运算符 $\rightarrow$ 来访问成员，而无需使用先解引用再加括号再用点号访问成员。对于内建的 $\rightarrow$ 运算符， $E1 \rightarrow E2$ 严格等价于 $(*E1).E2$ 。
- 任何类型的指针均可以隐式转换为 `void` 类型的指针，反之则需要 `static_cast` 或显示强制转换。`void` 类型指针不能解引用，不能参与指针算术，但可以进行比较。
- `void` 类型的指针常用于存储未知类型的地址。
- 函数指针可以解引用后作为函数调用运算符的左操作数，也可以不显式写出解引用。
- `new/new[]` 运算符可用于动态分配对象/对象数组的空间，返回该对象/对象数组的首地址。使用 `delete/delete[]`（必须匹配）解分配后，该指针成为无效指针。

指针类型

■ 示例:

```
1  int f(){return 1;}int*a,(*b)[],(*c)()=f;
2  //a 为 int*, b 为 int(*)[], c 为 int(*)(), 指向 f
3  int res1=(*c)(),res2=c();//相当于直接调用 f
4  int v[10],*p=v+7,*q=v+6,*r=v;
5  *p=1,p[-1]=2,p[2]=3;//v[7]=1,v[6]=2,v[9]=3;
6  //p[3]=p[-8]=0;//访问越界
7  int res3=p-q,*res4=p-7,*res5=q+4,*res6=0;
8  //res3=1,res4=&v[0],res5=v+10 (末尾后指针)
9  //*res5=*res6=0;//末尾后指针, 空指针不可解引用
10 void*res7=res4;
11 // *res7,res7+1,res7-res7;
12 //void 类型指针不可解引用, 作指针算术
13 struct nd{int x,y;}*ptr=new nd,*pa=new nd[3];
14 ptr->x=(*ptr).y=1,pa[2]=*ptr,delete ptr;
15 delete[]pa;//new 对应 delete, new[] 对应 delete[]
16 //nd*ptr2=ptr;//复制无效指针值
17 int**pp=&p,*****m;//多级指针
```

目录

1	注释	8	字面量	11	语句
2	标识符	9	类型	12	函数
3	关键字	■ 基础类型		13	模板
4	声明	■ 引用类型		14	命名空间
5	对象	■ 数组类型		15	属性
6	引用	■ 函数类型		16	static_assert
7	变量	■ 枚举类型		17	预处理指令
		■ 类类型		18	常用 C 库
		■ 指针类型			
		■ 成员指针类型			
		■ cv 限定符			
		■ typedef			
		■ auto 和			
		decltype			
		10	表达式		

成员指针类型

- 成员指针是对象类型，用来存放非静态成员相对于该类对象的地址偏移量。

成员指针类型

- 成员指针是对象类型，用来存放非静态成员相对于该类对象的地址偏移量。
- 成员指针变量声明是声明符拥有下列形式的简单声明：

1 嵌套名说明符 * 属性(可选) cv限定符(可选) 声明符

成员指针类型

- 成员指针是对象类型，用来存放非静态成员相对于该类对象的地址偏移量。
- 成员指针变量声明是声明符拥有下列形式的简单声明：

1 嵌套名说明符 * 属性(可选) cv限定符(可选) 声明符

- 嵌套名说明符是名字和作用域解析运算符的序列，不能用括号将嵌套名说明符和星号分隔开。

成员指针类型

- 成员指针是对象类型，用来存放非静态成员相对于该类对象的地址偏移量。
- 成员指针变量声明是声明符拥有下列形式的简单声明：

1 嵌套名说明符 * 属性(可选) cv限定符(可选) 声明符

- 嵌套名说明符是名字和作用域解析运算符的序列，不能用括号将嵌套名说明符和星号分隔开。
- 如 `S C::* D` 将 `D` 声明为指向 `C` 的【声明说明符序列 `S` 所确定类型】的非静态数据成员的指针。

成员指针类型

- 成员指针是对象类型，用来存放非静态成员相对于该类对象的地址偏移量。
- 成员指针变量声明是声明符拥有下列形式的简单声明：

1 嵌套名说明符 * 属性(可选) cv限定符(可选) 声明符

- 嵌套名说明符是名字和作用域解析运算符的序列，不能用括号将嵌套名说明符和星号分隔开。
- 如 `S C::* D` 将 `D` 声明为指向 `C` 的【声明说明符序列 `S` 所确定类型】的非静态数据成员的指针。
- 同时使用嵌套名说明符与取地址可以获取成员指针。`&C::D` 获取一个指向 `C` 的非静态成员 `D` 的指针。

成员指针类型

- 使用成员指针来进行成员访问需要使用成员指针访问运算符`.*`或`->*`。前者的左操作数为该类型对象，后者的左操作数为该类型指针。注意这两个运算符的优先级较低。

成员指针类型

- 使用成员指针来进行成员访问需要使用成员指针访问运算符`.*`或`->*`。前者的左操作数为该类型对象，后者的左操作数为该类型指针。注意这两个运算符的优先级较低。
- 成员指针的值可以修改，可以为空成员指针。空成员指针不能作为成员指针访问运算符的右操作数，否则行为未定义。

成员指针类型

- 使用成员指针来进行成员访问需要使用成员指针访问运算符 `*` 或 `->*`。前者的左操作数为该类型对象，后者的左操作数为该类型指针。注意这两个运算符的优先级较低。
- 成员指针的值可以修改，可以为空成员指针。空成员指针不能作为成员指针访问运算符的右操作数，否则行为未定义。
- 指向一个可访问且无歧义的非虚基类的数据成员的指针，可以隐式转换成指向派生类的同一数据成员的指针，反之需要 `static_cast` 和显式转换来进行。

成员指针类型

- 使用成员指针来进行成员访问需要使用成员指针访问运算符 `*` 或 `->*`。前者的左操作数为该类型对象，后者的左操作数为该类型指针。注意这两个运算符的优先级较低。
- 成员指针的值可以修改，可以为空成员指针。空成员指针不能作为成员指针访问运算符的右操作数，否则行为未定义。
- 指向一个可访问且无歧义的非虚基类的数据成员的指针，可以隐式转换成指向派生类的同一数据成员的指针，反之需要 `static_cast` 和显式转换来进行。
- 当左操作数的动态类型并不包含该成员指针所指代的成员时，其行为未定义。

成员指针类型

- 使用成员指针来进行成员访问需要使用成员指针访问运算符 `.*` 或 `->*`。前者的左操作数为该类型对象，后者的左操作数为该类型指针。注意这两个运算符的优先级较低。
- 成员指针的值可以修改，可以为空成员指针。空成员指针不能作为成员指针访问运算符的右操作数，否则行为未定义。
- 指向一个可访问且无歧义的非虚基类的数据成员的指针，可以隐式转换成指向派生类的同一数据成员的指针，反之需要 `static_cast` 和显式转换来进行。
- 当左操作数的动态类型并不包含该成员指针所指代的成员时，其行为未定义。
- 与指针不同，成员指针不能转换到 `void` 类型指针。不存在类型为 `void` 的成员指针，因为不存在 `void` 类型的数据成员。

成员指针类型

- 使用成员指针来进行成员访问需要使用成员指针访问运算符 `.*` 或 `->*`。前者的左操作数为该类型对象，后者的左操作数为该类型指针。注意这两个运算符的优先级较低。
- 成员指针的值可以修改，可以为空成员指针。空成员指针不能作为成员指针访问运算符的右操作数，否则行为未定义。
- 指向一个可访问且无歧义的非虚基类的数据成员的指针，可以隐式转换成指向派生类的同一数据成员的指针，反之需要 `static_cast` 和显式转换来进行。
- 当左操作数的动态类型并不包含该成员指针所指代的成员时，其行为未定义。
- 与指针不同，成员指针不能转换到 `void` 类型指针。不存在类型为 `void` 的成员指针，因为不存在 `void` 类型的数据成员。
- 类成员可以是本身的成员指针，成员指针也可指向该成员指针。成员指针也可以指向作为成员的指针，指针也可以指向成员指针。

成员指针类型

■ 示例:

```
1 struct T{int a,b;int f(int c){return b+=c;}};
2 T x={1,2},*ptr=&x;
3 int T::*p=&T::a,(T::*q)(int)=&T::f;
4 //p,q 是指向 T 的成员 a,f 的成员指针
5 (x.*p)++,ptr->*p=0,(x.*q)(6),(ptr->*q)(7);
6 p=&T::b,(ptr->*p)++,(ptr->*&T::a)=7;
7 //成员指针可以修改,可以直接用 &T:: 得到的成员指针
8 //(x.*(int node::*))++;//右操作数为空
9 struct S{int f()const{return 1;}int S::*a,*b;};
10 int(S::*pS)()const=&S::f;//可带有 cv 限定
11 int*S::*pS2=&S::b,*S::*pS3=&pS2;
12 int S::*S::pS4=&S::a;
13 S g;(g.*pS)(),g.*pS2=new int,*(g.**pS3)=114;
14 g.*pS4=0,delete(g.**pS3);
15 //pS2 是指向 S::b 的成员指针, g.b 是指针
16 //pS3 是指向 int*S::* 类型的 pS2 的普通指针
17 //pS4 是指向 S::a 的成员指针, g.a 是成员指针
```

目录

1	注释	8	字面量	11	语句
2	标识符	9	类型	12	函数
3	关键字		■ 基础类型	13	模板
4	声明		■ 引用类型	14	命名空间
5	对象		■ 数组类型	15	属性
6	引用		■ 函数类型	16	static_assert
7	变量		■ 枚举类型	17	预处理指令
			■ 类类型	18	常用 C 库
			■ 指针类型		
			■ 成员指针类型		
			■ cv 限定符		
			■ typedef		
			■ auto 和		
			decltype		
		10	表达式		

cv 限定符

- cv 限定符: `const` 和 `volatile` 类型限定符, 可出现于任何类型说明符中, 以指定被声明对象或被命名类型的常量性 (constness) 或易变性 (volatility)。

cv 限定符

- cv 限定符: `const` 和 `volatile` 类型限定符, 可出现于任何类型说明符中, 以指定被声明对象或被命名类型的常量性 (constness) 或易变性 (volatility)。
- 对于除函数和引用外的每个类型 `T`, 有另外三个独立的类型: `const-限定的 T`、`volatile-限定的 T` 及 `const-volatile-限定的 T`。

cv 限定符

- cv 限定符: `const` 和 `volatile` 类型限定符, 可出现于任何类型说明符中, 以指定被声明对象或被命名类型的常量性 (constness) 或易变性 (volatility)。
- 对于除函数和引用外的每个类型 `T`, 有另外三个独立的类型: `const`-限定的 `T`、`volatile`-限定的 `T` 及 `const-volatile`-限定的 `T`。
- 下面两个在 OI 中不会用到, 这里只展开 `const`-限定的类型。

cv 限定符

- cv 限定符: `const` 和 `volatile` 类型限定符, 可出现于任何类型说明符中, 以指定被声明对象或被命名类型的常量性 (constness) 或易变性 (volatility)。
- 对于除函数和引用外的每个类型 `T`, 有另外三个独立的类型: `const`-限定的 `T`、`volatile`-限定的 `T` 及 `const-volatile`-限定的 `T`。
- 下面两个在 OI 中不会用到, 这里只展开 `const`-限定的类型。
- 对象最初被创建时所用的 cv 限定符决定对象的常量性或易变性。

cv 限定符

- cv 限定符: `const` 和 `volatile` 类型限定符, 可出现于任何类型说明符中, 以指定被声明对象或被命名类型的常量性 (constness) 或易变性 (volatility)。
- 对于除函数和引用外的每个类型 `T`, 有另外三个独立的类型: `const`-限定的 `T`、`volatile`-限定的 `T` 及 `const-volatile`-限定的 `T`。
- 下面两个在 OI 中不会用到, 这里只展开 `const`-限定的类型。
- 对象最初被创建时所用的 cv 限定符决定对象的常量性或易变性。
- `const` 对象是类型为 `const`-限定的对象, 或 `const` 对象的非 `mutable` 子对象。这种对象不能被修改, 强行修改 (如将地址转换到非 `const` 指针再该) 会导致未定义行为。

cv 限定符

- cv 限定符: `const` 和 `volatile` 类型限定符, 可出现于任何类型说明符中, 以指定被声明对象或被命名类型的常量性 (`constness`) 或易变性 (`volatility`)。
- 对于除函数和引用外的每个类型 `T`, 有另外三个独立的类型: `const`-限定的 `T`、`volatile`-限定的 `T` 及 `const-volatile`-限定的 `T`。
- 下面两个在 OI 中不会用到, 这里只展开 `const`-限定的类型。
- 对象最初被创建时所用的 cv 限定符决定对象的常量性或易变性。
- `const` 对象是类型为 `const`-限定的对象, 或 `const` 对象的非 `mutable` 子对象。这种对象不能被修改, 强行修改 (如将地址转换到非 `const` 指针再该) 会导致未定义行为。
- `mutable` 说明符用于指定不影响类的外部可观察状态的成员。在即便包含该类的对象被声明为 `const` 时仍可修改声明为 `mutable` 的类成员。

cv 限定符

- 称一种类型 S 比另一种（除 cv 限定外与 S 完全相同的）类型 T 具有更多 cv 限定，当且仅当 T 的 cv 限定是 S 的真子集。

cv 限定符

- 称一种类型 S 比另一种（除 cv 限定外与 S 完全相同的）类型 T 具有更多 cv 限定，当且仅当 T 的 cv 限定是 S 的真子集。
- 拥有更少 cv 限定的类型的指针/引用可以隐式转换为拥有更多 cv 限定的类型指针/引用，反之则不行，必须使用 `const_cast`。

cv 限定符

- 称一种类型 S 比另一种（除 cv 限定外与 S 完全相同的）类型 T 具有更多 cv 限定，当且仅当 T 的 cv 限定是 S 的真子集。
- 拥有更少 cv 限定的类型的指针/引用可以隐式转换为拥有更多 cv 限定的类型指针/引用，反之则不行，必须使用 `const_cast`。
- 对于 n 级指针类型（或成员指针及其与指针混合）， a, b ，若每一层的类型除 cv 限定外类型均相同，则 a 能够隐式转换为 b 的充要条件是：

cv 限定符

- 称一种类型 S 比另一种（除 cv 限定外与 S 完全相同的）类型 T 具有更多 cv 限定，当且仅当 T 的 cv 限定是 S 的真子集。
- 拥有更少 cv 限定的类型的指针/引用可以隐式转换为拥有更多 cv 限定的类型指针/引用，反之则不行，必须使用 `const_cast`。
- 对于 n 级指针类型（或成员指针及其与指针混合）， a, b ，若每一层的类型除 cv 限定外类型均相同，则 a 能够隐式转换为 b 的充要条件是：
 - 1 每一层， a 的 cv 限定完全包含 b 的 cv 限定符。

cv 限定符

- 称一种类型 S 比另一种（除 cv 限定外与 S 完全相同的）类型 T 具有更多 cv 限定，当且仅当 T 的 cv 限定是 S 的真子集。
- 拥有更少 cv 限定的类型的指针/引用可以隐式转换为拥有更多 cv 限定的类型指针/引用，反之则不行，必须使用 `const_cast`。
- 对于 n 级指针类型（或成员指针及其与指针混合）， a, b ，若每一层的类型除 cv 限定外类型均相同，则 a 能够隐式转换为 b 的充要条件是：
 - 1 每一层， a 的 cv 限定完全包含 b 的 cv 限定符。
 - 2 若在第 k 层， a 存在含 b 没有的 cv 限定符，则从 1 到 $k-1$ 的每一层， a 需要有 `const` 限定符。

cv 限定符

- 称一种类型 S 比另一种（除 cv 限定外与 S 完全相同的）类型 T 具有更多 cv 限定，当且仅当 T 的 cv 限定是 S 的真子集。
- 拥有更少 cv 限定的类型的指针/引用可以隐式转换为拥有更多 cv 限定的类型指针/引用，反之则不行，必须使用 `const_cast`。
- 对于 n 级指针类型（或成员指针及其与指针混合）， a, b ，若每一层的类型除 cv 限定外类型均相同，则 a 能够隐式转换为 b 的充要条件是：
 - 1 每一层， a 的 cv 限定完全包含 b 的 cv 限定符。
 - 2 若在第 k 层， a 存在含 b 没有的 cv 限定符，则从 1 到 $k-1$ 的每一层， a 需要有 `const` 限定符。
- `const` 变量定义时需要立即初始化，拥有常量初始化的 `const` 限定类型对象可以用作常量。

cv 限定符

■ 示例:

```
1  int a,*c=&a;const int b=7;
2  const int*d=c;//int* 转 const int*
3  c=const_cast<int*>(d);//反过来要 const_cast
4  /*d++;//const int* 不能修改所指对象
5  /*static_cast<int*>(&b)=1;//未定义行为
6  int**e=&c;const int**f;
7  //若允许 int** 到 const int** 的转换
8  //则可以突破 const 限定
9  //f=e,*f=&b,*e=1;//试图修改 b 的值
10 const int*const*g=e;
11 //允许 int** 转 const int*const*
12 struct node{int x:b;};//位域长度必须是常量表达式
13 struct nd{mutable int x;int y;};
14 const nd n1={1,2};//n1.y 是常量
15 n1.x++;//const 对象 mutable 子对象无 const 限定
```


目录

1	注释	8	字面量	11	语句
2	标识符	9	类型	12	函数
3	关键字		■ 基础类型	13	模板
4	声明		■ 引用类型	14	命名空间
5	对象		■ 数组类型	15	属性
6	引用		■ 函数类型	16	static_assert
7	变量		■ 枚举类型	17	预处理指令
			■ 类类型	18	常用 C 库
			■ 指针类型		
			■ 成员指针类型		
			■ cv 限定符		
			■ typedef		
			■ auto 和		
			decltype		
		10	表达式		

typedef

- typedef 说明符用于创建能在任何位置替代（可能复杂的）类型名的别名，而非创建变量或函数声明。

typedef

- typedef 说明符用于创建能在任何位置替代（可能复杂的）类型名的别名，而非创建变量或函数声明。
- typedef 声明中引入的每个标识符都成为 typedef 名，它是假如关键词 typedef 被移除时标识符本应成为的对象或函数的类型的同义词。

typedef

- typedef 说明符用于创建能在任何位置替代（可能复杂的）类型名的别名，而非创建变量或函数声明。
- typedef 声明中引入的每个标识符都成为 typedef 名，它是假如关键词 typedef 被移除时标识符本应成为的对象或函数的类型的同义词。
- 除类型说明符外，typedef 说明符不能与其他说明符组合（如 `constexpr`）。

typedef

- typedef 说明符用于创建能在任何位置替代（可能复杂的）类型名的别名，而非创建变量或函数声明。
- typedef 声明中引入的每个标识符都成为 typedef 名，它是假如关键词 typedef 被移除时标识符本应成为的对象或函数的类型的同义词。
- 除类型说明符外，typedef 说明符不能与其他说明符组合（如 constexpr）。
- C++11 起，也可以使用 using 别名声明来进行类似的操作。

typedef

■ 示例:

```
1  int f(){return 1;}
2  typedef long long ll;
3  ll a,*b;//a 是 long long, b 是 long long*
4  typedef int aN[],func();
5  aN s={1,1,4,5,1,4};//s 是 int[6]
6  func*pfunc=&f,&g=f;//函数指针, 引用
7  typedef struct type1{int x;}type2,*ptype;
8  type2 ta={1};ptype ptb=&ta;
9  //typedef static int I;//static 不是类型说明符
10 typedef const int*pI;
11 pI pa,pb;//pa,pb 均为 const int*
12 typedef const int&rI;
13 rI ra=a;//rI 是引用类型
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

- 基础类型
- 引用类型
- 数组类型
- 函数类型
- 枚举类型
- 类类型
- 指针类型
- 成员指针类型
- cv 限定符
- typedef

■ auto 和
decltype

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

auto 和 decltype

- auto 占位类型说明符（C++11 起）：对于变量，指定要从其初始化器自动推导出其类型。对于函数，指定要从其 return 语句推导出其返回类型。（C++14 起）

auto 和 decltype

- auto 占位类型说明符 (C++11 起): 对于变量, 指定要从其初始化器自动推导出其类型。对于函数, 指定要从其 return 语句推导出其返回类型。(C++14 起)
- 如果用占位类型说明符声明多个变量, 那么推导出的类型必须互相匹配。

auto 和 decltype

- auto 占位类型说明符（C++11 起）：对于变量，指定要从其初始化器自动推导出其类型。对于函数，指定要从其 return 语句推导出其返回类型。（C++14 起）
- 如果用占位类型说明符声明多个变量，那么推导出的类型必须互相匹配。
- 不允许在一个声明中混合 auto 的变量和函数。

auto 和 decltype

- auto 占位类型说明符（C++11 起）：对于变量，指定要从其初始化器自动推导出其类型。对于函数，指定要从其 return 语句推导出其返回类型。（C++14 起）
- 如果用占位类型说明符声明多个变量，那么推导出的类型必须互相匹配。
- 不允许在一个声明中混合 auto 的变量和函数。
- decltype 说明符（C++11 起）用于检查实体的声明类型，或表达式的类型和值类别。

auto 和 decltype

- auto 占位类型说明符 (C++11 起): 对于变量, 指定要从其初始化器自动推导出其类型。对于函数, 指定要从其 return 语句推导出其返回类型。(C++14 起)
- 如果用占位类型说明符声明多个变量, 那么推导出的类型必须互相匹配。
- 不允许在一个声明中混合 auto 的变量和函数。
- decltype 说明符 (C++11 起) 用于检查实体的声明类型, 或表达式的类型和值类别。
- 如果实参是没有括号的标识表达式或没有括号的类成员访问表达式, 那么 decltype 产生以该表达式命名的实体的类型。

auto 和 decltype

- auto 占位类型说明符 (C++11 起): 对于变量, 指定要从其初始化器自动推导出其类型。对于函数, 指定要从其 return 语句推导出其返回类型。(C++14 起)
- 如果用占位类型说明符声明多个变量, 那么推导出的类型必须互相匹配。
- 不允许在一个声明中混合 auto 的变量和函数。
- decltype 说明符 (C++11 起) 用于检查实体的声明类型, 或表达式的类型和值类别。
- 如果实参是没有括号的标识表达式或没有括号的类成员访问表达式, 那么 decltype 产生以该表达式命名的实体的类型。
- 如果实参是其他类型为 T 的任何表达式, 则根据表达式的值类别 (左值, 右值, 纯右值) 产生 (T&, T&&, T)。

auto 和 decltype

■ 示例:

```
1 auto i=1,*p=&i;//i 为 int, p 为 int*
2 const auto*p2=&i;//p2 为 const int*
3 auto f(int)->int;
4 auto(*g)(int)->auto=f;//g 为 int(*) (int)
5 decltype(i)a=i;//a 为 int
6 decltype((i))b=i;//b 为 int&
7 decltype(i+1)c=i+1;//c 为 int
8 decltype((int)i)d=(int)i;//d 为 int&&
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 `static_assert`

17 预处理指令

18 常用 C 库

表达式

- 表达式是运算符和它们的操作数的序列，它指定一项计算。

表达式

- 表达式是运算符和它们的操作数的序列，它指定一项计算。
- 表达式的求值可以产生一个结果，也可能产生副作用。

表达式

- 表达式是运算符和它们的操作数的序列，它指定一项计算。
- 表达式的求值可以产生一个结果，也可能产生副作用。
- 运算符是一种告诉编译器执行特定的数学或逻辑操作的符号。
C++ 内置了丰富的运算符，如下表所示：

常见运算符						
赋值	自增自减	算术	逻辑	比较	成员访问	其他
<code>a = b</code> <code>a += b</code> <code>a -= b</code> <code>a *= b</code> <code>a /= b</code> <code>a %= b</code> <code>a &= b</code> <code>a = b</code> <code>a ^= b</code> <code>a <<= b</code> <code>a >>= b</code>	<code>++a</code> <code>--a</code> <code>a++</code> <code>a--</code>	<code>+a</code> <code>-a</code> <code>a + b</code> <code>a - b</code> <code>a * b</code> <code>a / b</code> <code>a % b</code> <code>~a</code> <code>a & b</code> <code>a b</code> <code>a ^ b</code> <code>a << b</code> <code>a >> b</code>	<code>!a</code> <code>a&&b</code> <code>a b</code>	<code>a == b</code> <code>a != b</code> <code>a < b</code> <code>a > b</code> <code>a <= b</code> <code>a >= b</code>	<code>a[b]</code> <code>*a</code> <code>&a</code> <code>a->b</code> <code>a.b</code> <code>a->*b</code> <code>a.*b</code>	<code>a(...)</code> <code>a,b</code> <code>a?b:c</code>

表达式

■ 以及

特殊运算符

`static_cast` 转换一个类型为另一相关类型

`dynamic_cast` 在继承层级中转换

`const_cast` 添加或移除 `cv` 限定符

`reinterpret_cast` 转换类型到无关类型

`(type) C` 风格类型转换

`new` 创建有动态存储期的对象

`delete` 销毁先前由 `new` 表达式创建的对象，并释放其所拥有的内存区域

`sizeof` 查询类型的大小

`sizeof...` 查询形参包的大小(C++11 起)

`typeid` 查询类型的类型信息

`noexcept` 查询表达式是否能抛出异常(C++11 起)

`alignof` 查询类型的对齐要求(C++11 起)

表达式

- 对于一个表达式，C++ 根据运算符的优先级和结合顺序来分析各运算符与实参的绑定顺序。对于同一个实参，优先级更高的（对应表中编号更小的）运算符先结合，同级运算符根据结合顺序来判断。

表达式

- 对于一个表达式，C++ 根据运算符的优先级和结合顺序来分析各运算符与实参的绑定顺序。对于同一个实参，优先级更高的（对应表中编号更小的）运算符先结合，同级运算符根据结合顺序来判断。
- 下面列出部分运算符的优先级和结合顺序：

运算符的优先级							
优先级	运算符	描述	结合顺序	优先级	运算符	描述	结合顺序
1	::	作用域解析	从左到右	7	a<<b a>>b	逐位左移右移	从左到右
2	a++ a-- type() a(...) a[b] a.b a->b	后缀自增自减 函数风格转型 函数调用 下标 成员访问		8	a<b a<=b a>b a>=b	比较	
				9	a==b a!=b		
				10	a&b	逐位与	
				11	a^b	逐位异或	
3	++a --a +a -a !a ~a (type) *a &a sizeof	前缀自增自减 一元加减 逻辑非, 逐位非 类型转换 间接 (解引用) 取址 取大小	从右到左	12	a b	逐位或	从右到左
				13	a&&b	逻辑与	
				14	a b	逻辑或	
				15	a?:b:c throw a=b	三元条件 抛出异常 直接赋值	
	new delete new[] delete[]	动态内存分配			复合赋值		
4	a.*b a->*b	成员指针	从左到右	16	a,b	逗号表达式	从左到右
5	a*b a/b a%b	乘、除、求余					
6	a+b a-b	加法与减法					

表达式

- 特别的, ?: 运算的第二操作数如同带有括号, 不考虑其与?: 的优先级。sizeof 的操作数不能为 C 风格类型转换, sizeof(int)*p 解释为(sizeof(int))*p 。

表达式

- 特别的, `?:` 运算的第二操作数如同带有括号, 不考虑其与`?:` 的优先级。`sizeof` 的操作数不能为 C 风格类型转换, `sizeof(int)*p` 解释为`(sizeof(int))*p`。
- 优先级与结合顺序不影响表达式的求值顺序, 如 `a++ + b++ + c++` 中对 `a, b, c` 的操作顺序。确定求值顺序的规则是序列点规则 (C++11 前) 按顺序早于规则 (C++11 起)。

表达式

- 特别的, `?:` 运算的第二操作数如同带有括号, 不考虑其与`?:` 的优先级。`sizeof` 的操作数不能为 C 风格类型转换, `sizeof(int)*p` 解释为`(sizeof(int))*p`。
- 优先级与结合顺序不影响表达式的求值顺序, 如 `a++ + b++ + c++` 中对 `a, b, c` 的操作顺序。确定求值顺序的规则是序列点规则 (C++11 前) 按顺序早于规则 (C++11 起)。
- 若同一标量对象上的两副作用的顺序, 或标量对象上的副作用与【使用同一标量对象的先前值的值计算】顺序无法判断, 则其行为未定义。如 `m=++i+ ++i`, `n=++i+i`, `r=(i=1)-(i=1)` 等。

表达式

- 特别的, `?:` 运算的第二操作数如同带有括号, 不考虑其与 `?:` 的优先级。`sizeof` 的操作数不能为 C 风格类型转换, `sizeof(int)*p` 解释为 `(sizeof(int))*p`。
- 优先级与结合顺序不影响表达式的求值顺序, 如 `a++ + b++ + c++` 中对 `a, b, c` 的操作顺序。确定求值顺序的规则是序列点规则 (C++11 前) 按顺序早于规则 (C++11 起)。
- 若同一标量对象上的两副作用的顺序, 或标量对象上的副作用与【使用同一标量对象的先前值的值计算】顺序无法判断, 则其行为未定义。如 `m=++i+ ++i`, `n=++i+i`, `r=(i=1)-(i=1)` 等。
- 对于 `&&`, `||`, `?:` 运算符, 有特殊的规则: 左操作数若为真, 则不计算 `||` 的右操作数, `?:` 的第三操作数。若为假, 则不计算 `&&` 的右操作数, `?:` 的第二操作数。该性质称为短路求值。

表达式

- 特别的, `?:` 运算的第二操作数如同带有括号, 不考虑其与`?:` 的优先级。`sizeof` 的操作数不能为 C 风格类型转换, `sizeof(int)*p` 解释为`(sizeof(int))*p`。
- 优先级与结合顺序不影响表达式的求值顺序, 如 `a++ + b++ + c++` 中对 `a, b, c` 的操作顺序。确定求值顺序的规则是序列点规则 (C++11 前) 按顺序早于规则 (C++11 起)。
- 若同一标量对象上的两副作用的顺序, 或标量对象上的副作用与【使用同一标量对象的先前值的值计算】顺序无法判断, 则其行为未定义。如 `m=++i+ ++i`, `n=++i+i`, `r=(i=1)-(i=1)` 等。
- 对于`&&`, `||`, `?:` 运算符, 有特殊的规则: 左操作数若为真, 则不计算 `||` 的右操作数, `?:` 的第三操作数。若为假, 则不计算`&&` 的右操作数, `?:` 的第二操作数。该性质称为短路求值。
- 运算符 `typeid`、`sizeof`、`noexcept` 和 `decltype` (C++11 起) 的操作数是不求值表达式, 除非运算符为 `typeid` 而操作数是多态泛左值。程序不会真的去执行不求值表达式, 而只是求出其操作数的编译期性质。

表达式

■ 示例:

```
1  int a=1,b=0,c;  
2  c=a+b,c=a-b,c=a*b,c=a/b;  
3  c=sizeof(int)*a;//等同于 c=(sizeof(int))*a  
4  c=b?a%b:a;//正确, b 为 0 时不会计算 a%b  
5  decltype(a++)d=a;//a=1 不改变  
6  //d=a*a++;//未定义行为  
7  a=b?c,d:c=d;//a=(b?(c,d):(c=d))
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

- 表达式语句
- 声明语句
- 复合语句
- 选择语句
- 循环语句
- 跳转语句
- try 块
- 跳转语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

语句

- 语句（statement）是依序执行的 C++ 程序片段。任何函数体都是语句的序列。C++ 包含下列类型的语句：

语句

- 语句（statement）是依序执行的 C++ 程序片段。任何函数体都是语句的序列。C++ 包含下列类型的语句：
 - 1 表达式语句（expression statement）；

语句

- 语句 (statement) 是依序执行的 C++ 程序片段。任何函数体都是语句的序列。C++ 包含下列类型的语句：
 - 1 表达式语句 (expression statement);
 - 2 复合语句 (compound statement);

语句

- 语句 (statement) 是依序执行的 C++ 程序片段。任何函数体都是语句的序列。C++ 包含下列类型的语句：
 - 1 表达式语句 (expression statement);
 - 2 复合语句 (compound statement);
 - 3 选择语句 (selection statement);

语句

- 语句 (statement) 是依序执行的 C++ 程序片段。任何函数体都是语句的序列。C++ 包含下列类型的语句：
 - 1 表达式语句 (expression statement);
 - 2 复合语句 (compound statement);
 - 3 选择语句 (selection statement);
 - 4 循环语句 (iteration statement);

语句

- 语句 (statement) 是依序执行的 C++ 程序片段。任何函数体都是语句的序列。C++ 包含下列类型的语句：
 - 1 表达式语句 (expression statement);
 - 2 复合语句 (compound statement);
 - 3 选择语句 (selection statement);
 - 4 循环语句 (iteration statement);
 - 5 跳转语句 (jump statement);

语句

- 语句 (statement) 是依序执行的 C++ 程序片段。任何函数体都是语句的序列。C++ 包含下列类型的语句：

- 1 表达式语句 (expression statement);
- 2 复合语句 (compound statement);
- 3 选择语句 (selection statement);
- 4 循环语句 (iteration statement);
- 5 跳转语句 (jump statement);
- 6 声明语句 (declaration statement);

语句

- 语句 (statement) 是依序执行的 C++ 程序片段。任何函数体都是语句的序列。C++ 包含下列类型的语句：

- 1 表达式语句 (expression statement);
- 2 复合语句 (compound statement);
- 3 选择语句 (selection statement);
- 4 循环语句 (iteration statement);
- 5 跳转语句 (jump statement);
- 6 声明语句 (declaration statement);
- 7 try 块;

语句

- 语句 (statement) 是依序执行的 C++ 程序片段。任何函数体都是语句的序列。C++ 包含下列类型的语句：
 - 1 表达式语句 (expression statement);
 - 2 复合语句 (compound statement);
 - 3 选择语句 (selection statement);
 - 4 循环语句 (iteration statement);
 - 5 跳转语句 (jump statement);
 - 6 声明语句 (declaration statement);
 - 7 try 块;
- 标签是标识符，case 后加整数常量表达式以及 default 后随一个冒号。第一种标签可以置于任何语句之前，作用域为该函数内的所有部分，可以作为 goto 语句的目标。一个函数中的两个标签不得拥有同一标识符，但标签能与程序中任何其他实体拥有相同的名字。后两种必须位于 switch 语句内部，作为该 switch 语句跳转的目标。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

- 表达式语句

- 声明语句

- 复合语句

- 选择语句

- 循环语句

- 跳转语句

- try 块

- 跳转语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

表达式语句

- 表达式是表达式后随一个分号。

表达式语句

- 表达式是表达式后随一个分号。
- 执行到表达式语句时，程序对表达式进行计算，执行表达式产生的副作用，丢弃整个表达式的值。

表达式语句

- 表达式是表达式后随一个分号。
- 执行到表达式语句时，程序对表达式进行计算，执行表达式产生的副作用，丢弃整个表达式的值。
- 一般的 C++ 程序的大部分语句都是表达式语句，如赋值与函数调用。

表达式语句

- 表达式是表达式后随一个分号。
- 执行到表达式语句时，程序对表达式进行计算，执行表达式产生的副作用，丢弃整个表达式的值。
- 一般的 C++ 程序的大部分语句都是表达式语句，如赋值与函数调用。
- 没有表达式的表达式语句被称作空语句（null statement）。它可以用来为 for 或 while 循环提供空体，在复合语句的末尾引入标签。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

- 表达式语句

- **声明语句**

- 复合语句

- 选择语句

- 循环语句

- 跳转语句

- try 块

- 跳转语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

声明语句

- 声明语句是一句块声明，在块中引入一个或多个标识符。它可以是下列之一：

声明语句

- 声明语句是一句块声明，在块中引入一个或多个标识符。它可以是下列之一：

1 汇编声明

声明语句

- 声明语句是一句块声明，在块中引入一个或多个标识符。它可以是下列之一：
 - 1 汇编声明
 - 2 类型别名声明(C++11 起)

声明语句

- 声明语句是一句块声明，在块中引入一个或多个标识符。它可以是下列之一：
 - 1 汇编声明
 - 2 类型别名声明(C++11 起)
 - 3 命名空间别名定义

声明语句

- 声明语句是一句块声明，在块中引入一个或多个标识符。它可以是下列之一：
 - 1 汇编声明
 - 2 类型别名声明(C++11 起)
 - 3 命名空间别名定义
 - 4 **using 声明**

声明语句

- 声明语句是一句块声明，在块中引入一个或多个标识符。它可以是下列之一：

- 1 汇编声明
- 2 类型别名声明(C++11 起)
- 3 命名空间别名定义
- 4 using 声明
- 5 using 指令

声明语句

- 声明语句是一句块声明，在块中引入一个或多个标识符。它可以是下列之一：
 - 1 汇编声明
 - 2 类型别名声明(C++11 起)
 - 3 命名空间别名定义
 - 4 using 声明
 - 5 using 指令
 - 6 `static_assert` 声明(C++11 起)

声明语句

- 声明语句是一句块声明，在块中引入一个或多个标识符。它可以是下列之一：

- 1 汇编声明
- 2 类型别名声明(C++11 起)
- 3 命名空间别名定义
- 4 using 声明
- 5 using 指令
- 6 static_assert 声明(C++11 起)
- 7 不可见 enum 声明(C++11 起)

声明语句

- 声明语句是一句块声明，在块中引入一个或多个标识符。它可以是下列之一：

- 1 汇编声明
- 2 类型别名声明(C++11 起)
- 3 命名空间别名定义
- 4 using 声明
- 5 using 指令
- 6 static_assert 声明(C++11 起)
- 7 不可见 enum 声明(C++11 起)
- 8 简单声明

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

- 表达式语句

- 声明语句

- **复合语句**

- 选择语句

- 循环语句

- 跳转语句

- try 块

- 跳转语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

复合语句

- 复合语句是由大括号环绕的语句序列。

复合语句

- 复合语句是由大括号环绕的语句序列。
- 执行到复合语句时，程序会按顺序从上到下执行其中的语句。

复合语句

- 复合语句是由大括号环绕的语句序列。
- 执行到复合语句时，程序会按顺序从上到下执行其中的语句。
- 每个复合语句会引入一个块作用域，在块作用域中声明的自动存储期局部变量在闭花括号处以逆序销毁。

复合语句

- 复合语句是由大括号环绕的语句序列。
- 执行到复合语句时，程序会按顺序从上到下执行其中的语句。
- 每个复合语句会引入一个块作用域，在块作用域中声明的自动存储期局部变量在闭花括号处以逆序销毁。
- 当选择语句的分支语句，循环语句的循环体语句需要多条语句时，可以使用复合语句将多条语句并为一行。

复合语句

- 复合语句是由大括号环绕的语句序列。
- 执行到复合语句时，程序会按顺序从上到下执行其中的语句。
- 每个复合语句会引入一个块作用域，在块作用域中声明的自动存储期局部变量在闭花括号处以逆序销毁。
- 当选择语句的分支语句，循环语句的循环体语句需要多条语句时，可以使用复合语句将多条语句并为一条。
- 所有函数定义的函数体是复合语句，或者 `try` 块。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

- 表达式语句
- 声明语句
- 复合语句
- **选择语句**
- 循环语句
- 跳转语句
- try 块
- 跳转语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

选择语句

- 选择语句在数个控制流中选择一个。

选择语句

- 选择语句在数个控制流中选择一个。

- 选择语句的形式为：

```
1  属性(可选) if ( 条件 ) true分支语句      (1)
2  属性(可选) if ( 条件 ) true分支语句
3                      else false分支语句 (2)
4  属性(可选) switch ( 条件 ) switch语句的体 (3)
```

选择语句

- 选择语句在数个控制流中选择一个。

- 选择语句的形式为：

```
1      属性(可选) if ( 条件 ) true分支语句      (1)
2      属性(可选) if ( 条件 ) true分支语句
3                        else false分支语句 (2)
4      属性(可选) switch ( 条件 ) switch语句的体 (3)
```

- 其中条件是可按语境隐式转换为【(1)(2)：布尔类型(3)：整型或枚举类型】的表达式，或单个这种类型的非数组变量的带花括号或等号初始化器的声明（不可拥有 typedef 或存储类说明符）。

选择语句

- 选择语句在数个控制流中选择一个。

- 选择语句的形式为：

```
1      属性(可选) if ( 条件 ) true分支语句      (1)
2      属性(可选) if ( 条件 ) true分支语句
3                      else false分支语句 (2)
4      属性(可选) switch ( 条件 ) switch语句的体 (3)
```

- 其中条件是可按语境隐式转换为【(1)(2)：布尔类型(3)：整型或枚举类型】的表达式，或单个这种类型的非数组变量的带花括号或等号初始化器的声明（不可拥有 typedef 或存储类说明符）。
- true 分支语句，false 分支语句，switch 语句的体可以是任何语句，通常为复合语句。

选择语句

- 选择语句始终引入块作用域，在其中声明的变量对于外部不可见。

选择语句

- 选择语句始终引入块作用域，在其中声明的变量对于外部不可见。
- 对于 `if` 语句，当条件的值为真时，执行 `true` 分支语句。否则若存在 `else`，则执行 `false` 分支语句。

选择语句

- 选择语句始终引入块作用域，在其中声明的变量对于外部不可见。
- 对于 if 语句，当条件的值为真时，执行 true 分支语句。否则若存在 else，则执行 false 分支语句。
- 如果 if 语句存在 else，且其 true 分支语句也是 if 语句，则 true 分支语句也必须存在 else。

选择语句

- 选择语句始终引入块作用域，在其中声明的变量对于外部不可见。
- 对于 if 语句，当条件的值为真时，执行 true 分支语句。否则若存在 else，则执行 false 分支语句。
- 如果 if 语句存在 else，且其 true 分支语句也是 if 语句，则 true 分支语句也必须存在 else。
- 这保证了 `if(a)if(b)c else d` 的结构中的 else 与第二个 if 匹配。

选择语句

- 选择语句始终引入块作用域，在其中声明的变量对于外部不可见。
- 对于 if 语句，当条件的值为真时，执行 true 分支语句。否则若存在 else，则执行 false 分支语句。
- 如果 if 语句存在 else，且其 true 分支语句也是 if 语句，则 true 分支语句也必须存在 else。
- 这保证了 if(a)if(b)c else d 的结构中的 else 与第二个 if 匹配。
- 如果条件是声明，则其引入的变量作用域为两个分支的并，在两个分支都可见。如果分支语句是声明，则退出 if 语句时，由这个声明引入的自动存储期局部变量将销毁。

选择语句

- 选择语句始终引入块作用域，在其中声明的变量对于外部不可见。
- 对于 if 语句，当条件的值为真时，执行 true 分支语句。否则若存在 else，则执行 false 分支语句。
- 如果 if 语句存在 else，且其 true 分支语句也是 if 语句，则 true 分支语句也必须存在 else。
- 这保证了 if(a)if(b)c else d 的结构中的 else 与第二个 if 匹配。
- 如果条件是声明，则其引入的变量作用域为两个分支的并，在两个分支都可见。如果分支语句是声明，则退出 if 语句时，由这个声明引入的自动存储期局部变量将销毁。
- 如果控制流通过跳转语句或 longjmp 进入 true 分支语句，那么不执行 false 分支语句。

选择语句

- 对于 switch 语句，体中可以有任意数量的不同值的 case 常量表达式标签，以及最多一个 default 标签。

选择语句

- 对于 switch 语句，体中可以有任意数量的不同值的 case 常量表达式标签，以及最多一个 default 标签。
- 控制流到达 switch 语句时，如果条件求值为等于常量表达式中的一个，那么控制被转移到带有该 case 常量表达式标签的语句。

选择语句

- 对于 switch 语句，体中可以有任意数量的不同值的 case 常量表达式标签，以及最多一个 default 标签。
- 控制流到达 switch 语句时，如果条件求值为等于常量表达式中的一个，那么控制被转移到带有该 case 常量表达式标签的语句。
- 否则，若存在 default 标签，那么控制被转移到带有 default 标签的语句。

选择语句

- 对于 switch 语句，体中可以有任意数量的不同值的 case 常量表达式标签，以及最多一个 default 标签。
- 控制流到达 switch 语句时，如果条件求值为等于常量表达式中的一个，那么控制被转移到带有该 case 常量表达式标签的语句。
- 否则，若存在 default 标签，那么控制被转移到带有 default 标签的语句。
- 当 switch 语句的体中遇到 break 语句时退出 switch 语句。

选择语句

- 对于 switch 语句，体中可以有任意数量的不同值的 case 常量表达式标签，以及最多一个 default 标签。
- 控制流到达 switch 语句时，如果条件求值为等于常量表达式中的一个，那么控制被转移到带有该 case 常量表达式标签的语句。
- 否则，若存在 default 标签，那么控制被转移到带有 default 标签的语句。
- 当 switch 语句的体中遇到 break 语句时退出 switch 语句。
- 若 switch 语句的体中有 switch 语句，则内外的标签互不干扰。

选择语句

- 对于 switch 语句，体中可以有任意数量的不同值的 case 常量表达式标签，以及最多一个 default 标签。
- 控制流到达 switch 语句时，如果条件求值为等于常量表达式中的一个，那么控制被转移到带有该 case 常量表达式标签的语句。
- 否则，若存在 default 标签，那么控制被转移到带有 default 标签的语句。
- 当 switch 语句的体中遇到 break 语句时退出 switch 语句。
- 若 switch 语句的体中有 switch 语句，则内外的标签互不干扰。
- 控制转移时不允许向前跳过带有初始化器的变量声明，因此通常带有标签的都是复合语句。

选择语句

■ 示例:

```
1  int a=1,b=0,c=0;
2  if(a<2)b=true;else{b=false;}//b=true
3  if(a)
4      if(!b){
5          c=1;
6      }else c=2;//c=2
7  if(a){
8      if(!b){
9          c=3;
10     }
11 }else c=4;//c 不变
12 switch(4){
13     case 1:{c=0;break;}
14     case 2:case 3:{c=2;break;}
15     case 4:c=3;//没有 break
16     default:c=4;
17 }//c=4
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

- 表达式语句
- 声明语句
- 复合语句
- 选择语句
- 循环语句
- 跳转语句
- try 块
- 跳转语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

循环语句

- 循环语句重复执行一些代码。

循环语句

- 循环语句重复执行一些代码。

- 循环语句的形式为:

- 1 属性(可选) `while` (条件) 循环体 (1)
- 2 属性(可选) `do` 循环体 `while` (条件表达式) ; (2)
- 3 属性(可选) `for` (初始化语句 条件(可选) ;
- 4 迭代表达式(可选)) 循环体 (3)
- 5 属性(可选) `for` (范围变量声明 :
- 6 范围表达式) 循环体 (4) (C++11 起)

循环语句

- 循环语句重复执行一些代码。

- 循环语句的形式为：

```
1  属性(可选) while ( 条件 ) 循环体 (1)
2  属性(可选) do 循环体 while ( 条件表达式 ) ; (2)
3  属性(可选) for ( 初始化语句 条件(可选) ;
4  迭代表达式(可选) ) 循环体 (3)
5  属性(可选) for ( 范围变量声明 :
6  范围表达式 ) 循环体 (4) (C++11 起)
```

- 其中条件是可按语境隐式转换为布尔类型的表达式，或单个这种类型的非数组变量的带花括号或等号初始化器的声明（不可拥有 `typedef` 或存储类说明符）。

循环语句

- 循环语句重复执行一些代码。

- 循环语句的形式为：

```
1 属性(可选) while ( 条件 ) 循环体 (1)
2 属性(可选) do 循环体 while ( 条件表达式 ) ; (2)
3 属性(可选) for ( 初始化语句 条件(可选) ;
4   迭代表达式(可选) ) 循环体 (3)
5 属性(可选) for ( 范围变量声明 :
6   范围表达式 ) 循环体 (4) (C++11 起)
```

- 其中条件是可按语境隐式转换为布尔类型的表达式，或单个这种类型的非数组变量的带花括号或等号初始化器的声明（不可拥有 typedef 或存储类说明符）。

- 条件表达式是可按语境隐式转换为布尔类型的表达式。

循环语句

- 循环语句重复执行一些代码。

- 循环语句的形式为：

```
1  属性(可选) while ( 条件 ) 循环体 (1)
2  属性(可选) do 循环体 while ( 条件表达式 ) ; (2)
3  属性(可选) for ( 初始化语句 条件(可选) ;
4  迭代表达式(可选) ) 循环体 (3)
5  属性(可选) for ( 范围变量声明 :
6  范围表达式 ) 循环体 (4) (C++11 起)
```

- 其中条件是可按语境隐式转换为布尔类型的表达式，或单个这种类型的非数组变量的带花括号或等号初始化器的声明（不可拥有 typedef 或存储类说明符）。
- 条件表达式是可按语境隐式转换为布尔类型的表达式。
- 初始化语句是简单声明或表达式语句。可以发现，初始化语句末尾一定带有分号。

循环语句

- 循环语句重复执行一些代码。

- 循环语句的形式为：

```
1  属性(可选) while ( 条件 ) 循环体 (1)
2  属性(可选) do 循环体 while ( 条件表达式 ) ; (2)
3  属性(可选) for ( 初始化语句 条件(可选) ;
4  迭代表达式(可选) ) 循环体 (3)
5  属性(可选) for ( 范围变量声明 :
6  范围表达式 ) 循环体 (4) (C++11 起)
```

- 其中条件是可按语境隐式转换为布尔类型的表达式，或单个这种类型的非数组变量的带花括号或等号初始化器的声明（不可拥有 typedef 或存储类说明符）。
- 条件表达式是可按语境隐式转换为布尔类型的表达式。
- 初始化语句是简单声明或表达式语句。可以发现，初始化语句末尾一定带有分号。
- 迭代表达式是任意表达式，循环体是任意语句，通常为复合语句。

循环语句

- 循环语句始终引入块作用域，在其中声明的变量对于外部不可见。

循环语句

- 循环语句始终引入块作用域，在其中声明的变量对于外部不可见。
- 控制流到达循环体末尾时，会销毁在循环体及条件中声明的自动存储期局部变量。

循环语句

- 循环语句始终引入块作用域，在其中声明的变量对于外部不可见。
- 控制流到达循环体末尾时，会销毁在循环体及条件中声明的自动存储期局部变量。
- 在循环体中遇到 `break` 语句则立刻终止循环，遇到 `continue` 语句则跳转到循环体末尾继续执行。

循环语句

- 循环语句始终引入块作用域，在其中声明的变量对于外部不可见。
- 控制流到达循环体末尾时，会销毁在循环体及条件中声明的自动存储期局部变量。
- 在循环体中遇到 `break` 语句则立刻终止循环，遇到 `continue` 语句则跳转到循环体末尾继续执行。
- 如果没有可观察行为（不调用 I/O 函数，访问 `volatile` 对象，或进行原子或同步操作）的循环不会终止，那么它的行为未定义。编译器可能会移除这种循环。

循环语句

- 循环语句始终引入块作用域，在其中声明的变量对于外部不可见。
- 控制流到达循环体末尾时，会销毁在循环体及条件中声明的自动存储期局部变量。
- 在循环体中遇到 `break` 语句则立刻终止循环，遇到 `continue` 语句则跳转到循环体末尾继续执行。
- 如果没有可观察行为（不调用 I/O 函数，访问 `volatile` 对象，或进行原子或同步操作）的循环不会终止，那么它的行为未定义。编译器可能会移除这种循环。
- 对于 `while` 循环，每次先对条件进行求值，若为假则终止循环。否则执行循环体，到达末尾后重新回到 `while` 语句开始。

循环语句

- 循环语句始终引入块作用域，在其中声明的变量对于外部不可见。
- 控制流到达循环体末尾时，会销毁在循环体及条件中声明的自动存储期局部变量。
- 在循环体中遇到 `break` 语句则立刻终止循环，遇到 `continue` 语句则跳转到循环体末尾继续执行。
- 如果没有可观察行为（不调用 I/O 函数，访问 `volatile` 对象，或进行原子或同步操作）的循环不会终止，那么它的行为未定义。编译器可能会移除这种循环。
- 对于 `while` 循环，每次先对条件进行求值，若为假则终止循环。否则执行循环体，到达末尾后重新回到 `while` 语句开始。
- 对于 `do-while` 循环，先执行循环体，再对条件表达式求值，若为假则终止循环。否则回到 `do-while` 语句开头再次执行。
`do-while` 语句保证至少执行循环体一次。

循环语句

- 对于 for 循环，先执行初始化语句，然后对条件进行求值（不存在默认为真），若为假则终止循环。否则执行循环体，到达末尾后计算迭代表达式（若存在），然后重新回到条件开始（而非初始化语句）

循环语句

- 对于 for 循环，先执行初始化语句，然后对条件进行求值（不存在默认为真），若为假则终止循环。否则执行循环体，到达末尾后计算迭代表达式（若存在），然后重新回到条件开始（而非初始化语句）
- 初始化语句该语句若为声明，则其作用域为 for 语句全体。若声明了自动存储期局部变量，则在终止循环后销毁，而不是每次进入循环体都销毁。

循环语句

- 对于 for 循环，先执行初始化语句，然后对条件进行求值（不存在默认为真），若为假则终止循环。否则执行循环体，到达末尾后计算迭代表达式（若存在），然后重新回到条件开始（而非初始化语句）
- 初始化语句该语句若为声明，则其作用域为 for 语句全体。若声明了自动存储期局部变量，则在终止循环后销毁，而不是每次进入循环体都销毁。
- 对于基于范围的 for 循环(4):

循环语句

- 对于 for 循环，先执行初始化语句，然后对条件进行求值（不存在默认为真），若为假则终止循环。否则执行循环体，到达末尾后计算迭代表达式（若存在），然后重新回到条件开始（而非初始化语句）
- 初始化语句该语句若为声明，则其作用域为 for 语句全体。若声明了自动存储期局部变量，则在终止循环后销毁，而不是每次进入循环体都销毁。
- 对于基于范围的 for 循环(4):
- 范围变量声明是一个具名变量的声明，它的类型是由范围表达式所表示的序列的元素的类型，或该类型的引用。

循环语句

- 对于 for 循环，先执行初始化语句，然后对条件进行求值（不存在默认为真），若为假则终止循环。否则执行循环体，到达末尾后计算迭代表达式（若存在），然后重新回到条件开始（而非初始化语句）
- 初始化语句该语句若为声明，则其作用域为 for 语句全体。若声明了自动存储期局部变量，则在终止循环后销毁，而不是每次进入循环体都销毁。
- 对于基于范围的 for 循环(4):
- 范围变量声明是一个具名变量的声明，它的类型是由范围表达式所表示的序列的元素的类型，或该类型的引用。
- 范围表达式可以表示一个合适的序列（数组，或定义了 begin 和 end 成员函数或自由函数的对象）的任意表达式，或一个花括号初始化器列表。

循环语句

- 对于 for 循环，先执行初始化语句，然后对条件进行求值（不存在默认为真），若为假则终止循环。否则执行循环体，到达末尾后计算迭代表达式（若存在），然后重新回到条件开始（而非初始化语句）
- 初始化语句该语句若为声明，则其作用域为 for 语句全体。若声明了自动存储期局部变量，则在终止循环后销毁，而不是每次进入循环体都销毁。
- 对于基于范围的 for 循环(4):
- 范围变量声明是一个具名变量的声明，它的类型是由范围表达式所表示的序列的元素的类型，或该类型的引用。
- 范围表达式可以表示一个合适的序列（数组，或定义了 begin 和 end 成员函数或自由函数的对象）的任意表达式，或一个花括号初始化器列表。
- 控制流到达基于范围的 for 循环时，让范围变量遍历范围表达式，每次执行循环体，范围变量在结束循环体后销毁重建。

循环语句

■ 示例:

```
1  int sum=0;
2  for(int i=1;i<=100;i++)sum+=i;//sum=5050
3  //sum+=i;//i 不在 for 语句外可见
4  while(sum--);//sum=-1
5  sum=5050;while(sum)sum--;//sum=0
6  sum=5050;do sum--;while(sum);//sum=0
7  sum=0;do{sum++;}while(0);//sum=1
8  int a[]={1,2,4,8,16,32,64};
9  for(auto x:a)sum+=x;//sum=128
10 for(auto&y:a)y--;//a 每一项减少 1
11 //for(;;sum=sum+0)sum=-sum;
12 //死循环, 未定义行为
```


目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

- 表达式语句
- 声明语句
- 复合语句
- 选择语句
- 循环语句
- 跳转语句
- try 块
- 跳转语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

跳转语句

- 跳转语句无条件地转移控制流。

跳转语句

- 跳转语句无条件地转移控制流。

- 跳转语句的形式为：

- 1 属性(可选) `break` ; (1)
- 2 属性(可选) `continue` ; (2)
- 3 属性(可选) `return` 表达式(可选) ; (3)
- 4 属性(可选) `return` 花括号初始化器列表 ; (4)(C++11 起)
- 5 属性(可选) `goto` 标识符 ; (5)

跳转语句

- 跳转语句无条件地转移控制流。

- 跳转语句的形式为：

- 1 属性(可选) `break` ; (1)
- 2 属性(可选) `continue` ; (2)
- 3 属性(可选) `return` 表达式(可选) ; (3)
- 4 属性(可选) `return` 花括号初始化器列表 ; (4)(C++11 起)
- 5 属性(可选) `goto` 标识符 ; (5)

- `break` 语句导致外围的循环语句或 `switch` 语句终止。

跳转语句

- 跳转语句无条件地转移控制流。

- 跳转语句的形式为：

- 1 属性(可选) `break` ; (1)
- 2 属性(可选) `continue` ; (2)
- 3 属性(可选) `return` 表达式(可选) ; (3)
- 4 属性(可选) `return` 花括号初始化器列表 ; (4)(C++11 起)
- 5 属性(可选) `goto` 标识符 ; (5)

- `break` 语句导致外围的循环语句或 `switch` 语句终止。

- `continue` 语句导致循环语句循环体的剩余部分被跳过。

跳转语句

- 跳转语句无条件地转移控制流。

- 跳转语句的形式为：

```
1 属性(可选) break ; (1)
2 属性(可选) continue ; (2)
3 属性(可选) return 表达式(可选) ; (3)
4 属性(可选) return 花括号初始化器列表 ; (4)(C++11 起)
5 属性(可选) goto 标识符 ; (5)
```

- break 语句导致外围的循环语句或 switch 语句终止。
- continue 语句导致循环语句循环体的剩余部分被跳过。
- return 语句终止当前函数并返回指定值（若存在）给调用方。

跳转语句

- 跳转语句无条件地转移控制流。

- 跳转语句的形式为：

```
1 属性(可选) break ; (1)
2 属性(可选) continue ; (2)
3 属性(可选) return 表达式(可选) ; (3)
4 属性(可选) return 花括号初始化器列表 ; (4)(C++11 起)
5 属性(可选) goto 标识符 ; (5)
```

- break 语句导致外围的循环语句或 switch 语句终止。
- continue 语句导致循环语句循环体的剩余部分被跳过。
- return 语句终止当前函数并返回指定值（若存在）给调用方。
- goto 语句无条件转移控制流到带有该标识符标签的语句。

跳转语句

- `return` 对表达式求值，终止当前函数，并将表达式隐式转换到函数返回类型（可以是 `void`）后的结果返回给调用方。表达式在返回类型是（可有 `cv` 限定的）`void` 的函数中可以不出现，在构造函数中和析构函数中不能出现。

跳转语句

- `return` 对表达式求值，终止当前函数，并将表达式隐式转换到函数返回类型（可以是 `void`）后的结果返回给调用方。表达式在返回类型是（可有 `cv` 限定的）`void` 的函数中可以不出现，在构造函数中和析构函数中不能出现。
- 如果控制流抵达返回（可有 `cv` 限定的）`void` 的函数结尾、构造函数结尾、析构函数结尾或返回（可有 `cv` 限定的）`void` 的函数的函数 `try` 块结尾，且没有 `return` 语句，那么执行 `return;`。

跳转语句

- `return` 对表达式求值，终止当前函数，并将表达式隐式转换到函数返回类型（可以是 `void`）后的结果返回给调用方。表达式在返回类型是（可有 `cv` 限定的）`void` 的函数中可以不出现，在构造函数中和析构函数中不能出现。
- 如果控制流抵达返回（可有 `cv` 限定的）`void` 的函数结尾、构造函数结尾、析构函数结尾或返回（可有 `cv` 限定的）`void` 的函数的函数 `try` 块结尾，且没有 `return` 语句，那么执行 `return;`。
- 如果控制流抵达 `main` 函数的结尾，那么执行 `return 0;`。除了 `main` 函数外，如果控制流抵达有返回值的函数的结尾却没有 `return` 语句，是未定义行为。

跳转语句

- `return` 对表达式求值，终止当前函数，并将表达式隐式转换到函数返回类型（可以是 `void`）后的结果返回给调用方。表达式在返回类型是（可有 `cv` 限定的）`void` 的函数中可以不出现，在构造函数中和析构函数中不能出现。
- 如果控制流抵达返回（可有 `cv` 限定的）`void` 的函数结尾、构造函数结尾、析构函数结尾或返回（可有 `cv` 限定的）`void` 的函数的函数 `try` 块结尾，且没有 `return` 语句，那么执行 `return;`。
- 如果控制流抵达 `main` 函数的结尾，那么执行 `return 0;`。除了 `main` 函数外，如果控制流抵达有返回值的函数的结尾却没有 `return` 语句，是未定义行为。
- 当函数返回类型是占位类型是，会根据 `return` 值自动推导它的具体类型。

跳转语句

- goto 无条件转移控制流到带有该标识符标签的语句。

跳转语句

- goto 无条件转移控制流到带有该标识符标签的语句。
- goto 必须与该标识符在同一函数。跨函数跳转使用 longjmp 函数。

跳转语句

- goto 无条件转移控制流到带有该标识符标签的语句。
- goto 必须与该标识符在同一函数。跨函数跳转使用 longjmp 函数。
- 如果控制的转移退出了任何自动存储期局部变量的作用域，那么为所有退出作用域的自动存储期局部变量以其构造顺序的逆序析构。

跳转语句

- goto 无条件转移控制流到带有该标识符标签的语句。
- goto 必须与该标识符在同一函数。跨函数跳转使用 longjmp 函数。
- 如果控制的转移退出了任何自动存储期局部变量的作用域，那么为所有退出作用域的自动存储期局部变量以其构造顺序的逆序析构。
- goto 不能将控制转移到 try 块或 catch 子句之内，但能将控制转移离开 try 块或 catch 子句。

跳转语句

- goto 无条件转移控制流到带有该标识符标签的语句。
- goto 必须与该标识符在同一函数。跨函数跳转使用 longjmp 函数。
- 如果控制的转移退出了任何自动存储期局部变量的作用域，那么为所有退出作用域的自动存储期局部变量以其构造顺序的逆序析构。
- goto 不能将控制转移到 try 块或 catch 子句之内，但能将控制转移离开 try 块或 catch 子句。
- goto 不能将控制转移进入任何自动存储期局部变量的作用域，除非该变量的类型为标量类型或拥有平凡默认构造函数和析构函数的类类型或这两种类型的 cv 限定版本或这以上三种类型的数组，且声明不带初始化器。

跳转语句

- goto 无条件转移控制流到带有该标识符标签的语句。
- goto 必须与该标识符在同一函数。跨函数跳转使用 longjmp 函数。
- 如果控制的转移退出了任何自动存储期局部变量的作用域，那么为所有退出作用域的自动存储期局部变量以其构造顺序的逆序析构。
- goto 不能将控制转移到 try 块或 catch 子句之内，但能将控制转移离开 try 块或 catch 子句。
- goto 不能将控制转移进入任何自动存储期局部变量的作用域，除非该变量的类型为标量类型或拥有平凡默认构造函数和析构函数的类类型或这两种类型的 cv 限定版本或这以上三种类型的数组，且声明不带初始化器。
- 对于 switch 语句的跳转，遵循同样的规则。

跳转语句

■ 示例:

```
1 void p1(){}//自动 return;
2 void p2(int){return p1();}//可以 return void 类型
3 auto func(int a)->auto{return~a;}//返回 int 类型
4 int main(){
5     p1(),p2(func(1));while(1)break;
6     int p=0;
7     to:int sum=0;
8     for(int i=1;i<=100;i++){
9         if(i==50)continue;
10        sum+=i;
11    }//sum=5000
12    if(!p){p=1;goto to;}//回到 to;
13    //goto to2;//不能进入 ss 作用域
14    goto to3;///能进入 sr 作用域
15    int sr;to3:
16    int ss=0;to2:;
17 }//自动 return 0;
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

- 表达式语句
- 声明语句
- 复合语句
- 选择语句
- 循环语句
- 跳转语句
- **try 块**
- 跳转语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

try 块

- try 块提供当执行其他语句时捕获其所抛出的异常的能力。

try 块

- try 块提供当执行其他语句时捕获其所抛出的异常的能力。
- try 块的形式为:

1 try 复合语句 处理块序列

try 块

- try 块提供当执行其他语句时捕获其所抛出的异常的能力。
- try 块的形式为:

1 try 复合语句 处理块序列

- 其中处理块序列是一或多个处理块的序列，处理块的形式为:

1 catch (属性(可选) 类型说明符序列 声明符) 复合
 语句 (1)
2 catch (属性(可选) 类型说明符序列 抽象声明符(可
 选)) 复合语句 (2)
3 catch (...) 复合语句 (3)

try 块

- try 块提供当执行其他语句时捕获其所抛出的异常的能力。
- try 块的形式为:

1 try 复合语句 处理块序列

- 其中处理块序列是一或多个处理块的序列，处理块的形式为:

```
1     catch ( 属性(可选) 类型说明符序列 声明符 ) 复合  
      语句 (1)  
2     catch ( 属性(可选) 类型说明符序列 抽象声明符(可  
      选) ) 复合语句 (2)  
3     catch ( ... ) 复合语句 (3)
```

- 其中，(1)/(2)声明一个具名/无名形参的 catch 子句。(3)可被任何异常激活，若存在，必须是处理块序列的最后一个。

try 块

- try 块提供当执行其他语句时捕获其所抛出的异常的能力。

try 块

- try 块提供当执行其他语句时捕获其所抛出的异常的能力。
- try 块的形式为:

1 try 复合语句 处理块序列

try 块

- try 块提供当执行其他语句时捕获其所抛出的异常的能力。
- try 块的形式为:

1 try 复合语句 处理块序列

- 其中处理块序列是一或多个处理块的序列，处理块的形式为:

1 catch (属性(可选) 类型说明符序列 声明符) 复合
 语句 (1)
2 catch (属性(可选) 类型说明符序列 抽象声明符(可
 选)) 复合语句 (2)
3 catch (...) 复合语句 (3)

try 块

- try 块提供当执行其他语句时捕获其所抛出的异常的能力。
- try 块的形式为:

1 try 复合语句 处理块序列

- 其中处理块序列是一或多个处理块的序列，处理块的形式为:

```
1     catch ( 属性(可选) 类型说明符序列 声明符 ) 复合  
      语句 (1)  
2     catch ( 属性(可选) 类型说明符序列 抽象声明符(可  
      选) ) 复合语句 (2)  
3     catch ( ... ) 复合语句 (3)
```

- 其中，(1)/(2)声明一个具名/无名形参的 catch 子句。(3)可被任何异常激活，若存在，必须是处理块序列的最后一个。

try 块

- 通过 `throw` 表达式可以抛出异常对象，异常对象的值是表达式的值。

try 块

- 通过 throw 表达式可以抛出异常对象，异常对象的值是表达式的值。
- 抛出异常时，控制流立即反向（沿调用栈向上）直到它抵达一个 try 块的起点。这称为栈回溯，此过程中会销毁从进入 try 块之后声明的的自动存储期局部变量。

try 块

- 通过 `throw` 表达式可以抛出异常对象，异常对象的值是表达式的值。
- 抛出异常时，控制流立即反向（沿调用栈向上）直到它抵达一个 `try` 块的起点。这称为栈回溯，此过程中会销毁从进入 `try` 块之后声明的自动存储期局部变量。
- `try` 块中的复合语句中的任何语句抛出异常时，根据异常对象的类型，找到第一个匹配的 `catch` 子句进入。找不到则继续栈回溯。

try 块

- 通过 `throw` 表达式可以抛出异常对象，异常对象的值是表达式的值。
- 抛出异常时，控制流立即反向（沿调用栈向上）直到它抵达一个 `try` 块的起点。这称为栈回溯，此过程中会销毁从进入 `try` 块之后声明的自动存储期局部变量。
- `try` 块中的复合语句中的任何语句抛出异常时，根据异常对象的类型，找到第一个匹配的 `catch` 子句进入。找不到则继续栈回溯。
- `catch` 子句中可以通过不带表达式的 `throw` 来重抛异常，让外围的 `try` 块来处理。

try 块

- 通过 `throw` 表达式可以抛出异常对象，异常对象的值是表达式的值。
- 抛出异常时，控制流立即反向（沿调用栈向上）直到它抵达一个 `try` 块的起点。这称为栈回溯，此过程中会销毁从进入 `try` 块之后声明的自动存储期局部变量。
- `try` 块中的复合语句中的任何语句抛出异常时，根据异常对象的类型，找到第一个匹配的 `catch` 子句进入。找不到则继续栈回溯。
- `catch` 子句中可以通过不带表达式的 `throw` 来重抛异常，让外围的 `try` 块来处理。
- 如果一个函数声明带有 `noexcept` 说明符，则该函数不会抛出异常。每当抛出异常且对处理块的查找遇到了不会抛出的函数的最外层块时，程序直接异常退出。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

- 表达式语句
- 声明语句
- 复合语句
- 选择语句
- 循环语句
- 跳转语句
- try 块
- 跳转语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

跳转语句

■ 示例:

```
1  int f()noexcept{
2      try{
3          throw 11;
4      }catch(char*){
5          return 1;
6      }
7      catch(int a){
8          return a;
9      }catch(...){
10         throw;//重抛异常
11     }
12 }//return 11;不抛出异常
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

- 默认实参
- 友元函数
- 函数重载
- 运算符重载
- 函数对象
- 非静态成员函

数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

函数

- 函数 (function) 是把一个语句序列 (函数体, function body) 关联到一个名字和零或更多个函数形参 (function parameter) 的列表的 C++ 实体。

函数

- 函数 (function) 是把一个语句序列 (函数体, function body) 关联到一个名字和零或更多个函数形参 (function parameter) 的列表的 C++ 实体。
- 函数定义将函数名/类型与函数体关联。非成员函数的定义只能出现在命名空间作用域中 (不存在嵌套函数)。成员函数的定义亦可出现在类定义的体内。

函数

- 函数 (function) 是把一个语句序列 (函数体, function body) 关联到一个名字和零或多个函数形参 (function parameter) 的列表的 C++ 实体。
- 函数定义将函数名/类型与函数体关联。非成员函数的定义只能出现在命名空间作用域中 (不存在嵌套函数)。成员函数的定义亦可出现在类定义的体内。

- 函数定义的形式为:

- 1 属性 (可选) 声明说明符序列 (可选) 声明符 虚声明符
 序列 (可选) 函数体

函数

- 函数 (function) 是把一个语句序列 (函数体, function body) 关联到一个名字和零或更多个函数形参 (function parameter) 的列表的 C++ 实体。
- 函数定义将函数名/类型与函数体关联。非成员函数的定义只能出现在命名空间作用域中 (不存在嵌套函数)。成员函数的定义亦可出现在类定义的体内。
- 函数定义的形式为:

```
1      属性(可选) 声明说明符序列(可选) 声明符 虚声明符  
      序列(可选) 函数体
```

- 可以看出, 函数定义只比函数声明多了一个函数体。函数体的形式为:

```
1      构造函数初始化器(可选) 复合语句 (1)  
2      构造函数初始化器(可选) try-块 (2)  
3      = delete ; (3)(C++11 起)  
4      = default ; (4)(C++11 起)
```

函数

- 其中构造函数初始化器只能用于构造函数，(4)只能用于特殊成员函数。

函数

- 其中构造函数初始化器只能用于构造函数，(4)只能用于特殊成员函数。
- 对于(1)，调用函数时，以各个实际参数（由调用场所提供，或默认）来初始化各个形参，然后执行函数体中的语句。

函数

- 其中构造函数初始化器只能用于构造函数，(4)只能用于特殊成员函数。
- 对于(1)，调用函数时，以各个实际参数（由调用场所提供，或默认）来初始化各个形参，然后执行函数体中的语句。
- (2) 主要用于接受构造函数初始化成员时抛出的异常，除此之外与在(1) 中直接写 try-块等价。

函数

- 其中构造函数初始化器只能用于构造函数，(4)只能用于特殊成员函数。
- 对于(1)，调用函数时，以各个实际参数（由调用场所提供，或默认）来初始化各个形参，然后执行函数体中的语句。
- (2) 主要用于接受构造函数初始化成员时抛出的异常，除此之外与在(1) 中直接写 try-块等价。
- (3) 用于定义函数为弃置，不允许使用弃置函数，否则编译错误。

函数

- 其中构造函数初始化器只能用于构造函数，(4)只能用于特殊成员函数。
- 对于(1)，调用函数时，以各个实际参数（由调用场所提供，或默认）来初始化各个形参，然后执行函数体中的语句。
- (2) 主要用于接受构造函数初始化成员时抛出的异常，除此之外与在(1) 中直接写 try-块等价。
- (3) 用于定义函数为弃置，不允许使用弃置函数，否则编译错误。
- 一些成员函数是特殊的，即使用户不定义编译器也会定义它们。
(4)显式定义特殊成员函数为默认。

函数

- 其中构造函数初始化器只能用于构造函数，(4)只能用于特殊成员函数。
- 对于(1)，调用函数时，以各个实际参数（由调用场所提供，或默认）来初始化各个形参，然后执行函数体中的语句。
- (2) 主要用于接受构造函数初始化成员时抛出的异常，除此之外与在(1) 中直接写 try-块等价。
- (3) 用于定义函数为弃置，不允许使用弃置函数，否则编译错误。
- 一些成员函数是特殊的，即使用户不定义编译器也会定义它们。
(4)显式定义特殊成员函数为默认。
- 使用函数调用运算符（括号）可以调用函数，括号内写实际参数。
一般的函数体内可以调用自身，这称为递归。

函数

■ 示例:

```
1  int gcd(int a,int b){return b?gcd(b,a%b):a;}
2  //{}部分是函数体, gcd(b,a%b)是函数调用
3  //这个函数体内调用自身, 当 b 为 0 时层层返回, 得到结果
4  struct node{
5      int x,y;
6      node():x(),y(0){}//:x(),y(0) 是构造函数初始化器
7      ~node()=default;//定义析构函数为默认
8      void print()const=delete;//定义 print 为弃置
9      int getans()const{
10         return gcd(x,y);//执行到此处时, 调用 gcd 函数
11     }
12 };
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

- 默认实参

- 友元函数

- 函数重载

- 运算符重载

- 函数对象

- 非静态成员函

数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

默认实参

- 允许调用函数时不提供一个或多个尾部的实参。通过为函数声明的形参列表中的形参使用初始化器来指定默认实参。

默认实参

- 允许调用函数时不提供一个或多个尾部的实参。通过为函数声明的形参列表中的形参使用初始化器来指定默认实参。
- 拥有默认实参的形参之后的形参必须拥有在这个或同一作用域中先前的声明中所提供的默认实参，除非该形参是从某个形参包展开得到的。

默认实参

- 允许调用函数时不提供一个或多个尾部的实参。通过为函数声明的形参列表中的形参使用初始化器来指定默认实参。
- 拥有默认实参的形参之后的形参必须拥有在这个或同一作用域中先前的声明中所提供的默认实参，除非该形参是从某个形参包展开得到的。
- 默认实参不是函数类型的一部分，不能函数指针、到函数的引用以及 `typedef` 声明中出现。

默认实参

- 允许调用函数时不提供一个或多个尾部的实参。通过为函数声明的形参列表中的形参使用初始化器来指定默认实参。
- 拥有默认实参的形参之后的形参必须拥有在这个或同一作用域中先前的声明中所提供的默认实参，除非该形参是从某个形参包展开得到的。
- 默认实参不是函数类型的一部分，不能函数指针、到函数的引用以及 `typedef` 声明中出现。
- 当在同一作用域中重声明非模板函数时，可以向已声明的函数添加默认实参。在函数调用点，可用的默认实参是由该函数所有可见的声明中所提供的默认实参的并集。重声明不能为已有可见默认值的实参引入默认值（即使值相同）。内层作用域中的重声明不从外层作用域获得默认实参。

默认实参

- 允许调用函数时不提供一个或多个尾部的实参。通过为函数声明的形参列表中的形参使用初始化器来指定默认实参。
- 拥有默认实参的形参之后的形参必须拥有在这个或同一作用域中先前的声明中所提供的默认实参，除非该形参是从某个形参包展开得到的。
- 默认实参不是函数类型的一部分，不能函数指针、到函数的引用以及 typedef 声明中出现。
- 当在同一作用域中重声明非模板函数时，可以向已声明的函数添加默认实参。在函数调用点，可用的默认实参是由该函数所有可见的声明中所提供的默认实参的并集。重声明不能为已有可见默认值的实参引入默认值（即使值相同）。内层作用域中的重声明不从外层作用域获得默认实参。
- 默认实参中使用的名字绑定于声明点，但函数调用点才执行默认实参的计算。因此若这个默认实参的值发生了改变，实际传入的参数也会相应地改变。

默认实参

- 对于非模板类的成员函数，类外的定义中允许出现默认实参，并与类体内的声明所提供的默认实参组合。如果类外的默认实参会使成员函数变成默认、复制或移动构造函数，那么程序非良构。

默认实参

- 对于非模板类的成员函数，类外的定义中允许出现默认实参，并与类体内的声明所提供的默认实参组合。如果类外的默认实参会使成员函数变成默认、复制或移动构造函数，那么程序非良构。
- 虚函数的覆盖函数不会从基类定义获得默认实参，而在进行虚函数调用时，默认实参根据对象的静态类型确定。

默认实参

- 对于非模板类的成员函数，类外的定义中允许出现默认实参，并与类体内的声明所提供的默认实参组合。如果类外的默认实参会使成员函数变成默认、复制或移动构造函数，那么程序非良构。
- 虚函数的覆盖函数不会从基类定义获得默认实参，而在进行虚函数调用时，默认实参根据对象的静态类型确定。
- 默认实参中不允许使用局部变量，不允许 `this` 指针，不允许使用非静态的类成员，不允许使用前面出现的函数形参。除了函数调用运算符外，运算符函数不能有默认参数。

默认实参

■ 示例:

```
1  int mod=1e9+7;
2  int inv(int a,int b=mod){
3      int x=1,y=0;const int m=b;
4      while(b){const int t=a/b,ly=y,lb=b;
5          y=x-t*y,x=ly,b=a-t*b,a=lb;}return(x+m)%m;
6  }void g(){
7      int a=inv(2,998244353),b=inv(4);
8      //可以传实参,不使用默认,也可以不传实参,使用默认
9      mod=1e9+9;int c=inv(4);//c=inv(4,1e9+9;
10     //int inv(int=3,int);//内层作用域不获得默认实参
11     int inv(int,int),d=inv(2,mod);//必须传实参
12     //int inv(int,int=a);//默认实参不能使用局部变量
13     int inv(int,int=998244353),e=inv(1);//重声明
14     int inv(int=3,int b),f=inv();//添加默认实参
15     //int inv(int=3,int=998244353);
16     //同一作用域内不能重新声明已经声明的默认实参
17 }
```


目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

- 默认实参

- **友元函数**

- 函数重载

- 运算符重载

- 函数对象

- 非静态成员函

数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

友元函数

- 友元声明在类体内出现，并向一个函数或另一个类授予对包含友元声明的类的私有及受保护成员的访问权。

友元函数

- 友元声明在类体内出现，并向一个函数或另一个类授予对包含友元声明的类的私有及受保护成员的访问权。
- 友元声明的形式为：

```
1 friend 函数声明 (1)
2 friend 函数定义 (2)
3 friend 详述类说明符 ; (3)
4 friend 简单类型说明符 ; (4) (C++11 起)
5 friend typename-说明符 ; (4) (C++11 起)
```

友元函数

- 友元声明在类体内出现，并向一个函数或另一个类授予对包含友元声明的类的私有及受保护成员的访问权。
- 友元声明的形式为：

```
1 friend 函数声明 (1)
2 friend 函数定义 (2)
3 friend 详述类说明符 ; (3)
4 friend 简单类型说明符 ; (4) (C++11 起)
5 friend typename-说明符 ; (4) (C++11 起)
```

- (1)指明一个或多个函数为此类的友元。友元函数可以访问该类的非 public 成员。

友元函数

- 友元声明在类体内出现，并向一个函数或另一个类授予对包含友元声明的类的私有及受保护成员的访问权。
- 友元声明的形式为：

```
1 friend 函数声明 (1)
2 friend 函数定义 (2)
3 friend 详述类说明符 ; (3)
4 friend 简单类型说明符 ; (4) (C++11 起)
5 friend typename-说明符 ; (4) (C++11 起)
```

- (1)指明一个或多个函数为此类的友元。友元函数可以访问该类的非 public 成员。
- (2)定义一个非成员函数，同时令其为此类的友元。只允许在非局部类中使用。

友元函数

- 友元声明在类体内出现，并向一个函数或另一个类授予对包含友元声明的类的私有及受保护成员的访问权。
- 友元声明的形式为：

```
1 friend 函数声明 (1)
2 friend 函数定义 (2)
3 friend 详述类说明符 ; (3)
4 friend 简单类型说明符 ; (4) (C++11 起)
5 friend typename-说明符 ; (4) (C++11 起)
```

- (1)指明一个或多个函数为此类的友元。友元函数可以访问该类的非 public 成员。
- (2)定义一个非成员函数，同时令其为此类的友元。只允许在非局部类中使用。
- (3)指定以详述类说明符指名的类类型为此类的友元。友元类的所有成员声明和定义都可以访问该类的非 public 成员。

友元函数

- (4)指定以简单类型说明符或 `typename`-说明符指名的类型，如果该类型是类类型，那么该类型是此类的友元，否则忽略。与(3)不同的是该声明不引入新类型，因此若友元的类类型还没声明，则需要前置声明。

友元函数

- (4)指定以简单类型说明符或 `typename`-说明符指名的类型，如果该类型是类类型，那么该类型是此类的友元，否则忽略。与(3)不同的是该声明不引入新类型，因此若友元的类类型还没声明，则需要前置声明。
- 友元关系不传递，友元关系不继承。

友元函数

- (4)指定以简单类型说明符或 typename-说明符指名的类型，如果该类型是类类型，那么该类型是此类的友元，否则忽略。与(3)不同的是该声明不引入新类型，因此若友元的类类型还没声明，则需要前置声明。
- 友元关系不传递，友元关系不继承。
- 友元函数声明中不允许使用存储类说明符。访问说明符对于友元声明的含义没有影响。友元类声明不能定义新的类。

友元函数

- (4)指定以简单类型说明符或 `typename`-说明符指名的类型，如果该类型是类类型，那么该类型是此类的友元，否则忽略。与(3)不同的是该声明不引入新类型，因此若友元的类类型还没声明，则需要前置声明。
- 友元关系不传递，友元关系不继承。
- 友元函数声明中不允许使用存储类说明符。访问说明符对于友元声明的含义没有影响。友元类声明不能定义新的类。
- 可以声明派生类为友元，此时派生类可访问基类的 `private` 成员。

友元函数

■ 示例:

```
1  class C; //前置声明
2  class A{
3      int x;
4      friend class B; friend C;
5      friend int getx(A a){return a.x;}
6  };
7  class B:A{
8      int y;
9      B():y(x){}
10     void add(){++x;}
11 };
12 class C{
13     A a; B b;
14     int getx()const{return a.x;}
15     //int gety()const{return b.y;} //友元不继承
16 };
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

- 默认实参
- 友元函数
- 函数重载
- 运算符重载
- 函数对象
- 非静态成员函

数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

函数重载

- 具有不同形参列表，cv 限定，引用声明符的函数可以具有相同的名字。这称之为函数重载。

函数重载

- 具有不同形参列表，cv 限定，引用声明符的函数可以具有相同的名字。这称之为函数重载。
- 进行函数调用和取函数地址时，会根据实参类型或目标所期待的函数指针、函数引用或成员函数指针类型来确定具体使用的重载函数。

函数重载

- 具有不同形参列表，cv 限定，引用声明符的函数可以具有相同的名字。这称之为函数重载。
- 进行函数调用和取函数地址时，会根据实参类型或目标所期待的函数指针、函数引用或成员函数指针类型来确定具体使用的重载函数。
- 根据实参类型确定时，会选择与实参类型最匹配的重载函数。这一步中，完全匹配优于提升优于转换。

函数重载

■ 示例:

```
1  int f(int){return 1;}
2  //char f(int){return 2;}//形参列表相同
3  int f(char){return 2;}
4  void g(){
5      f(0);//调用 f(int)
6      f('a');//调用 f(char)
7      f((unsigned char)'a');//调用 f(int) 而非 f(char)
8      //f(111);//非良构, 两者均为转换
9      int(*pf)(int)=f;//f(int)
10     int(&rf)(char)=f;//f(char)
11     //int(*pg)(unsigned char)=f;//不匹配
12 }
```


目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

- 默认实参
- 友元函数
- 函数重载
- 运算符重载
- 函数对象
- 非静态成员函

数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

运算符重载

- 重载的运算符是具有特殊的函数名的函数。函数名称为 `operator` 后随运算符，类型或双引号加后缀标识符（C++11 起）。

运算符重载

- 重载的运算符是具有特殊的函数名的函数。函数名称为 operator 后随运算符，类型或双引号加后缀标识符（C++11 起）。
- 不能重载运算符::（作用域解析）、.（成员访问）、.*（通过成员指针的成员访问）及?:（三元条件）。

运算符重载

- 重载的运算符是具有特殊的函数名的函数。函数名称为 operator 后随运算符，类型或双引号加后缀标识符（C++11 起）。
- 不能重载运算符::（作用域解析）、.（成员访问）、.*（通过成员指针的成员访问）及?:（三元条件）。
- 运算符 =（赋值）、()（函数调用）、[]（下标访问）及->（成员访问）只能重载为成员函数。

运算符重载

- 重载的运算符是具有特殊的函数名的函数。函数名称为 operator 后随运算符，类型或双引号加后缀标识符（C++11 起）。
- 不能重载运算符::（作用域解析）、.（成员访问）、.*（通过成员指针的成员访问）及?:（三元条件）。
- 运算符=（赋值）、()（函数调用）、[]（下标访问）及->（成员访问）只能重载为成员函数。
- 不能创建新运算符，不能更改运算符的优先级、结合方向或操作数的数量。运算符&& 与 || 的重载失去短路求值。

运算符重载

- 重载的运算符是具有特殊的函数名的函数。函数名称为 operator 后随运算符，类型或双引号加后缀标识符（C++11 起）。
- 不能重载运算符::（作用域解析）、.（成员访问）、.*（通过成员指针的成员访问）及?:（三元条件）。
- 运算符=（赋值）、()（函数调用）、[]（下标访问）及->（成员访问）只能重载为成员函数。
- 不能创建新运算符，不能更改运算符的优先级、结合方向或操作数的数量。运算符&& 与 || 的重载失去短路求值。
- 特别的，后置自增自减运算符的形参列表需要在最后增加一个 int，以与前置自增自减运算符区分。

运算符重载

- 重载的运算符是具有特殊的函数名的函数。函数名称为 operator 后随运算符，类型或双引号加后缀标识符（C++11 起）。
- 不能重载运算符::（作用域解析）、.（成员访问）、.*（通过成员指针的成员访问）及?:（三元条件）。
- 运算符=（赋值）、()（函数调用）、[]（下标访问）及->（成员访问）只能重载为成员函数。
- 不能创建新运算符，不能更改运算符的优先级、结合方向或操作数的数量。运算符&& 与 || 的重载失去短路求值。
- 特别的，后置自增自减运算符的形参列表需要在最后增加一个 int，以与前置自增自减运算符区分。
- 调用这种函数时，可以直接写函数名，也可以按该运算符原有的格式进行调用。

运算符重载

■ 示例:

```
1  const int mod=998244353;
2  struct mint{int x;
3      mint(int a):x(a){}mint(const mint&a):x(a.x){}
4      mint&operator=(const mint&b){x=b.x;return*this;}
5      operator int()const{return x;}//类型转换运算符
6      //int operator?:(int a,int b);//不能重载三元条件
7      mint&operator++(){x=x==mod?0:x+1;return*this;}
8      mint operator++(int){mint a=x;++*this;return a;}
9  };mint&operator+=(mint&a,const mint&b){
10     a.x+=b.x;if(a.x>=mod)a.x-=mod;return a;
11 }//赋值运算符必须是成员函数,复合赋值运算符可以不是
12 mint operator""_m(unsigned long long a){return a;}
13 //字面量运算符( C++11 起)
14 void g(){mint a=1_m;a++,++a,a+=a;int b=(int)a;
15     a.operator++(),a.operator++(0),operator+=(a,a);
16 }
```


目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

- 默认实参
- 友元函数
- 函数重载
- 运算符重载
- 函数对象
- 非静态成员函

数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

函数对象

- 函数对象（仿函数）类型是重载了函数调用运算符的类类型，这种类型的对象能用于函数调用式的表达式。

函数对象

- 函数对象（仿函数）类型是重载了函数调用运算符的类类型，这种类型的对象能用于函数调用式的表达式。
- 许多标准算法（如 `std::sort`）都接受函数对象以定制其行为。

函数对象

- 函数对象（仿函数）类型是重载了函数调用运算符的类类型，这种类型的对象能用于函数调用式的表达式。
- 许多标准算法（如 `std::sort`）都接受函数对象以定制其行为。
- Lambda 表达式(C++11 起)是纯右值表达式，它的类型是独有的无名非联合非聚合类类型，被称为闭包类型（closure type）。其格式为：

```
1 [ 捕获 ](形参) lambda说明符 约束(可选) { 函数体 }  
   (1)  
2 [ 捕获 ] { 函数体 } (2)
```

函数对象

- 函数对象（仿函数）类型是重载了函数调用运算符的类类型，这种类型的对象能用于函数调用式的表达式。
- 许多标准算法（如 `std::sort`）都接受函数对象以定制其行为。
- Lambda 表达式(C++11 起)是纯右值表达式，它的类型是独有的无名非联合非聚合类类型，被称为闭包类型（closure type）。其格式为：

```
1 [ 捕获 ](形参) lambda说明符 约束(可选) { 函数体 }  
   (1)  
2 [ 捕获 ] { 函数体 } (2)
```

- 该类型重载了形参列表为(1)中形参的函数调用运算符，是一种函数对象类型。

函数重载

■ 示例:

```
1 struct func{
2     int a,b,c;//用于计算二次函数  $y=ax^2+bx+c$ 
3     int operator()(int x){return c+(b+a*x)*x;}
4 };
5 func f1={1,1,1},f2={2,3,4};
6 int res1=f1(1),res2=f2(2);
7 //f1,f2 是仿函数, 可以像函数一样调用
8 void g(){
9     int a=1,b=2,c=5;
10    auto f3=[=](int x)->int{return c+(b+a*x)*x;};
11    a=2,b=3,c=4;
12    int res3=f3(7);//使用现在的 a,b,c 计算
13 }
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

- 默认实参
- 友元函数
- 函数重载
- 运算符重载
- 函数对象
- 非静态成员函数

- 构造函数
- 析构函数
- 虚函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

数

非静态成员函数

- 非静态成员函数非静态成员函数是在类的成员说明中不带 `static` 或 `friend` 说明符声明的函数。

非静态成员函数

- 非静态成员函数非静态成员函数是在类的成员说明中不带 `static` 或 `friend` 说明符声明的函数。
- 下列语法元素只能用于非静态成员函数：纯说明符，`cv` 限定符，引用限定符，`final` 与 `override` 说明符(C++11 起)，以及成员初始化器列表。

非静态成员函数

- 非静态成员函数非静态成员函数是在类的成员说明中不带 `static` 或 `friend` 说明符声明的函数。
- 下列语法元素只能用于非静态成员函数：纯说明符，`cv` 限定符，引用限定符，`final` 与 `override` 说明符(C++11 起)，以及成员初始化器列表。
- 非静态成员函数中，关键词 `this` 是一个纯右值表达式，它的值是隐式对象形参（即调用该函数的对象）的地址。

非静态成员函数

- 非静态成员函数非静态成员函数是在类的成员说明中不带 `static` 或 `friend` 说明符声明的函数。
- 下列语法元素只能用于非静态成员函数：纯说明符，`cv` 限定符，引用限定符，`final` 与 `override` 说明符(C++11 起)，以及成员初始化器列表。
- 非静态成员函数中，关键词 `this` 是一个纯右值表达式，它的值是隐式对象形参（即调用该函数的对象）的地址。
- 在 `X` 的非静态成员函数的体内，任何解析为 `X` 或 `X` 的某个基类的非类型非静态成员的标识符 `e` 均变换为 `(*this).e`（除非出现在成员调用运算符的第二操作数）。其余被解析为成员的无限定标识变换为对应的有限定标识。

非静态成员函数

- 非静态成员函数非静态成员函数是在类的成员说明中不带 `static` 或 `friend` 说明符声明的函数。
- 下列语法元素只能用于非静态成员函数：纯说明符，`cv` 限定符，引用限定符，`final` 与 `override` 说明符(C++11 起)，以及成员初始化器列表。
- 非静态成员函数中，关键词 `this` 是一个纯右值表达式，它的值是隐式对象形参（即调用该函数的对象）的地址。
- 在 `X` 的非静态成员函数的体内，任何解析为 `X` 或 `X` 的某个基类的非类型非静态成员的标识符 `e` 均变换为 `(*this).e`（除非出现在成员调用运算符的第二操作数）。其余被解析为成员的无限定标识变换为对应的有限定标识。
- `cv` 限定性用于限定 `this` 指针所指的对象。`const` 对象只能调用带有 `const` 限定的成员函数（`volatile` 同理）。`const` 成员函数不能调用非 `const` 限定的成员函数，不能修改非 `mutable` 成员对象。

构造函数

- 构造函数是类的一种特殊的非静态成员函数，用于初始化该类类型的对象。

构造函数

- 构造函数是类的一种特殊的非静态成员函数，用于初始化该类类型的对象。
- 声明构造函数时使用该类的名称，构造函数没有返回类型，不允许 `cv` 及引用限定符，不允许声明为 `static`。

构造函数

- 构造函数是类的一种特殊的非静态成员函数，用于初始化该类类型的对象。
- 声明构造函数时使用该类的名称，构造函数没有返回类型，不允许 `cv` 及引用限定符，不允许声明为 `static`。
- 任何构造函数的函数定义的函数体可以在复合语句的开花括号之前包含成员初始化器列表，其语法是冒号字符：后随一个或多个成员初始化器的逗号分隔列表

构造函数

- 构造函数是类的一种特殊的非静态成员函数，用于初始化该类类型的对象。
- 声明构造函数时使用该类的名称，构造函数没有返回类型，不允许 `cv` 及引用限定符，不允许声明为 `static`。
- 任何构造函数的函数定义的函数体可以在复合语句的开花括号之前包含成员初始化器列表，其语法是冒号字符：后随一个或多个成员初始化器的逗号分隔列表
- 构造函数没有名字且无法被直接调用它们在发生初始化时调用，且它们按照初始化的规则进行选择。

构造函数

- 没有 `explicit` 说明符（且可以用单个参数调用(C++11 前)）的构造函数是转换构造函数。

构造函数

- 没有 `explicit` 说明符（且可以用单个参数调用(C++11 前)）的构造函数是转换构造函数。
- 默认构造函数是形参列表为空的函数。复制构造函数是形参为该类类型只读左值引用的构造函数，移动构造函数是形参为该类类型右值引用的构造函数。

构造函数

- 没有 `explicit` 说明符（且可以用单个参数调用(C++11 前)）的构造函数是转换构造函数。
- 默认构造函数是形参列表为空的函数。复制构造函数是形参为该类类型只读左值引用的构造函数，移动构造函数是形参为该类类型右值引用的构造函数。
- 委托构造函数(C++11 起) 是初始化器列表出现类名作为被初始化目标的构造函数。这种情况下初始化器列表只允许出现这一项，调用被委托的构造函数。

析构函数

- 析构函数是对象生存期终结时调用的特殊成员函数。析构函数的目的是释放对象可能在其生存期间获得的资源。

析构函数

- 析构函数是对象生存期终结时调用的特殊成员函数。析构函数的目的是释放对象可能在其生存期间获得的资源。
- 析构函数的名字是 后随类名。析构函数的形参列表必须为空，没有返回类型，不允许 cv 及引用限定符，不允许声明为 static。

析构函数

- 析构函数是对象生存期终结时调用的特殊成员函数。析构函数的目的是释放对象可能在其生存期间获得的资源。
- 析构函数的名字是 后随类名。析构函数的形参列表必须为空，没有返回类型，不允许 cv 及引用限定符，不允许声明为 static。
- 每当对象的生存期结束时都会调用析构函数。对于用布置 new 或通过分配器成员函数，需要直接调用析构函数。

析构函数

- 析构函数是对象生存期终结时调用的特殊成员函数。析构函数的目的是释放对象可能在其生存期间获得的资源。
- 析构函数的名字是 后随类名。析构函数的形参列表必须为空，没有返回类型，不允许 cv 及引用限定符，不允许声明为 static。
- 每当对象的生存期结束时都会调用析构函数。对于用布置 new 或通过分配器成员函数，需要直接调用析构函数。
- 对于普通对象，直接调用析构函数会导致在作用域结束处再次调用析构函数时引发未定义行为。

虚函数

- 虚函数是其行为可以在派生类中覆盖的成员函数。

虚函数

- 虚函数是其行为可以在派生类中覆盖的成员函数。
- 对于非虚基类函数，派生类中声明（名称，形参列表，cv 限定符和引用限定符）相同的函数会隐藏基类函数。

虚函数

- 虚函数是其行为可以在派生类中覆盖的成员函数。
- 对于非虚基类函数，派生类中声明（名称，形参列表，cv 限定符和引用限定符）相同的函数会隐藏基类函数。
- 与重载和隐藏不同，覆盖是运行期行为，当使用到基类的指针或引用来处理派生类时，对被覆盖的虚函数的调用，将会调用定义于派生类中的行为。若通过指向基类的指针 delete 动态分配的多态类型对象，则基类析构函数必须为虚函数。

虚函数

- 虚函数是其行为可以在派生类中覆盖的成员函数。
- 对于非虚基类函数，派生类中声明（名称，形参列表，cv 限定符和引用限定符）相同的函数会隐藏基类函数。
- 与重载和隐藏不同，覆盖是运行期行为，当使用到基类的指针或引用来处理派生类时，对被覆盖的虚函数的调用，将会调用定义于派生类中的行为。若通过指向基类的指针 delete 动态分配的多态类型对象，则基类析构函数必须为虚函数。
- **virtual 函数说明符指定非静态成员函数为虚函数，它只能在该函数的首个声明中出现。**

虚函数

- 虚函数是其行为可以在派生类中覆盖的成员函数。
- 对于非虚基类函数，派生类中声明（名称，形参列表，cv 限定符和引用限定符）相同的函数会隐藏基类函数。
- 与重载和隐藏不同，覆盖是运行期行为，当使用到基类的指针或引用来处理派生类时，对被覆盖的虚函数的调用，将会调用定义于派生类中的行为。若通过指向基类的指针 delete 动态分配的多态类型对象，则基类析构函数必须为虚函数。
- virtual 函数说明符指定非静态成员函数为虚函数，它只能在该函数的首个声明中出现。
- 纯说明符 =0 指明虚函数为纯虚函数，拥有纯虚函数的类是抽象类，不能被实例化。可以为纯虚函数提供定义。

虚函数

- 虚函数是其行为可以在派生类中覆盖的成员函数。
- 对于非虚基类函数，派生类中声明（名称，形参列表，cv 限定符和引用限定符）相同的函数会隐藏基类函数。
- 与重载和隐藏不同，覆盖是运行期行为，当使用到基类的指针或引用来处理派生类时，对被覆盖的虚函数的调用，将会调用定义于派生类中的行为。若通过指向基类的指针 delete 动态分配的多态类型对象，则基类析构函数必须为虚函数。
- virtual 函数说明符指定非静态成员函数为虚函数，它只能在该函数的首个声明中出现。
- 纯说明符 =0 指明虚函数为纯虚函数，拥有纯虚函数的类是抽象类，不能被实例化。可以为纯虚函数提供定义。
- 虚说明符 override（C++11 起）指明函数覆盖基类虚函数，若不存在则程序非良构，常用于检查该函数是否正确覆盖基类虚函数。

虚函数

- 虚函数是其行为可以在派生类中覆盖的成员函数。
- 对于非虚基类函数，派生类中声明（名称，形参列表，cv 限定符和引用限定符）相同的函数会隐藏基类函数。
- 与重载和隐藏不同，覆盖是运行期行为，当使用到基类的指针或引用来处理派生类时，对被覆盖的虚函数的调用，将会调用定义于派生类中的行为。若通过指向基类的指针 delete 动态分配的多态类型对象，则基类析构函数必须为虚函数。
- virtual 函数说明符指定非静态成员函数为虚函数，它只能在该函数的首个声明中出现。
- 纯说明符 =0 指明虚函数为纯虚函数，拥有纯虚函数的类是抽象类，不能被实例化。可以为纯虚函数提供定义。
- 虚说明符 override（C++11 起）指明函数覆盖基类虚函数，若不存在则程序非良构，常用于检查该函数是否正确覆盖基类虚函数。
- 虚说明符 final（C++11 起）指明虚函数不能被覆盖，否则程序非良构。

非静态成员函数

■ 示例:

```
1  struct node{int x,y;
2      node(){}//默认构造函数
3      node(int a,int b):x(a),y(b){}//构造函数
4      virtual~node(){}//虚析构函数
5      static int f0(){return 0;}//静态成员函数
6      int getx()const{return x;}
7      //void addx()const{x++;}//this->x 不可修改
8      virtual int getz()const=0;//纯虚函数
9      virtual int isnode(){return 1;}//虚函数
10 };
11 //node a;//抽象类不可实例化
12 struct node2:node{int z;
13     int getz()const override{return z;}//覆盖
14     int isnode()override{return 0;}
15 }b;
16 node*p=&b;int res=p->isnode();//res=0
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 `static_assert`

17 预处理指令

18 常用 C 库

模板

- 模板是一个 C++ 实体，它定义以下其一：

模板

- 模板是一个 C++ 实体，它定义以下其一：

- 1 一族类（类模板），可以是嵌套类

模板

- 模板是一个 C++ 实体，它定义以下其一：
 - 1 一族类（类模板），可以是嵌套类
 - 2 一族函数（函数模板），可以是成员函数

模板

- 模板是一个 C++ 实体，它定义以下其一：
 - 1 一族类（类模板），可以是嵌套类
 - 2 一族函数（函数模板），可以是成员函数
 - 3 一族类型的别名（别名模板）（C++11 起）

模板

- 模板是一个 C++ 实体，它定义以下其一：
 - 1 一族类（类模板），可以是嵌套类
 - 2 一族函数（函数模板），可以是成员函数
 - 3 一族类型的别名（别名模板）（C++11 起）
- 模板声明的形式为：

1 `template < 形参列表 > 声明`

模板

- 模板是一个 C++ 实体，它定义以下其一：

- 1 一族类（类模板），可以是嵌套类
- 2 一族函数（函数模板），可以是成员函数
- 3 一族类型的别名（别名模板）（C++11 起）

- 模板声明的形式为：

1 `template < 形参列表 > 声明`

- 其中声明是类类型，成员枚举类型，函数声明，命名空间作用域的静态数据成员，或别名模板(C++11 起)的声明。

模板

- 模板是一个 C++ 实体，它定义以下其一：

- 1 一族类（类模板），可以是嵌套类
- 2 一族函数（函数模板），可以是成员函数
- 3 一族类型的别名（别名模板）（C++11 起）

- 模板声明的形式为：

1 `template < 形参列表 > 声明`

- 其中声明是类类型，成员枚举类型，函数声明，命名空间作用域的静态数据成员，或别名模板（C++11 起）的声明。
- 形参列表是非空的模板形参的逗号分隔列表，其中每项是非类型形参、类型形参、模板形参或任何这些的形参包（C++11 起）之一。

模板

- 模板是一个 C++ 实体，它定义以下其一：

- 1 一族类（类模板），可以是嵌套类
- 2 一族函数（函数模板），可以是成员函数
- 3 一族类型的别名（别名模板）（C++11 起）

- 模板声明的形式为：

1 `template < 形参列表 > 声明`

- 其中声明是类类型，成员枚举类型，函数声明，命名空间作用域的静态数据成员，或别名模板（C++11 起）的声明。
- 形参列表是非空的模板形参的逗号分隔列表，其中每项是非类型形参、类型形参、模板形参或任何这些的形参包（C++11 起）之一。
- 为使模板被实例化，它的每个模板形参（类型、非类型或模板）都必须被一个对应的模板实参替换。对于类模板，实参可以被显式提供，或为默认。对于函数模板，实参可以被显式提供，或从语境推导，或为默认。

模板

- 非类型模板形参必须是左值引用类型，整数类型，指针类型，成员指针类型，枚举类型，`std::nullptr_t` 之一。

模板

- 非类型模板形参必须是左值引用类型，整数类型，指针类型，成员指针类型，枚举类型，`std::nullptr_t` 之一。
- 数组与函数类型可以写在非类型模板形参中，但它们会被自动替换为适合的对象指针和函数指针。

模板

- 非类型模板形参必须是左值引用类型，整数类型，指针类型，成员指针类型，枚举类型，`std::nullptr_t` 之一。
- 数组与函数类型可以写在非类型模板形参中，但它们会被自动替换为适合的对象指针和函数指针。
- 在类模板体内使用的非类型模板形参的名字是不可修改的纯右值，除非它的类型是左值引用类型。

模板

- 非类型模板形参必须是左值引用类型，整数类型，指针类型，成员指针类型，枚举类型，`std::nullptr_t` 之一。
- 数组与函数类型可以写在非类型模板形参中，但它们会被自动替换为适合的对象指针和函数指针。
- 在类模板体内使用的非类型模板形参的名字是不可修改的纯右值，除非它的类型是左值引用类型。
- 在实例化拥有非类型模板形参的模板时，模板实参必须满足：

模板

- 非类型模板形参必须是左值引用类型，整数类型，指针类型，成员指针类型，枚举类型，`std::nullptr_t` 之一。
- 数组与函数类型可以写在非类型模板形参中，但它们会被自动替换为适合的对象指针和函数指针。
- 在类模板体内使用的非类型模板形参的名字是不可修改的纯右值，除非它的类型是左值引用类型。
- 在实例化拥有非类型模板形参的模板时，模板实参必须满足：
 - 1 对于整型和算术类型，必须是该类型的经转换常量表达式。

模板

- 非类型模板形参必须是左值引用类型，整数类型，指针类型，成员指针类型，枚举类型，`std::nullptr_t` 之一。
- 数组与函数类型可以写在非类型模板形参中，但它们会被自动替换为适合的对象指针和函数指针。
- 在类模板体内使用的非类型模板形参的名字是不可修改的纯右值，除非它的类型是左值引用类型。
- 在实例化拥有非类型模板形参的模板时，模板实参必须满足：
 - 1 对于整型和算术类型，必须是该类型的经转换常量表达式。
 - 2 对于对象指针，必须指定某个具有静态存储期和连接的完整对象的地址或求值为空指针的常量表达式。

模板

- 非类型模板形参必须是左值引用类型，整数类型，指针类型，成员指针类型，枚举类型，`std::nullptr_t` 之一。
- 数组与函数类型可以写在非类型模板形参中，但它们会被自动替换为适合的对象指针和函数指针。
- 在类模板体内使用的非类型模板形参的名字是不可修改的纯右值，除非它的类型是左值引用类型。
- 在实例化拥有非类型模板形参的模板时，模板实参必须满足：
 - 1 对于整型和算术类型，必须是该类型的经转换常量表达式。
 - 2 对于对象指针，必须指定某个具有静态存储期和连接的完整对象的地址或求值为空指针的常量表达式。
 - 3 对于函数指针，必须是指向具有连接的函数的指针或求值为空指针的常量表达式。

模板

- 非类型模板形参必须是左值引用类型，整数类型，指针类型，成员指针类型，枚举类型，`std::nullptr_t` 之一。
- 数组与函数类型可以写在非类型模板形参中，但它们会被自动替换为适合的对象指针和函数指针。
- 在类模板体内使用的非类型模板形参的名字是不可修改的纯右值，除非它的类型是左值引用类型。
- 在实例化拥有非类型模板形参的模板时，模板实参必须满足：
 - 1 对于整型和算术类型，必须是该类型的经转换常量表达式。
 - 2 对于对象指针，必须指定某个具有静态存储期和连接的完整对象的地址或求值为空指针的常量表达式。
 - 3 对于函数指针，必须是指向具有连接的函数的指针或求值为空指针的常量表达式。
 - 4 对于左值引用形参，实参必须具有连接。

模板

- 非类型模板形参必须是左值引用类型，整数类型，指针类型，成员指针类型，枚举类型，`std::nullptr_t` 之一。
- 数组与函数类型可以写在非类型模板形参中，但它们会被自动替换为适合的对象指针和函数指针。
- 在类模板体内使用的非类型模板形参的名字是不可修改的纯右值，除非它的类型是左值引用类型。
- 在实例化拥有非类型模板形参的模板时，模板实参必须满足：
 - 1 对于整型和算术类型，必须是该类型的经转换常量表达式。
 - 2 对于对象指针，必须指定某个具有静态存储期和连接的完整对象的地址或求值为空指针的常量表达式。
 - 3 对于函数指针，必须是指向具有连接的函数的指针或求值为空指针的常量表达式。
 - 4 对于左值引用形参，实参必须具有连接。
 - 5 对于成员指针，必须是表示成 `&Class::Member` 的成员指针或求值为空指针的常量表达式。

模板

- 类型模板形参使用类型形参关键词 `typename` 或 `class` 后随可选的名字组成，可以带有默认实参（规则与函数默认实参类似）。

模板

- 类型模板形参使用类型形参关键词 `typename` 或 `class` 后随可选的名字组成，可以带有默认实参（规则与函数默认实参类似）。
- 在模板声明体内，类型形参的名字是 `typedef` 名字，它是当模板被实例化时所提供的类型的别名。

模板

- 类型模板形参使用类型形参关键词 `typename` 或 `class` 后随可选的名字组成，可以带有默认实参（规则与函数默认实参类似）。
- 在模板声明体内，类型形参的名字是 `typedef` 名字，它是当模板被实例化时所提供的类型的别名。
- 类型模板形参的模板实参必须是类型标识，它可以指名不完整类型。

模板

- 类型模板形参使用类型形参关键词 `typename` 或 `class` 后随可选的名字组成，可以带有默认实参（规则与函数默认实参类似）。
- 在模板声明体内，类型形参的名字是 `typedef` 名字，它是当模板被实例化时所提供的类型的别名。
- 类型模板形参的模板实参必须是类型标识，它可以指名不完整类型。
- 如果实参可以同时被解释为类型标识和表达式，那么它始终会被解释为类型标识。

模板

- 模板模板形参使用 `template < 形参列表>class` 后随可选的名字组成，可以带有默认实参。

模板

- 模板模板形参使用 `template < 形参列表>class` 后随可选的名字组成，可以带有默认实参。
- 在模板声明体内，此形参的名字是一个模板名。

模板

- 模板模板形参使用 `template < 形参列表>class` 后随可选的名字组成，可以带有默认实参。
- 在模板声明体内，此形参的名字是一个模板名。
- 模板模板形参的模板实参是必须是一个标识表达式，它指名一个类模板或模板别名。

模板

- 模板模板形参使用 `template < 形参列表>class` 后随可选的名字组成，可以带有默认实参。
- 在模板声明体内，此形参的名字是一个模板名。
- 模板模板形参的模板实参是必须是一个标识表达式，它指名一个类模板或模板别名。
- 模板形参包（C++11 起）是在普通模板形参名字前加上...，函数参数包是包名加... 加包形参名的函数形参。

模板

- 模板模板形参使用 `template < 形参列表>class` 后随可选的名字组成，可以带有默认实参。
- 在模板声明体内，此形参的名字是一个模板名。
- 模板模板形参的模板实参是必须是一个标识表达式，它指名一个类模板或模板别名。
- 模板形参包（C++11 起）是在普通模板形参名字前加上...，函数参数包是包名加... 加包形参名的函数形参。
- 模式（其中至少带有一个形参包）后跟随...可以使形参包展开，展开为逗号分隔的列表。`sizeof...` 可以得到包的元素数量。

模板

- 模板模板形参使用 `template < 形参列表>class` 后随可选的名字组成，可以带有默认实参。
- 在模板声明体内，此形参的名字是一个模板名。
- 模板模板形参的模板实参是必须是一个标识表达式，它指名一个类模板或模板别名。
- 模板形参包（C++11 起）是在普通模板形参名字前加上...，函数参数包是包名加... 加包形参名的函数形参。
- 模式（其中至少带有一个形参包）后跟随...可以使形参包展开，展开为逗号分隔的列表。`sizeof...` 可以得到包的元素数量。
- 模板特化可以为特定的实参类别模板代码，格式为将模板声明的声明符后加实参列表。完全提供实参列表的特化称为全特化，否则为部分特化。只有类模板可以提供部分特化。

模板

■ 示例:

```
1  template<typename T=int,int sz=10>struct arr{
2      T v[sz];//T 是类型, mod 是整数常量
3      T&operator[](int b){return v[b];}
4      T operator[](int b)const{return v[b];}
5  };arr<>a;arr<char>b;arr<char,20>c;
6  //a.v 是 int[10] b.v 是 char[10] c.v 是 char[20]
7  template<int sz>struct arr<bool,sz>{//模板部分特化
8      unsigned long long v[(sz+63)/64];
9      bool operator[](int b)const
10         {return v[b>>6]>>(b&63)&1;}
11  };arr<bool,512>d;//d 使用特化的模板类
12  template<class T,int n>T sum(T(&a)[n]){
13      T s=0;for(int i=0;i<n;s+=a[i++]);return s;
14  }int res1=sum(a.v);//sum 接受可用 += 类型的数组
15  template<class...T>int getnum(T...v)
16  {return sizeof...(v);}//getnum 求传入实参的数量
17  int res2=getnum(1,1.0,arr<>(),d);//res2=4
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

命名空间

- 命名空间提供了一种在大项目中避免名字冲突的方法。

命名空间

- 命名空间提供了一种在大项目中避免名字冲突的方法。
- 在命名空间块内声明的符号被放入一个具名的作用域中，避免这些符号被误认为其他作用域中的同名符号。

命名空间

- 命名空间提供了一种在大项目中避免名字冲突的方法。
- 在命名空间块内声明的符号被放入一个具名的作用域中，避免这些符号被误认为其他作用域中的同名符号。
- 有关命名空间的语法如下：

```
1 namespace 命名空间名 { 声明序列 } (1)
2 inline namespace 命名空间名 { 声明序列 } (2)
3 namespace { 声明序列 } (3)
4 命名空间名::成员名 (4)
5 using namespace 命名空间名; (5)
6 using 命名空间名::成员名; (6)
7 namespace 名字 = 有限定命名空间 ; (7)
```


命名空间

- 命名空间提供了一种在大项目中避免名字冲突的方法。
- 在命名空间块内声明的符号被放入一个具名的作用域中，避免这些符号被误认为其他作用域中的同名符号。
- 有关命名空间的语法如下：

```
1 namespace 命名空间名 { 声明序列 } (1)
2 inline namespace 命名空间名 { 声明序列 } (2)
3 namespace { 声明序列 } (3)
4 命名空间名::成员名 (4)
5 using namespace 命名空间名; (5)
6 using 命名空间名::成员名; (6)
7 namespace 名字 = 有限定命名空间 ; (7)
```

- (1)为具名命名空间定义，其中的声明在外部不可见（对于无限定名字查找来说）。

命名空间

- 命名空间提供了一种在大项目中避免名字冲突的方法。
- 在命名空间块内声明的符号被放入一个具名的作用域中，避免这些符号被误认为其他作用域中的同名符号。
- 有关命名空间的语法如下：

```
1 namespace 命名空间名 { 声明序列 } (1)
2 inline namespace 命名空间名 { 声明序列 } (2)
3 namespace { 声明序列 } (3)
4 命名空间名::成员名 (4)
5 using namespace 命名空间名; (5)
6 using 命名空间名::成员名; (6)
7 namespace 名字 = 有限定命名空间 ; (7)
```

- (1)为具名命名空间定义，其中的声明在外部不可见（对于无限定名字查找来说）。
- (2)为内联命名空间定义（C++11 起），其中的声明在外围命名空间可见。

命名空间

- (3)为无名命名空间定义，其中的声明在其他翻译单元不可见

命名空间

- (3)为无名命名空间定义，其中的声明在其他翻译单元不可见
- (4)命名空间名可以出现在作用域解析运算符的左侧，可以通过这种方式来访问命名空间内声明的符号（有限定名字查找）。

命名空间

- (3)为无名命名空间定义，其中的声明在其他翻译单元不可见
- (4)命名空间名可以出现在作用域解析运算符的左侧，可以通过这种方式来访问命名空间内声明的符号（有限定名字查找）。
- (5)using 指令：从 using 指令之后到指令出现的作用域结尾为止，来自命名空间名的声明均可见。using 指令是块声明。

命名空间

- (3)为无名命名空间定义，其中的声明在其他翻译单元不可见
- (4)命名空间名可以出现在作用域解析运算符的左侧，可以通过这种方式来访问命名空间内声明的符号（有限定名字查找）。
- (5)using 指令：从 using 指令之后到指令出现的作用域结尾为止，来自命名空间名的声明均可见。using 指令是块声明。
- (6)using 声明：从 using 声明之后到声明出现的作用域结尾为止，来自命名空间命名空间名的符号成员名可见。using 声明是块声明。

命名空间

- (3)为无名命名空间定义，其中的声明在其他翻译单元不可见
- (4)命名空间名可以出现在作用域解析运算符的左侧，可以通过这种方式来访问命名空间内声明的符号（有限定名字查找）。
- (5)using 指令：从 using 指令之后到指令出现的作用域结尾为止，来自命名空间名的声明均可见。using 指令是块声明。
- (6)using 声明：从 using 声明之后到声明出现的作用域结尾为止，来自命名空间命名空间名的符号成员名可见。using 声明是块声明。
- (7)命名空间别名定义：令名字成为另一命名空间的同义词。

命名空间

- (3)为无名命名空间定义，其中的声明在其他翻译单元不可见
- (4)命名空间名可以出现在作用域解析运算符的左侧，可以通过这种方式来访问命名空间内声明的符号（有限定名字查找）。
- (5)using 指令：从 using 指令之后到指令出现的作用域结尾为止，来自命名空间名的声明均可见。using 指令是块声明。
- (6)using 声明：从 using 声明之后到声明出现的作用域结尾为止，来自命名空间命名空间名的符号成员名可见。using 声明是块声明。
- (7)命名空间别名定义：令名字成为另一命名空间的同义词。
- (5)(6)(7)是块声明，可以出现在块作用域中。

命名空间

- (3)为无名命名空间定义，其中的声明在其他翻译单元不可见
- (4)命名空间名可以出现在作用域解析运算符的左侧，可以通过这种方式来访问命名空间内声明的符号（有限定名字查找）。
- (5)using 指令：从 using 指令之后到指令出现的作用域结尾为止，来自命名空间名的声明均可见。using 指令是块声明。
- (6)using 声明：从 using 声明之后到声明出现的作用域结尾为止，来自命名空间命名空间名的符号成员名可见。using 声明是块声明。
- (7)命名空间别名定义：令名字成为另一命名空间的同义词。
- (5)(6)(7)是块声明，可以出现在块作用域中。
- 标准库声明的名字位于命名空间 `std` 内，通过 `using namespace std`；可以将其中的所有名字引入（但会增加命名冲突的风险）。

命名空间

■ 示例:

```
1  #include<iostream>
2  //已知该头文件在 std 中声明了 cin,cout
3  namespace f{
4      namespace g{//嵌套命名空间
5          int y;
6          namespace{int z;}//无名命名空间
7      }
8  }int main(){
9      int a;//cin<<a,cout>>a;//找不到 cin,cout
10     std::cin>>a,std::cout<<a;//有限定名字查找
11     using std::cout;cout<<a;//using 声明
12     using namespace std;//using 指令
13     int b;cin>>b,cout<<b;//直接使用
14     namespace fg=f::g;//命名空间别名
15     fg::z++,fg::y++;
16 }
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 `static_assert`

17 预处理指令

18 常用 C 库

属性

- `[[ 属性列表 ]]` (C++11 起) 可以为类型、对象、代码等引入由实现定义的属性。

属性

- [[属性列表]] (C++11 起) 可以为类型、对象、代码等引入由实现定义的属性。
- C++ 标准仅定义下列属性:

属性

- `[[ 属性列表 ]]` (C++11 起) 可以为类型、对象、代码等引入由实现定义的属性。
- C++ 标准仅定义下列属性：
 - 1 `[[noreturn]]` 指示函数不返回。

属性

- `[[ 属性列表 ]]` (C++11 起) 可以为类型、对象、代码等引入由实现定义的属性。
- C++ 标准仅定义下列属性：
 - 1 `[[noreturn]]` 指示函数不返回。
 - 2 `[[carries_dependency]]` 指示释放消费 `std::memory_order` 中的依赖链传入和传出该函数。

属性

- `[[ 属性列表]]` (C++11 起) 可以为类型、对象、代码等引入由实现定义的属性。
- C++ 标准仅定义下列属性：
 - 1 `[[noreturn]]` 指示函数不返回。
 - 2 `[[carries_dependency]]` 指示释放消费 `std::memory_order` 中的依赖链传入和传出该函数。
 - 3 `[[deprecated]]`(C++14) 或 `[[deprecated("原因")]]`(C++14) 指示允许使用声明有此属性的名称或实体，但因原因而不鼓励使用。

属性

- `[[ 属性列表 ]]` (C++11 起) 可以为类型、对象、代码等引入由实现定义的属性。
- C++ 标准仅定义下列属性：
 - 1 `[[noreturn]]` 指示函数不返回。
 - 2 `[[carries_dependency]]` 指示释放消费 `std::memory_order` 中的依赖链传入和传出该函数。
 - 3 `[[deprecated]]`(C++14) 或 `[[deprecated("原因")]]`(C++14) 指示允许使用声明有此属性的名称或实体，但因原因而不鼓励使用。
- `gcc` 中的属性可用作 `[[gnu::属性]]`。

属性

- `[[ 属性列表 ]]` (C++11 起) 可以为类型、对象、代码等引入由实现定义的属性。
- C++ 标准仅定义下列属性：
 - 1 `[[noreturn]]` 指示函数不返回。
 - 2 `[[carries_dependency]]` 指示释放消费 `std::memory_order` 中的依赖链传入和传出该函数。
 - 3 `[[deprecated]]`(C++14) 或 `[[deprecated("原因")]]`(C++14) 指示允许使用声明有此属性的名称或实体，但因原因而不鼓励使用。
- gcc 中的属性可用作 `[[gnu::属性]]`。
- 在 OI 中这些东西其实没什么用。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 `static_assert`

17 预处理指令

18 常用 C 库

static_assert

- `static_assert` 是块声明，进行编译时断言检查。

static_assert

- `static_assert` 是块声明，进行编译时断言检查。
- 格式为 `static_assert(布尔常量表达式, 字符串字面量)` 。

static_assert

- `static_assert` 是块声明，进行编译时断言检查。
- 格式为 `static_assert(布尔常量表达式, 字符串字面量)` 。
- 如果布尔常量表达式返回 `true`，那么该声明没有效果。否则将发布编译时错误，且诊断消息中会包含字符串字面量中的文本。

static_assert

- `static_assert` 是块声明，进行编译时断言检查。
- 格式为 `static_assert(布尔常量表达式, 字符串字面量)` 。
- 如果布尔常量表达式返回 `true`，那么该声明没有效果。否则将发布编译时错误，且诊断消息中会包含字符串字面量中的文本。
- `static_assert` 常用于在编译期禁止某些模板实例化。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 `static_assert`

17 预处理指令

- 文本宏替换
- 条件包含
- 文件包含
- 其它指令

18 常用 C 库

预处理指令

- 预处理器在编译前执行。预处理的结果是单个文件，接下来它会被传递给实际编译器。

预处理指令

- 预处理器在编译前执行。预处理的结果是单个文件，接下来它会被传递给实际编译器。
- 预处理指令控制预处理器的行为。每个指令占据一行并拥有下列格式：

1 # 预处理指令 实参

预处理指令

- 预处理器在编译前执行。预处理的结果是单个文件，接下来它会被传递给实际编译器。
- 预处理指令控制预处理器的行为。每个指令占据一行并拥有下列格式：

1 # 预处理指令 实参

- 允许空指令（即在# 后直接换行），而它没有效果。

预处理指令

- 预处理器在编译前执行。预处理的结果是单个文件，接下来它会被传递给实际编译器。
- 预处理指令控制预处理器的行为。每个指令占据一行并拥有下列格式：

1 # 预处理指令 实参

- 允许空指令（即在# 后直接换行），而它没有效果。
- 预处理指令不得来自宏展开。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 `static_assert`

17 预处理指令

- 文本宏替换
- 条件包含
- 文件包含
- 其它指令

18 常用 C 库

文本宏替换

■ 预处理器支持文本宏替换和仿函数文本宏替换，格式为：

- 1 `#define` 标识符 替换列表(可选) (1)
- 2 `#define` 标识符(形参) 替换列表(可选) (2)
- 3 `#define` 标识符(形参, ...) 替换列表(可选) (3) (C
++11 起)
- 4 `#define` 标识符(...) 替换列表(可选) (4) (C++11 起)
- 5 `#undef` 标识符 (5)

文本宏替换

■ 预处理器支持文本宏替换和仿函数文本宏替换，格式为：

- 1 `#define` 标识符 替换列表(可选) (1)
- 2 `#define` 标识符(形参) 替换列表(可选) (2)
- 3 `#define` 标识符(形参, ...) 替换列表(可选) (3) (C++11 起)
- 4 `#define` 标识符(...) 替换列表(可选) (4) (C++11 起)
- 5 `#undef` 标识符 (5)

■ (1) 定义仿对象宏 (object-like), 以替换列表替换每次出现的被定义标识符。

文本宏替换

- 预处理器支持文本宏替换和仿函数文本宏替换，格式为：

- 1 `#define` 标识符 替换列表(可选) (1)
- 2 `#define` 标识符(形参) 替换列表(可选) (2)
- 3 `#define` 标识符(形参, ...) 替换列表(可选) (3) (C++11 起)
- 4 `#define` 标识符(...) 替换列表(可选) (4) (C++11 起)
- 5 `#undef` 标识符 (5)

- (1) 定义仿对象宏 (object-like), 以替换列表替换每次出现的被定义标识符。
- (2)(3)(4) 定义仿函数宏 (function-like), 以替换列表替换每次出现的被定义标识符, 可选地接受一定量的实参, 它们随即替换掉替换列表中出现的任何对应的形参。如果标识符未使用函数写法, 即它自身之后没有括号, 那么它完全不被替换。

文本宏替换

- 预处理器支持文本宏替换和仿函数文本宏替换，格式为：

- 1 `#define` 标识符 替换列表(可选) (1)
- 2 `#define` 标识符(形参) 替换列表(可选) (2)
- 3 `#define` 标识符(形参, ...) 替换列表(可选) (3) (C++11 起)
- 4 `#define` 标识符(...) 替换列表(可选) (4) (C++11 起)
- 5 `#undef` 标识符 (5)

- (1) 定义仿对象宏 (object-like), 以替换列表替换每次出现的被定义标识符。
- (2)(3)(4) 定义仿函数宏 (function-like), 以替换列表替换每次出现的被定义标识符, 可选地接受一定量的实参, 它们随即替换掉替换列表中出现的任何对应的形参。如果标识符未使用函数写法, 即它自身之后没有括号, 那么它完全不被替换。
- (3)(4)具有可变数量实参, 可变实参可用 `__VA_ARGS__` 标识符访问。

文本宏替换

- 仿函数宏中，如果替换列表中一个标识符前有# 运算符，那么该标识符在运行形参替换的基础上以引号包围，实际上创建一个字符串字面量。自动添加转义字符，移除所有前导和尾随空白符，并将文本中间（除内嵌字符串字面量中间外）的任何空白符序列缩减成单个空格。此操作被称为“字符串化”。

文本宏替换

- 仿函数宏中，如果替换列表中一个标识符前有# 运算符，那么该标识符在运行形参替换的基础上以引号包围，实际上创建一个字符串字面量。自动添加转义字符，移除所有前导和尾随空白符，并将文本中间（除内嵌字符串字面量中间外）的任何空白符序列缩减成单个空格。此操作被称为“字符串化”。
- 如果替换列表中任何两个相继标识符之间有## 运算符，那么这两个标识符（首先未被宏展开）在运行形参替换的基础上将结果进行拼接。此操作被称为“拼接”或“记号粘贴”。不能通过拼接来创建注释/* 或//。

文本宏替换

- 仿函数宏中，如果替换列表中一个标识符前有# 运算符，那么该标识符在运行形参替换的基础上以引号包围，实际上创建一个字符串字面量。自动添加转义字符，移除所有前导和尾随空白符，并将文本中间（除内嵌字符串字面量中间外）的任何空白符序列缩减成单个空格。此操作被称为“字符串化”。
- 如果替换列表中任何两个相继标识符之间有## 运算符，那么这两个标识符（首先未被宏展开）在运行形参替换的基础上将结果进行拼接。此操作被称为“拼接”或“记号粘贴”。不能通过拼接来创建注释/* 或//。
- (5)取消#define 指令所作的标识符定义。如果标识符未关联到宏，那么忽略该指令。

文本宏替换

- 仿函数宏中，如果替换列表中一个标识符前有# 运算符，那么该标识符在运行形参替换的基础上以引号包围，实际上创建一个字符串字面量。自动添加转义字符，移除所有前导和尾随空白符，并将文本中间（除内嵌字符串字面量中间外）的任何空白符序列缩减成单个空格。此操作被称为“字符串化”。
- 如果替换列表中任何两个相继标识符之间有## 运算符，那么这两个标识符（首先未被宏展开）在运行形参替换的基础上将结果进行拼接。此操作被称为“拼接”或“记号粘贴”。不能通过拼接来创建注释/* 或//。
- (5)取消#define 指令所作的标识符定义。如果标识符未关联到宏，那么忽略该指令。
- 包含标准库头文件的翻译单元不可#define 或#undef 声明于任何标准库头文件中所声明的名字。

文本宏替换

- 仿函数宏中，如果替换列表中一个标识符前有# 运算符，那么该标识符在运行形参替换的基础上以引号包围，实际上创建一个字符串字面量。自动添加转义字符，移除所有前导和尾随空白符，并将文本中间（除内嵌字符串字面量中间外）的任何空白符序列缩减成单个空格。此操作被称为“字符串化”。
- 如果替换列表中任何两个相继标识符之间有## 运算符，那么这两个标识符（首先未被宏展开）在运行形参替换的基础上将结果进行拼接。此操作被称为“拼接”或“记号粘贴”。不能通过拼接来创建注释/* 或//。
- (5)取消#define 指令所作的标识符定义。如果标识符未关联到宏，那么忽略该指令。
- 包含标准库头文件的翻译单元不可#define 或#undef 声明于任何标准库头文件中所声明的名字。
- 使用标准库任何部分的翻译单元不可#define 或#undef 词法上等同于关键字的名称。否则行为未定义。

文本宏替换

- 下列宏名已预定义于每个翻译单元中:

文本宏替换

- 下列宏名已预定义于每个翻译单元中:

1 `__cplusplus`: 代表所用的 C++ 标准版本, 展开成值
199711L(C++11 前), 201103L(C++11), 201402L(C++14), 201703L(C++17), 或 202002L(C++20)

文本宏替换

- 下列宏名已预定义于每个翻译单元中:

- 1 `__cplusplus`: 代表所用的 C++ 标准版本, 展开成值 199711L(C++11 前), 201103L(C++11), 201402L(C++14), 201703L(C++17), 或 202002L(C++20)
- 2 `__STDC_HOSTED__`(C++11): 如果实现有宿主(在操作系统下运行)则展开成整数常量 1, 如果实现自立(不在操作系统下运行)则展开成 0

文本宏替换

- 下列宏名已预定义于每个翻译单元中:

- 1 `__cplusplus` : 代表所用的 C++ 标准版本, 展开成值 199711L(C++11 前), 201103L(C++11), 201402L(C++14), 201703L(C++17), 或 202002L(C++20)
- 2 `__STDC_HOSTED__`(C++11) : 如果实现有宿主(在操作系统下运行)则展开成整数常量 1, 如果实现自立(不在操作系统下运行)则展开成 0
- 3 `__FILE__` : 展开成当前文件名, 作为字符串字面量, 可用 `#line` 指令更改

文本宏替换

- 下列宏名已预定义于每个翻译单元中:

- 1 `__cplusplus`: 代表所用的 C++ 标准版本, 展开成值 199711L(C++11 前), 201103L(C++11), 201402L(C++14), 201703L(C++17), 或 202002L(C++20)
- 2 `__STDC_HOSTED__`(C++11): 如果实现有宿主(在操作系统下运行)则展开成整数常量 1, 如果实现自立(不在操作系统下运行)则展开成 0
- 3 `__FILE__`: 展开成当前文件名, 作为字符串字面量, 可用 `#line` 指令更改
- 4 `__DATE__`: 展开成翻译日期, 形式为 Mmm dd yyyy 的字符串。如果月中日期数小于 10 则 dd 的首字符为空格。

文本宏替换

■ 下列宏名已预定义于每个翻译单元中:

- 1 `__cplusplus` : 代表所用的 C++ 标准版本, 展开成值 199711L(C++11 前), 201103L(C++11), 201402L(C++14), 201703L(C++17), 或 202002L(C++20)
- 2 `__STDC_HOSTED__` (C++11) : 如果实现有宿主 (在操作系统下运行) 则展开成整数常量 1, 如果实现自立 (不在操作系统下运行) 则展开成 0
- 3 `__FILE__` : 展开成当前文件名, 作为字符串字面量, 可用 `#line` 指令更改
- 4 `__DATE__` : 展开成翻译日期, 形式为 `Mmm dd yyyy` 的字符串。如果月中日期数小于 10 则 `dd` 的首字符为空格。
- 5 `__TIME__` : 展开成翻译时间, 形式为 `hh:mm:ss` 的字符串字面量

文本宏替换

- 下列宏名已预定义于每个翻译单元中:

- 1 `__cplusplus` : 代表所用的 C++ 标准版本, 展开成值 199711L(C++11 前), 201103L(C++11), 201402L(C++14), 201703L(C++17), 或 202002L(C++20)
- 2 `__STDC_HOSTED__` (C++11) : 如果实现有宿主(在操作系统下运行)则展开成整数常量 1, 如果实现自立(不在操作系统下运行)则展开成 0
- 3 `__FILE__` : 展开成当前文件名, 作为字符串字面量, 可用 `#line` 指令更改
- 4 `__DATE__` : 展开成翻译日期, 形式为 `Mmm dd yyyy` 的字符串。如果月中日期数小于 10 则 `dd` 的首字符为空格。
- 5 `__TIME__` : 展开成翻译时间, 形式为 `hh:mm:ss` 的字符串字面量

- 试图重定义或取消定义这些宏会导致未定义行为。

文本宏替换

■ 示例:

```
1  #define ll long long//仿对象宏
2  #define text(...) #__VA_ARGS__
3  #define sqr(x) x*x
4  #define link(a,b)a##b
5  const char*s=text(sqr(1)+"3"/*aaa*/$#@%?);
6  //s="sqr(1)+\"3\" $#@%?"
7  ll x=sqr(3),sqr;//x=3*3,sqr 不带括号不替换
8  ll y=sqr(1+2);//y=1+2*1+2=5,宏只是简单的替换
9  link(1,1 z);//long long z
10 char sqr(*)d;//char *** d;
11 ll e=link(123,45611);//e=12345611
12 const char*f=__FILE__;//文件名
13 const char*now=__TIME__;//时间
14 ll line=__LINE__;//line=14
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 `static_assert`

17 预处理指令

- 文本宏替换

- 条件包含

- 文件包含

- 其它指令

18 常用 C 库

条件包含

- 预处理器支持有条件地编译源文件的某些部分，格式为：

```
1 #if 表达式
2 #ifdef 标识符
3 #ifndef 标识符
4 #elif 表达式
5 #else
6 #endif
```


条件包含

- 预处理器支持有条件地编译源文件的某些部分，格式为：

```
1 #if 表达式
2 #ifdef 标识符
3 #ifndef 标识符
4 #elif 表达式
5 #else
6 #endif
```

- 表达式是常量表达式。表达式可含有形如“defined 标识符”或“defined (标识符)”的一元运算符，当此标识符已经被定义为宏名时为 1 否则为 0。当表达式求值为非零值时，包含其所控制的代码块，否则跳过该代码块。

条件包含

- 预处理器支持有条件地编译源文件的某些部分，格式为：

```
1 #if 表达式
2 #ifdef 标识符
3 #ifndef 标识符
4 #elif 表达式
5 #else
6 #endif
```

- 表达式是常量表达式。表达式可含有形如“defined 标识符”或“defined (标识符)”的一元运算符，当此标识符已经被定义为宏名时为 1 否则为 0。当表达式求值为非零值时，包含其所控制的代码块，否则跳过该代码块。
- 在进行所有宏展开和 defined 及的求值后，任何非布尔字面量的标识符都被替换成数字 0。（这包含词法上为关键字的标识符）

条件包含

- 预处理器支持有条件地编译源文件的某些部分，格式为：

```
1  #if 表达式
2  #ifdef 标识符
3  #ifndef 标识符
4  #elif 表达式
5  #else
6  #endif
```

- 表达式是常量表达式。表达式可含有形如“defined 标识符”或“defined (标识符)”的一元运算符，当此标识符已经被定义为宏名时为 1 否则为 0。当表达式求值为非零值时，包含其所控制的代码块，否则跳过该代码块。
- 在进行所有宏展开和 defined 及的求值后，任何非布尔字面量的标识符都被替换成数字 0。（这包含词法上为关键字的标识符）
- “#ifdef 标识符”与“#if defined 标识符”实质上等价。
“#ifndef 标识符”与“#if !defined 标识符”实质上等价。

条件包含

■ 示例:

```
1  int g(int a,int b,int c){
2  #if __cplusplus<201103LL
3  //若标准低于 C++11 则使用以下代码
4      struct{int a,b,c;
5          int operator()(int x)const
6              {return c+(b+a*x)*x;}
7          }f={a,b,c};
8  #else//否则使用以下代码
9      auto f=[=](int x)->int{return c+(b+a*x)*x;};
10 #endif
11     return f(100);
12 }//计算 f(100) , 其中 f(x)=ax^2+bx+c
13 #if static_cast<bool>(1)
14 上面相当于 #if 0<0>(1) 即 #if 0
15 #elif defined wcr
16 被忽略的部分不需要能够通过编译
17 #endif
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

- 文本宏替换
- 条件包含
- 文件包含
- 其它指令

18 常用 C 库

文件包含

- 文件包含指令将其他源文件包含到当前源文件中紧随指令之后的一行，格式为：

```
1 #include < h字符序列 >(1)
2 #include " q字符序列 "(2)
3 #include 记号序列 (3)
```

文件包含

- 文件包含指令将其他源文件包含到当前源文件中紧随指令之后的一行，格式为：

```
1 #include < h字符序列 >(1)
2 #include " q字符序列 "(2)
3 #include 记号序列 (3)
```

- 其中 h 字符是源字符集除了换行符和> 以外的字符，q 字符是源字符集除了换行符和" 以外的字符。

文件包含

- 文件包含指令将其他源文件包含到当前源文件中紧随指令之后的一行，格式为：

```
1 #include < h字符序列 >(1)
2 #include " q字符序列 "(2)
3 #include 记号序列 (3)
```

- 其中 h 字符是源字符集除了换行符和> 以外的字符，q 字符是源字符集除了换行符和" 以外的字符。
- (1) 的意图是搜索由实现所掌控的文件。典型实现仅搜索标准包含目录。

文件包含

- 文件包含指令将其他源文件包含到当前源文件中紧随指令之后的一行，格式为：

```
1 #include < h字符序列 >(1)
2 #include " q字符序列 "(2)
3 #include 记号序列 (3)
```

- 其中 h 字符是源字符集除了换行符和> 以外的字符，q 字符是源字符集除了换行符和" 以外的字符。
- (1) 的意图是搜索由实现所掌控的文件。典型实现仅搜索标准包含目录。
- (2) 的意图是搜索不被实现所掌控的文件。典型实现首先于当前文件所在的目录搜索，然后退回使用语法(1)。

文件包含

- 文件包含指令将其他源文件包含到当前源文件中紧随指令之后的一行，格式为：

```
1 #include < h字符序列 >(1)
2 #include " q字符序列 "(2)
3 #include 记号序列 (3)
```

- 其中 h 字符是源字符集除了换行符和> 以外的字符，q 字符是源字符集除了换行符和" 以外的字符。
- (1) 的意图是搜索由实现所掌控的文件。典型实现仅搜索标准包含目录。
- (2) 的意图是搜索不被实现所掌控的文件。典型实现首先于当前文件所在的目录搜索，然后退回使用语法(1)。
- (3) 如果(1) 和(2) 都不匹配，记号序列会经历宏替换。该指令在替换后会再次尝试匹配(1) 和(2)。如果还是不匹配前面两种语法，那么行为未定义。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 `static_assert`

17 预处理指令

- 文本宏替换
- 条件包含
- 文件包含
- 其它指令

18 常用 C 库

其它指令

■ 还有格式如下的预处理指令：

- 1 #error 错误消息 (1)
- 2 #pragma 语用形参 (2)
- 3 #line 行号 "文件名"(可选) (3)

其它指令

- 还有格式如下的预处理指令：

```
1      #error  错误消息 (1)
2      #pragma 语用形参 (2)
3      #line  行号 "文件名"(可选) (3)
```

- (1)显示给定的错误消息并使程序非良构。与它作用相似的一种常用的非标准扩展是#warning 指令，它在编译期间放出一条用户定义的消息。

其它指令

- 还有格式如下的预处理指令：

```
1      #error  错误消息 (1)
2      #pragma 语用形参 (2)
3      #line  行号 "文件名"(可选) (3)
```

- (1)显示给定的错误消息并使程序非良构。与它作用相似的一种常用的非标准扩展是#warning 指令，它在编译期间放出一条用户定义的消息。
- (2)控制实现定义的行为，一般用于手动开启优化。（有些比赛时禁用）

其它指令

- 还有格式如下的预处理指令：

```
1      #error  错误消息 (1)
2      #pragma 语用形参 (2)
3      #line  行号 "文件名"(可选) (3)
```

- (1)显示给定的错误消息并使程序非良构。与它作用相似的一种常用的非标准扩展是#warning 指令，它在编译期间放出一条用户定义的消息。
- (2)控制实现定义的行为，一般用于手动开启优化。（有些比赛时禁用）
- (3)修改__LINE__ 和__FILE__ 的值，宏__LINE__ 在该点后的展开将产生行号加上自此遇到的实际代码行数。若行号为0 或大于32767 (C++11 前)2147483647 (C++11 起)，则行为未定义。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

- <cassert>
- <cctype>
- <cfloat>
- <climits>
- <cmath>
- <csetjmp>
- <cstdarg>
- <cstddef>
- <stdio>
- <stdlib>
- <string>
- <ctime>

常用 C 库

- 对于一些形式为 `xxx.h` 的 C 标准库头文件，C++ 标准库同时包含同名的头文件和拥有形式 `cxxx` 的头文件。

常用 C 库

- 对于一些形式为 `xxx.h` 的 C 标准库头文件，C++ 标准库同时包含同名的头文件和拥有形式 `cxxx` 的头文件。
- `cxxx` 的头文件将其中的名字放在 `std` 命名空间中，同时 C++ 允许这些头文件也在全局命名空间中声明同样的名字。

常用 C 库

- 对于一些形式为 `xxx.h` 的 C 标准库头文件，C++ 标准库同时包含同名的头文件和拥有形式 `cxxx` 的头文件。
- `cxxx` 的头文件将其中的名字放在 `std` 命名空间中，同时 C++ 允许这些头文件也在全局命名空间中声明同样的名字。
- 下面列出一些 OI 中用得到的 C 库，以及其介绍。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

■ <cassert>

■ <cctype>

■ <cfloating>

■ <climits>

■ <cmath>

■ <csetjmp>

■ <cstdarg>

■ <cstddef>

■ <cstdio>

■ <cstdlib>

■ <cstring>

■ <ctime>

<cassert>

- `assert` 是头文件<cassert> 中定义的唯一的一个宏，除此之外没有其他东西。

<cassert>

- `assert` 是头文件<cassert> 中定义的唯一的一个宏，除此之外没有其他东西。
- 用法 `assert(条件)` 若条件为假则中断程序。可以在发行构建中禁用。

<cassert>

- `assert` 是头文件<cassert> 中定义的唯一的一个宏，除此之外没有其他东西。
- 用法 `assert(条件)` 若条件为假则中断程序。可以在发行构建中禁用。
- 常用于调试。

<cassert>

■ 示例:

```
1 #include<cassert>
2 int inv(int a,int b){
3     int x=1,y=0;const int m=b,n=a;
4     while(b){const int t=a/b,ly=y,lb=b;
5         y=x-t*y,x=ly,b=a-t*b,a=lb;}int res=(x+m)%m;
6     assert(res*n%m==1);return res;
7 }//用 assert 判断求逆元程序是否写挂
```


目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

■ <cassert>

■ <cctype>

■ <cfloating>

■ <climits>

■ <cmath>

■ <csetjmp>

■ <cstdarg>

■ <cstddef>

■ <cstdio>

■ <cstdlib>

■ <cstring>

■ <ctime>

<cctype>

- <cctype>提供了一些函数，可用于测试和映射字符。

<cctype>

- <cctype>提供了一些函数，可用于测试和映射字符。
- 这些函数接受 `int` 作为参数，它的值必须是 `EOF` 或表示为一个无符号字符，否则行为未定义。

<cctype>

- <cctype>提供了一些函数，可用于测试和映射字符。
- 这些函数接受 `int` 作为参数，它的值必须是 `EOF` 或表示为一个无符号字符，否则行为未定义。
- 常用的函数有 `isdigit` 检查字符是否为数字，`isspace` 检查字符是否为空白间隔字符，`tolower` 转换字符为小写，`toupper` 转换字符为大写。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

■ <cassert>

■ <cctype>

■ <cfloating>

■ <climits>

■ <cmath>

■ <csetjmp>

■ <cstdint>

■ <stddef>

■ <stdio>

■ <stdlib>

■ <string>

■ <ctime>

<float>

- <float>提供了一系列与浮点值相关的依赖于平台的宏常量。

<float>

- <float>提供了一系列与浮点值相关的依赖于平台的宏常量。
- 如 FLT_MIN , DBL_MIN , LDBL_MIN 分别为 float、double 及 long double 最小规格化正数值。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

■ <cassert>

■ <cctype>

■ <cmath>

■ <climits>

■ <cmath>

■ <csetjmp>

■ <cstdarg>

■ <cstddef>

■ <cstdio>

■ <cstdlib>

■ <cstring>

■ <ctime>

<climits>

- <climits>提供了一系列决定了各种变量类型的各种属性，这些限制指定了变量不能存储任何超出这些限制的值。

<climits>

- <climits>提供了一系列决定了各种变量类型的各种属性，这些限制指定了变量不能存储任何超出这些限制的值。
- 如 INT_MIN , LONG_MIN , LLONG_MIN 分别为 int、long 及 long long 的最小值。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

- <cassert>
- <cctype>
- <cfloating>
- <climits>
- <cmath>
- <csetjmp>
- <cstdint>
- <cstdlib>
- <cstdio>
- <stdlib.h>
- <string>
- <ctime>

<cmath>

- <cmath>定义了各种数学函数（包括基本运算，指数函数，幂函数，三角函数，双曲函数，误差及伽马函数，临近整数的浮点运算，浮点数操作函数）。

<cmath>

- <cmath>定义了各种数学函数（包括基本运算，指数函数，幂函数，三角函数，双曲函数，误差及伽马函数，临近整数的浮点运算，浮点数操作函数）。
- 这些函数接受 `double` 作为参数，如果要用 `float` 或 `long double` 版本，需要在函数名称后加 `f` 或 `l`。

<cmath>

■ 示例:

```
1  #include<cmath>
2  const long double pi=acosl(-1);//用  $\cos^{-1}(-1)$  求出  $\pi$ 
3  double a=1.0,b=0.7,c=fmod(a,b);//浮点数取余
4  const int N=1e6+7,S=sqrt(N);//求出 n 的算术平方根
5  struct point{
6      long double x,y;
7      long double get()const{return atan2l(x,y);}
8      friend int cmp(const point&a,const point&b){
9          return a.get()<b.get();
10         //使用 atan2 做按极角排序的比较函数
11     }
12 };
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

■ <cassert>

■ <cctype>

■ <cfloating>

■ <climits>

■ <cmath>

■ <csetjmp>

■ <cstdint>

■ <stddef>

■ <stdio>

■ <stdlib>

■ <string>

■ <ctime>

<csetjmp>

- <csetjmp>定义了类型 jmp_buf , 仿函数宏 setjmp , 函数 longjmp。

<csetjmp>

- <csetjmp>定义了类型 jmp_buf , 仿函数宏 setjmp , 函数 longjmp。
- 一般来说 OI 中用不到。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

- <cassert>
- <cctype>
- <cfloating>
- <climits>
- <cmath>
- <csetjmp>
- <cstdint>
- <stddef>
- <stdio>
- <stdlib>
- <string>
- <ctime>

<stdarg>

- <stdarg>提供对 C 风格变参函数的支持。

<cstdarg>

- <cstdarg>提供对 C 风格变参函数的支持。
- <cstdarg>定义了类型 `va_list` , 宏 `va_start` , `va_arg` , `va_end`。

<cstdarg>

- <cstdarg>提供对 C 风格变参函数的支持。
- <cstdarg>定义了类型 `va_list` , 宏 `va_start` , `va_arg` , `va_end`。
- 一般来说 OI 中不是很常用。

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

- <cassert>
- <cctype>
- <cfloating>
- <climits>
- <cmath>
- <csetjmp>
- <cstdarg>
- <cstddef>
- <cstdio>
- <cstdlib>
- <cstring>
- <ctime>

<stddef>

- <stddef>定义了宏 NULL (C 风格空指针常量), 仿函数宏
offsetof (从标准布局类型的起始到其指定成员的字节偏移量)。

<stddef>

- <stddef>定义了宏 NULL (C 风格空指针常量), 仿函数宏 offsetof (从标准布局类型的起始到其指定成员的字节偏移量)。
- <stddef>定义了类型 size_t (sizeof 运算符返回的无符号整数类型), ptrdiff_t (在两个指针相减时返回的有符号整数类型), nullptr_t (C++11) (空指针字面量 nullptr 的类型), max_align_t (C++11) (具有不小于任何基础类型的内存对齐需求的平凡类型)

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

- <cassert>
- <cctype>
- <cfloating>
- <climits>
- <cmath>
- <csetjmp>
- <cstdint>
- <cstdlib>
- <cstring>
- <ctime>

<stdio>

- <stdio>提供了对 C 风格输入输出的支持。

<stdio>

- <stdio>提供了对 C 风格输入输出的支持。
- <stdio>定义了类型 FILE (对象类型, 不可复制, 足以保有控制 C I/O 流所需的全部信息), fpos_t (完整非数组对象类型, 足以唯一指定文件中的位置), size_t (sizeof 运算符返回的无符号整数类型)

<stdio>

- <stdio>提供了对 C 风格输入输出的支持。
- <stdio>定义了类型 FILE (对象类型, 不可复制, 足以保有控制 C I/O 流所需的全部信息), fpos_t (完整非数组对象类型, 足以唯一指定文件中的位置), size_t (sizeof 运算符返回的无符号整数类型)
- <stdio>提供了包括常量 NULL, EOF , FILE* 类型表达式 stdin, stdout, stderr 在内的宏。

<stdio>

- <stdio>提供了对 C 风格输入输出的支持。
- <stdio>定义了类型 FILE (对象类型, 不可复制, 足以保有控制 C I/O 流所需的全部信息), fpos_t (完整非数组对象类型, 足以唯一指定文件中的位置), size_t (sizeof 运算符返回的无符号整数类型)
- <stdio>提供了包括常量 NULL, EOF , FILE* 类型表达式 stdin, stdout, stderr 在内的宏。
- <stdio>提供了包括文件访问, 直接输入/输出, 无格式输入/输出, 有格式输入/输出, 文件寻位, 错误处理, 文件操作在内的函数。

<stdio>

- <stdio>提供了对 C 风格输入输出的支持。
- <stdio>定义了类型 FILE (对象类型, 不可复制, 足以保有控制 C I/O 流所需的全部信息), fpos_t (完整非数组对象类型, 足以唯一指定文件中的位置), size_t (sizeof 运算符返回的无符号整数类型)
- <stdio>提供了包括常量 NULL, EOF , FILE* 类型表达式 stdin, stdout, stderr 在内的宏。
- <stdio>提供了包括文件访问, 直接输入/输出, 无格式输入/输出, 有格式输入/输出, 文件寻位, 错误处理, 文件操作在内的函数。
- 常用的函数有文件访问 (freopen, fclose, fflush), 直接输入/输出 (fread, fwrite), 无格式输入/输出 (getchar, putchar, puts), 有格式输入/输出 (scanf, printf 系列) 等。

<stdio>

■ 示例:

```
1  #include<stdio>
2  const int iz=1<<21,oz=1<<22;
3  char is[iz],*i1=is,*i2=is,os[oz],*o1=os,*o2=os+oz;
4  int fgc(){return i1==i2&&is==(i2=is+
5  fread(i1=is,1,iz,stdin))?-1:*i1++;} //fread 快读
6  void fout(){fwrite(os,o1-os,1,stdout),o1=os;}
7  int fpc(int c){if(o1==o2)fout();return*o1++=c;}
8  int main(){freopen("a.in","r",stdin);
9      freopen("a.out","w",stdout); //freopen 重定向
10     int a;long long b;float c;double d;
11     long double e;char s[122];
12     scanf("%d%lld%f%Lf%100s",&a,&b,&c,&d,&e,s);
13     printf("%d%lld%f%Lf%s",a,b,c,d,e,s);
14     puts(s),putchar(getchar()); //无格式读入输出
15     char ch=fgc();fpc(ch),fout(); //读入
16     fclose(stdin),fclose(stdout); //fclose 关闭文件流
17 }
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

- <cassert>
- <cctype>
- <cfloating>
- <climits>
- <cmath>
- <csetjmp>
- <cstdint>
- <stddef>
- <stdio>
- <stdlib>
- <string>
- <ctime>

<cstdlib>

- <cstdlib>提供了各种通用工具。

<cstdlib>

- <cstdlib>提供了各种通用工具。
- <cstdlib>定义了 NULL , RAND_MAX (rand 函数返回的最大值) 等宏常量。

<cstdlib>

- <cstdlib>提供了各种通用工具。
- <cstdlib>定义了 NULL , RAND_MAX (rand 函数返回的最大值) 等宏常量。
- <cstdlib>定义了类型 size_t, div_t, ldiv_t, lldiv_t(后三者分别为 div, ldiv, lldiv 的返回类型))。

<cstdlib>

- <cstdlib>提供了各种通用工具。
- <cstdlib>定义了 NULL , RAND_MAX (rand 函数返回的最大值) 等宏常量。
- <cstdlib>定义了类型 size_t, div_t, ldiv_t, lldiv_t(后三者分别为 div, ldiv, lldiv 的返回类型)。
- <cstdlib>提供了包括进程控制，内存管理，数值字符串转换，宽字符串操纵，杂项算法与数学在内的函数。

<cstdlib>

- <cstdlib>提供了各种通用工具。
- <cstdlib>定义了 NULL , RAND_MAX (rand 函数返回的最大值) 等宏常量。
- <cstdlib>定义了类型 size_t, div_t, ldiv_t, lldiv_t(后三者分别为 div, ldiv, lldiv 的返回类型)。
- <cstdlib>提供了包括进程控制，内存管理，数值字符串转换，宽字符串操纵，杂项算法与数学在内的函数。
- 常用的函数有进程控制 (exit, atexit, system) , 内存管理 (malloc, calloc, realloc, free) , 数值字符串转换 (atoi, strtou 系列)，杂项算法与数学 (rand, srand, abs, div)。

<cstdlib>

■ 示例:

```
1  #include<cstdlib>
2  #include<stdio>
3  void g(){puts("g");} //程序结束处调用 g
4  [[noreturn]] void f(){
5      int a=32,b=14;
6      div_t c=div(a,b);
7      printf("%d %d\n",c.quot,c.rem);
8      exit(0); //结束程序, 相当于在 main 函数内返回 0
9  } int main(){
10     atexit(g); //程序结束处调用 g
11     srand(114514); //设置随机数种子
12     int a=(unsigned)rand()*RAND_MAX+rand();
13     //生成随机数
14     int*s=(int*)malloc(sizeof(int)*32);
15     s[2]=0,s[3]=rand()+a, free(s); //malloc 搭配 free
16     f();
17 }
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

- <cassert>
- <cctype>
- <cfloating>
- <climits>
- <cmath>
- <csetjmp>
- <cstdint>
- <stddef>
- <stdio>
- <stdlib>
- <string>
- <ctime>

<cstring>

- <cstring>提供了对 C 风格的空终止字符串及字符数组进行操作的函数。

<cstring>

- <cstring>提供了对 C 风格的空终止字符串及字符数组进行操作的函数。
- <cstring>定义了 `size_t` , `NULL`。

<cstring>

- <cstring>提供了对 C 风格的空终止字符串及字符数组进行操作的函数。
- <cstring>定义了 `size_t` , `NULL`。
- <cstring>提供了 `str` 系列函数与 `mem` 系列函数, 分别用于操作空终止字符串和字符数组。后者也常用于操作可平凡复制的对象。

<cstdlib>

■ 示例:

```
1  #include<cstring>
2  void g(){
3      int a[122],b[122];
4      memset(a,0,sizeof a);//可以用 memset 清零
5      a[0]++,a[1]++;
6      memcpy(b,a,sizeof a);//可以用 memcpy 复制
7      memmove(a+1,a,sizeof(int)*8);
8      //目标与原数组重叠时必须改用 memmove
9      char s[]="1234567",t[9];
10     int n=strlen(s);//获取长度, 为 7
11     strcpy(t+1,s),t[0]=1;//将 s[i] 复制到 t[i+1]
12     int res=strcmp(s,t);//res 是正数
13     //strcmp 比较 s,t 的字典序
14     //s 小则小于 0, t 小则大于 0, 否则等于 0
15 }
```

目录

1 注释

2 标识符

3 关键字

4 声明

5 对象

6 引用

7 变量

8 字面量

9 类型

10 表达式

11 语句

12 函数

13 模板

14 命名空间

15 属性

16 static_assert

17 预处理指令

18 常用 C 库

- <cassert>
- <cctype>
- <cfloating>
- <climits>
- <cmath>
- <csetjmp>
- <cstdarg>
- <cstddef>
- <cstdio>
- <cstdlib>
- <cstring>
- <ctime>

<ctime>

- <ctime>提供了各种操作日期和时间的函数。

<ctime>

- <ctime>提供了各种操作日期和时间的函数。
- <ctime>定义了类型 `size_t`, `clock_t` (`clock` 函数返回类型), `time_t` (`time` 函数返回类型), `tm` (用来保存时间和日期的结构)。

<ctime>

- <ctime>提供了各种操作日期和时间的函数。
- <ctime>定义了类型 `size_t`, `clock_t` (`clock` 函数返回类型), `time_t` (`time` 函数返回类型), `tm` (用来保存时间和日期的结构)。
- <ctime>定义了宏常量 `NULL`, `CLOCKS_PER_SEC` (每秒的处理器时钟个数)。

<ctime>

- <ctime>提供了各种操作日期和时间的函数。
- <ctime>定义了类型 `size_t`, `clock_t` (`clock` 函数返回类型), `time_t` (`time` 函数返回类型), `tm` (用来保存时间和日期的结构)。
- <ctime>定义了宏常量 `NULL`, `CLOCKS_PER_SEC` (每秒的处理器时钟个数)。
- <ctime>提供用于时间操作, 格式转换的函数。常用的函数是 `time`, `clock`。

<ctime>

■ 示例:

```
1  #include<cstdlib>
2  #include<ctime>
3  void g(){
4      srand(time(0));//以时间作为随机数种子
5      while(clock()<=CLOCKS_PER_SEC*9/10){
6          //这里写一些随机化算法
7      }
8  }//在不超过 1s 的时间内跑尽量多的次数
```